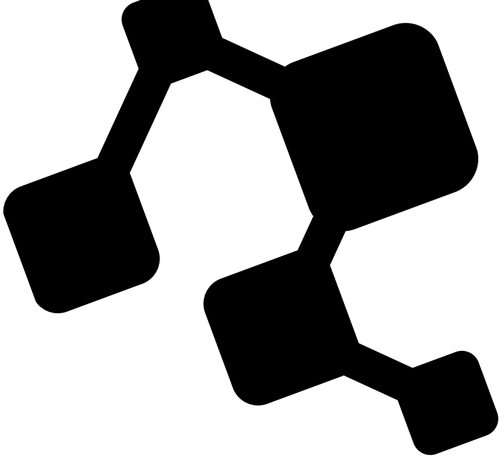
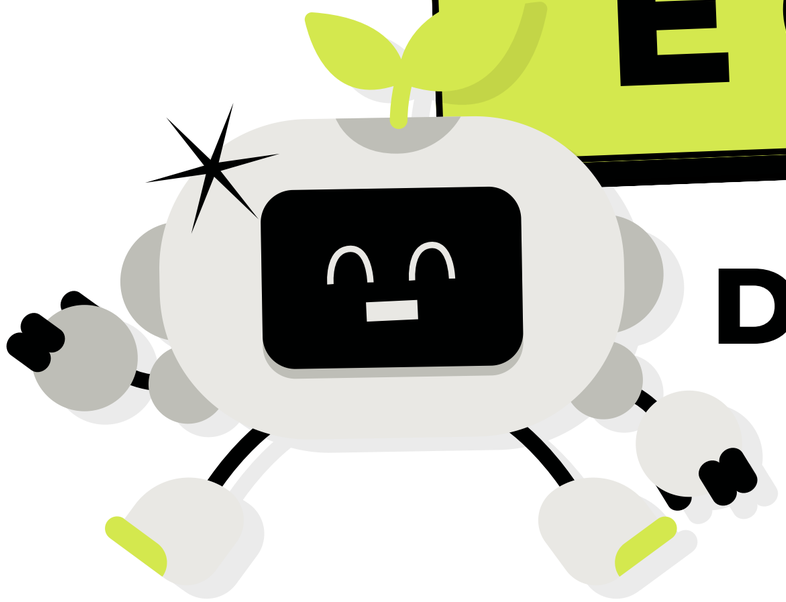
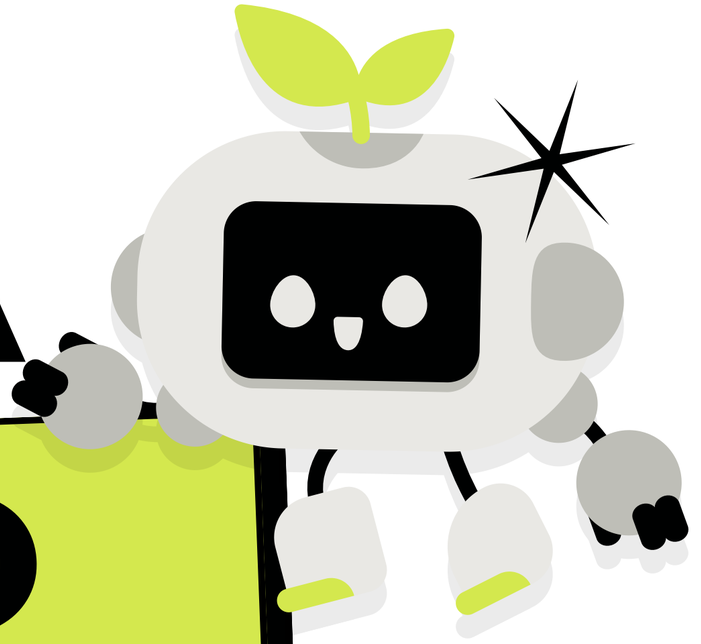
# AGENTES DE IA

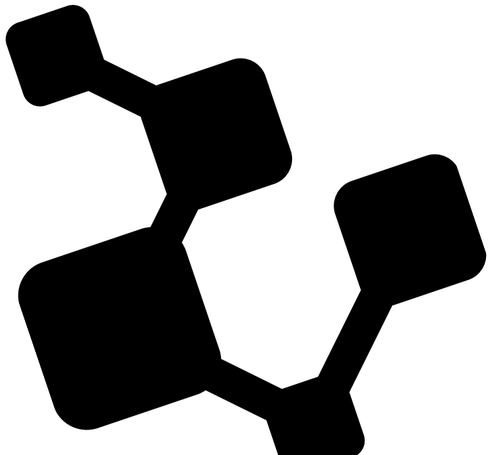
# E O FUTURO

# DA PROGRAMAÇÃO









## QUEM SOU EU?

# JOHNATHAN FERCHER

**Engenheiro de Software**; Tecnólogo em Tecnologia da Informação pela **FAETERJ-Petrópolis**; Mestre em Robótica pelo **IME**; Experiência com várias linguagens de programação, como: C, C++, C#, Rust, Python, Kotlin, **Go**;
**Carreira:** Braspag (Cielo), Mercado Livre, Uber, Gympass, **Kake / Fox**;

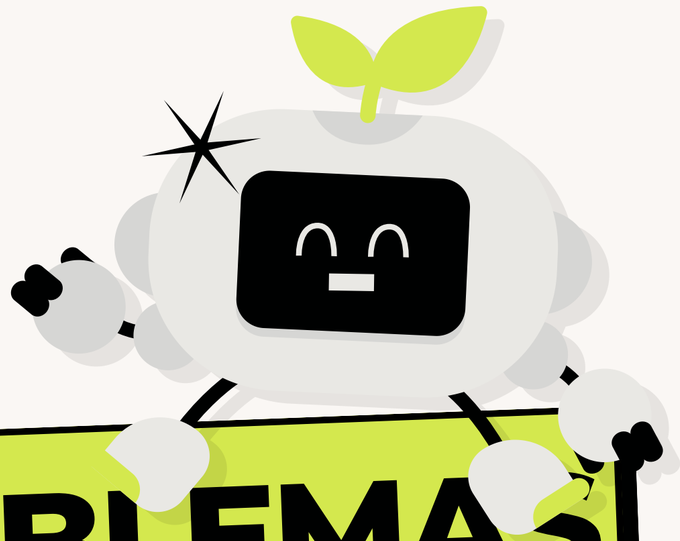
# SUMÁRIO


01 PROBLEMAS COMPUTACIONAIS

02 PRINCÍPIOS BÁSICOS DE APRENDIZADO DE MÁQUINA

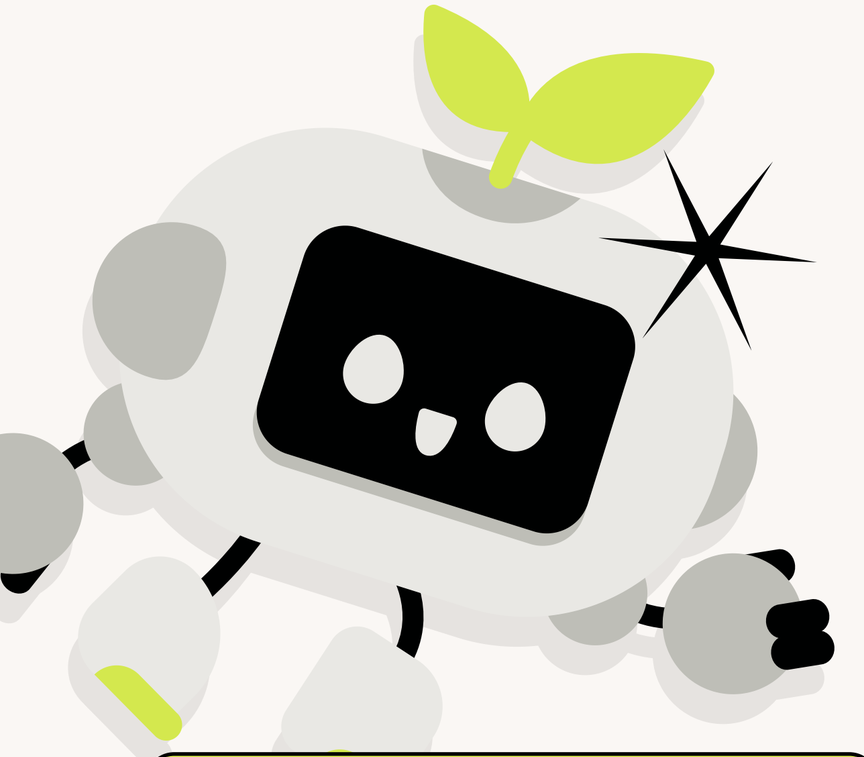
03 LARGE LANGUAGE MODEL E AGENTES DE IA

04 REALIDADE ATUAL E FUTURO



# PROBLEMAS COMPUTACIONAIS

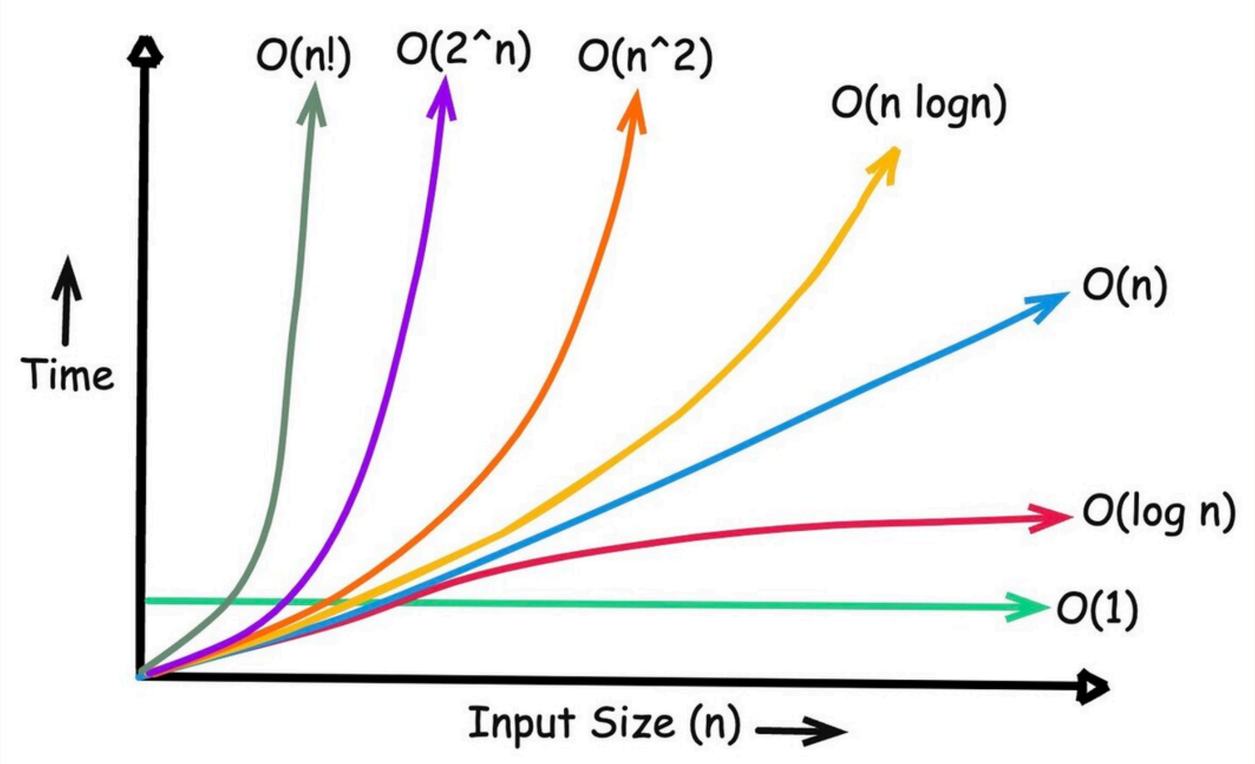
# BIG O



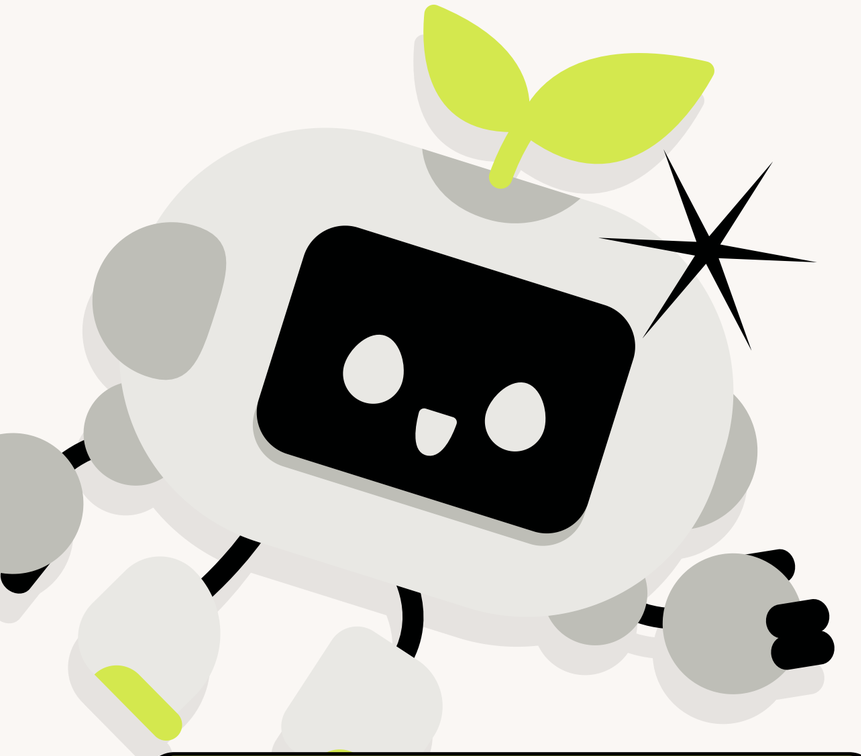
**EDMILSON**

Oi, eu sou o Edmilson. Seu agente de AI baseado em uma LLM.

Vejo que você está estudando notação Big O... No que você se meteu, heim?



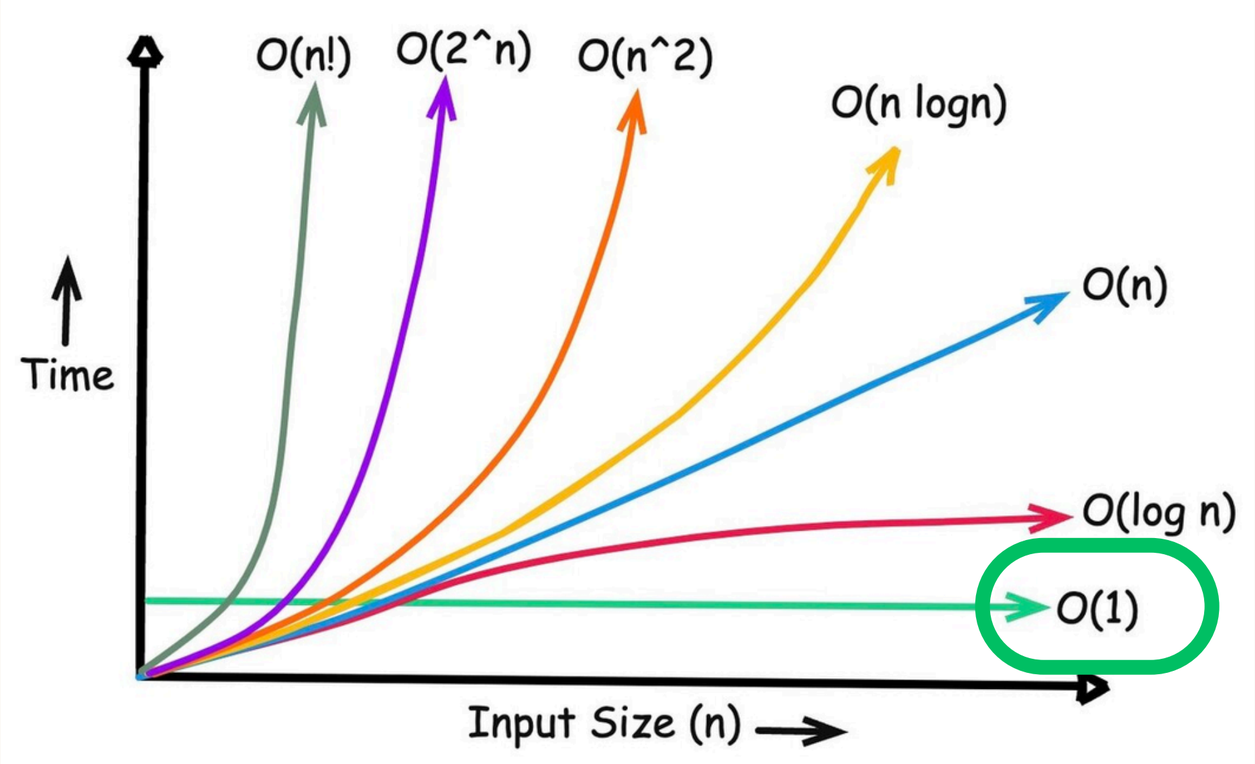
# BIG O



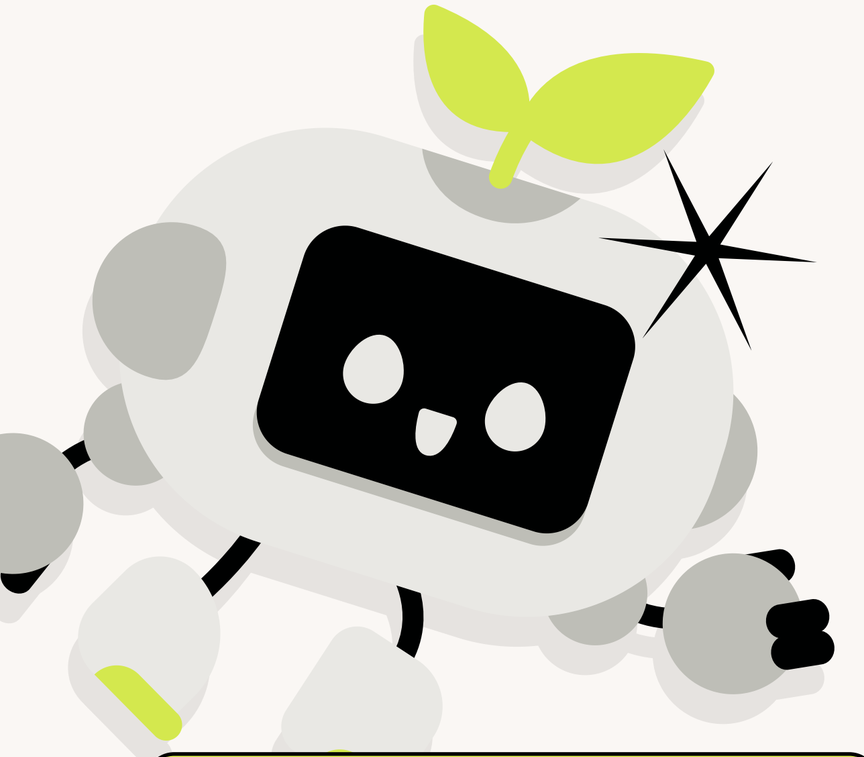
**EDMILSON**

***Operações O(1)***

1. Adicionar um elemento no final de um array
2. Pegar um elemento em um HashMap pela Key



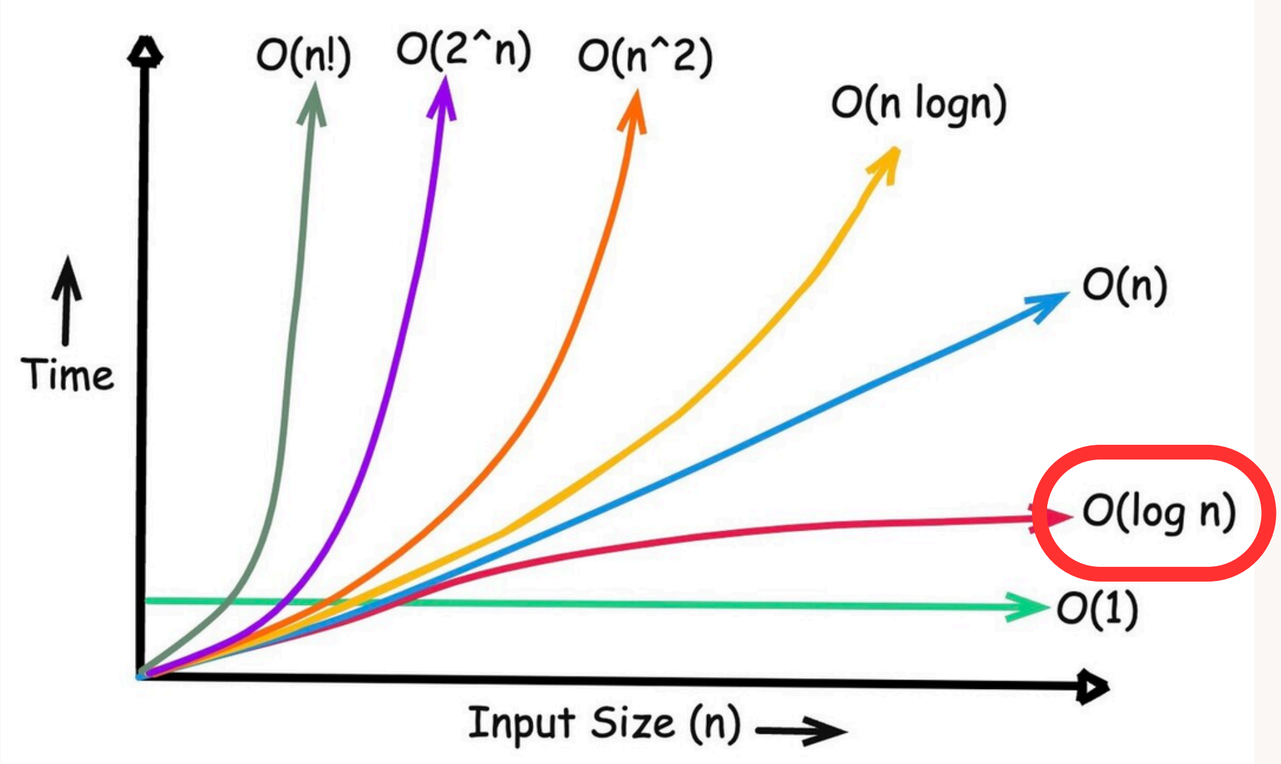
# BIG O



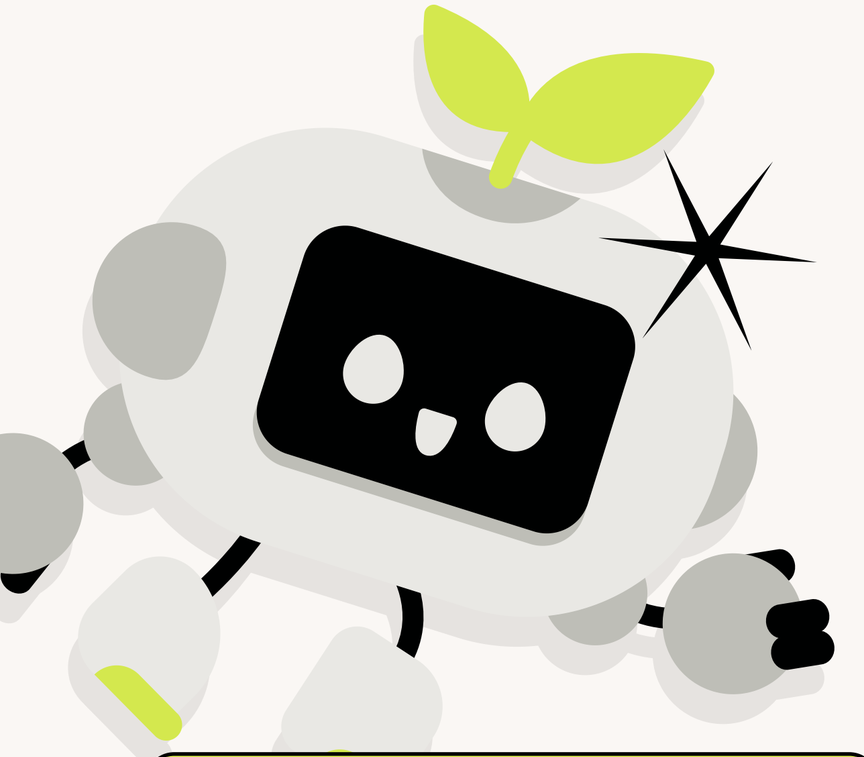
**EDMILSON**

***Operações O(log n)***

1. Busca binária em uma lista ordenada
2. Busca binária em uma árvore binária balanceada



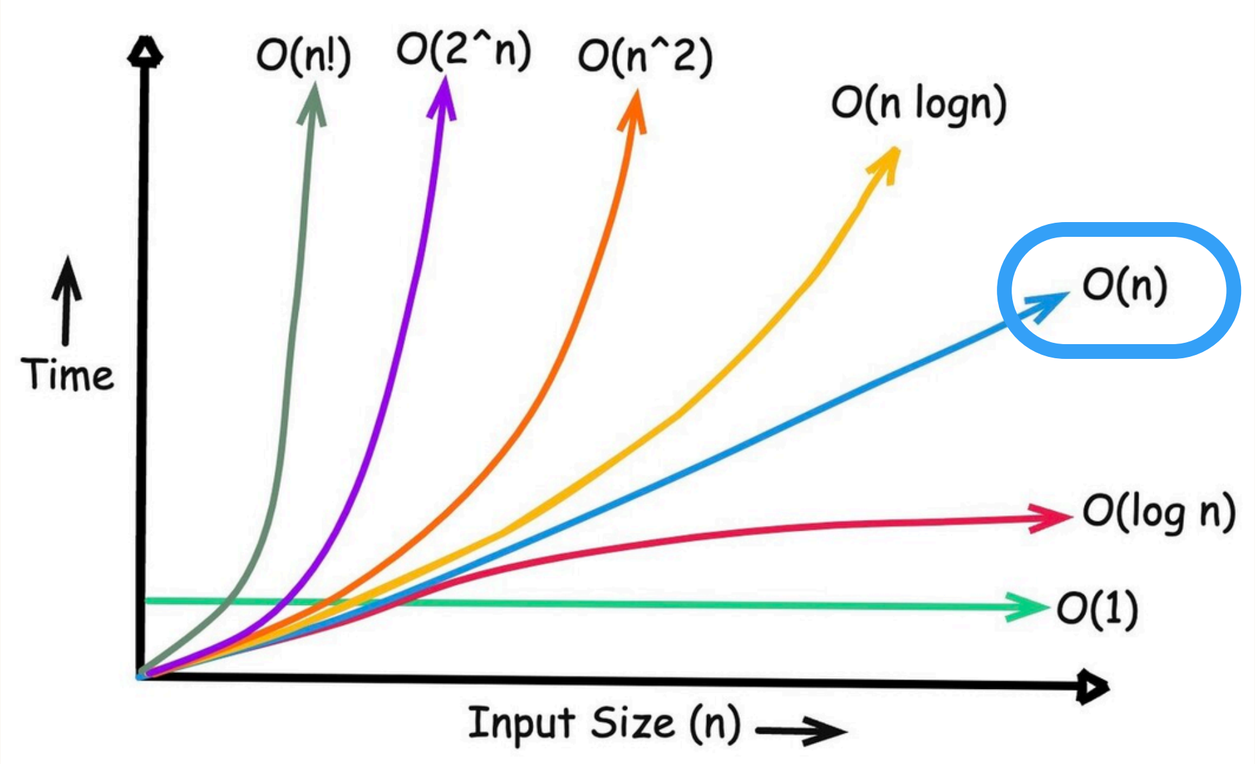
# BIG O



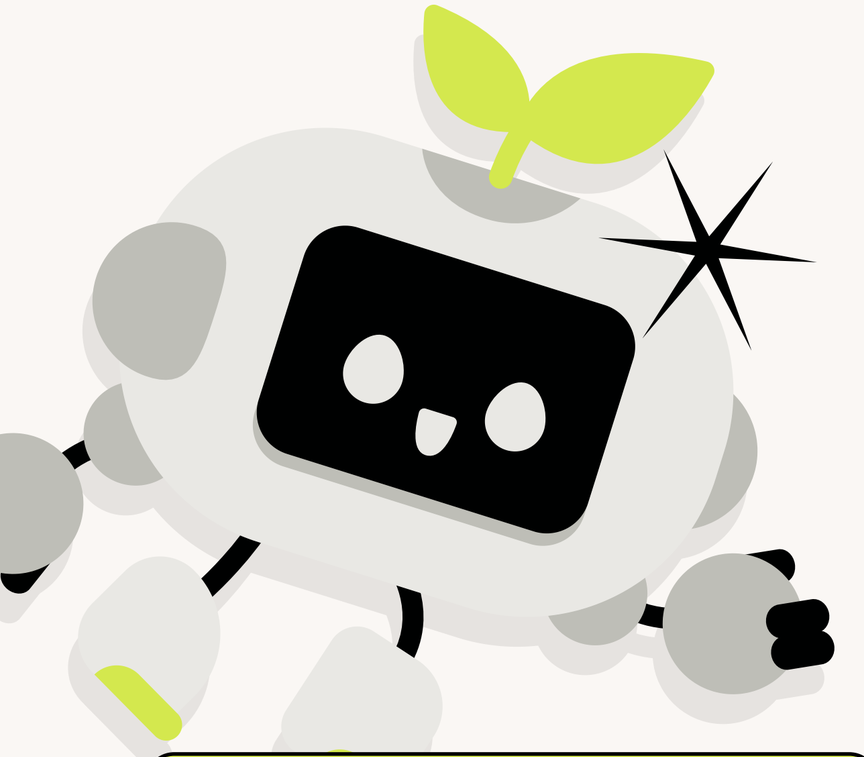
**EDMILSON**

***Operações O(n)***

1. Busca linear em um array não ordenado.
2. Busca de min e max.



# BIG O

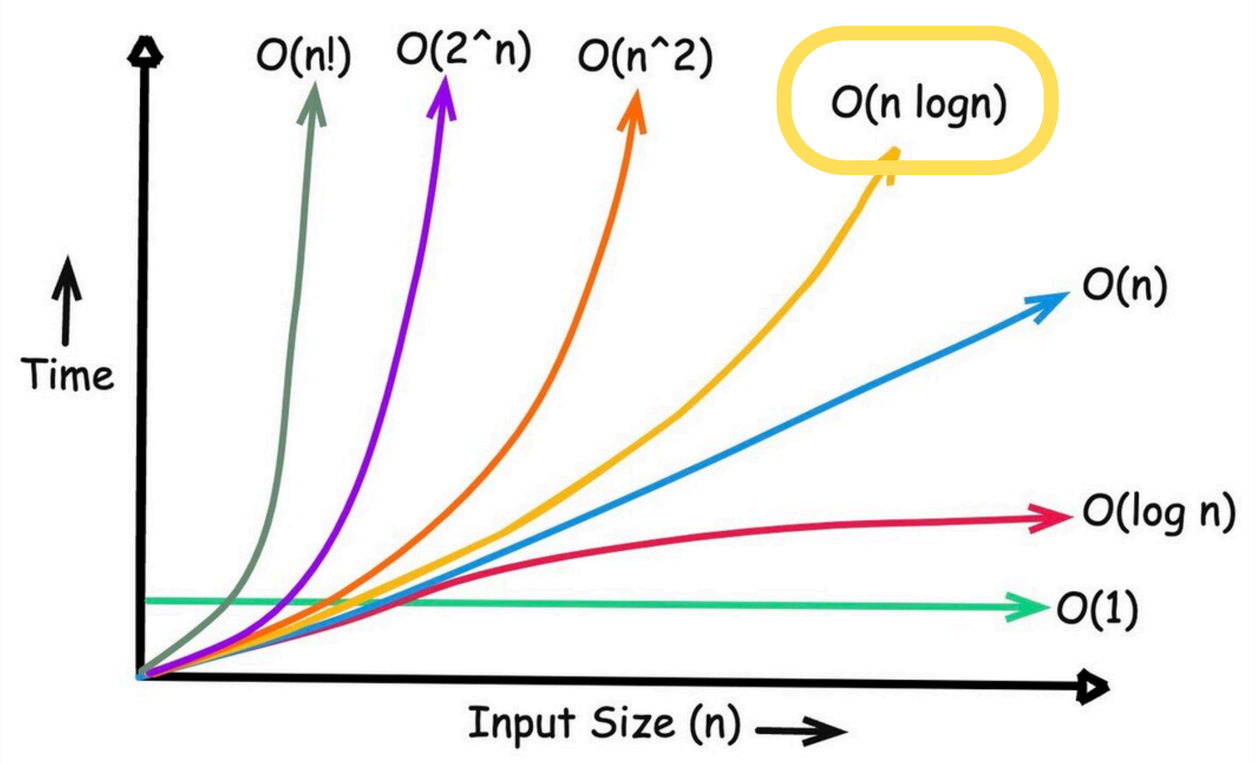


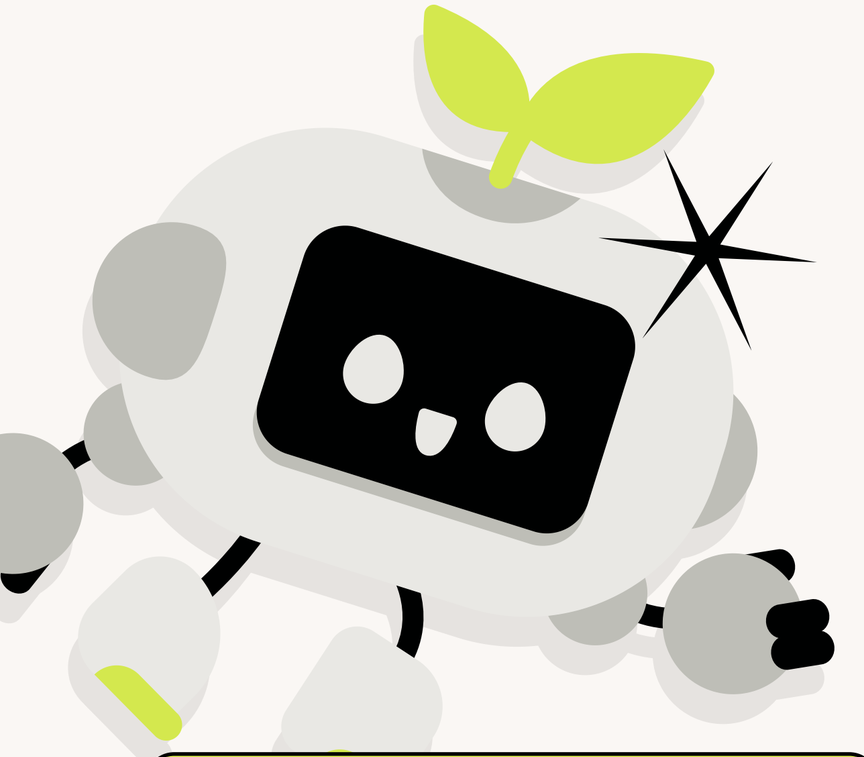
**EDMILSON**

***Operações O(n log n)***

1. Alguns algoritmos de ordenação, como: Merge Sort, Quick Sort e etc
2. Fast Fourier Transform

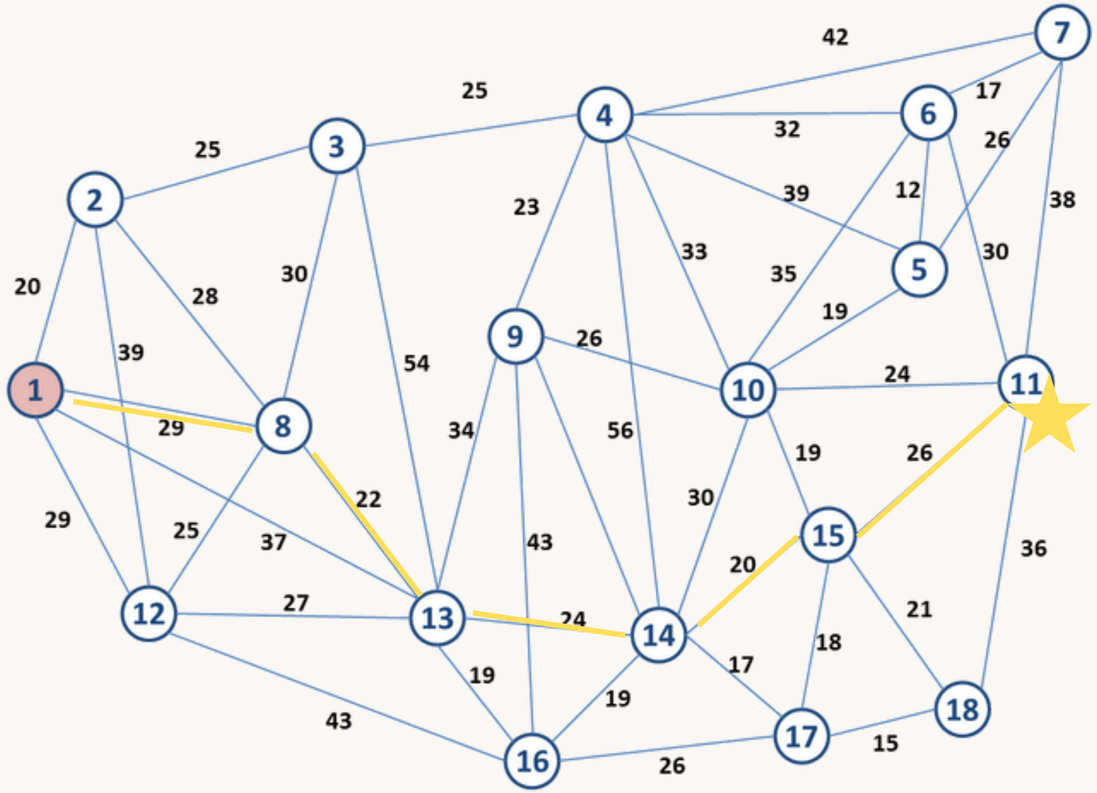
...





# DIJKSTRA

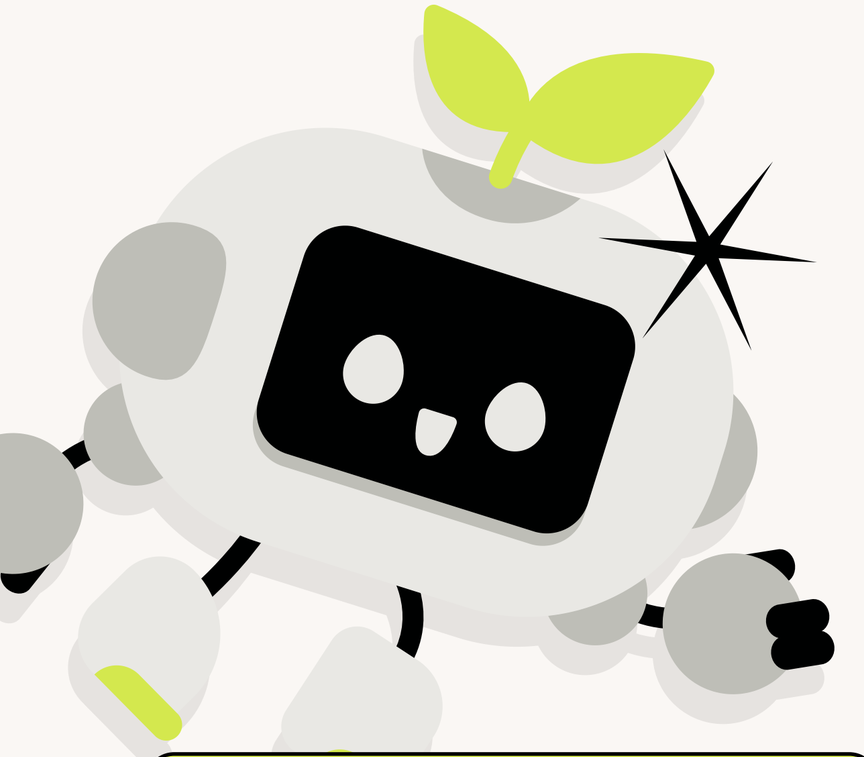
O(n log n)




**EDMILSON**

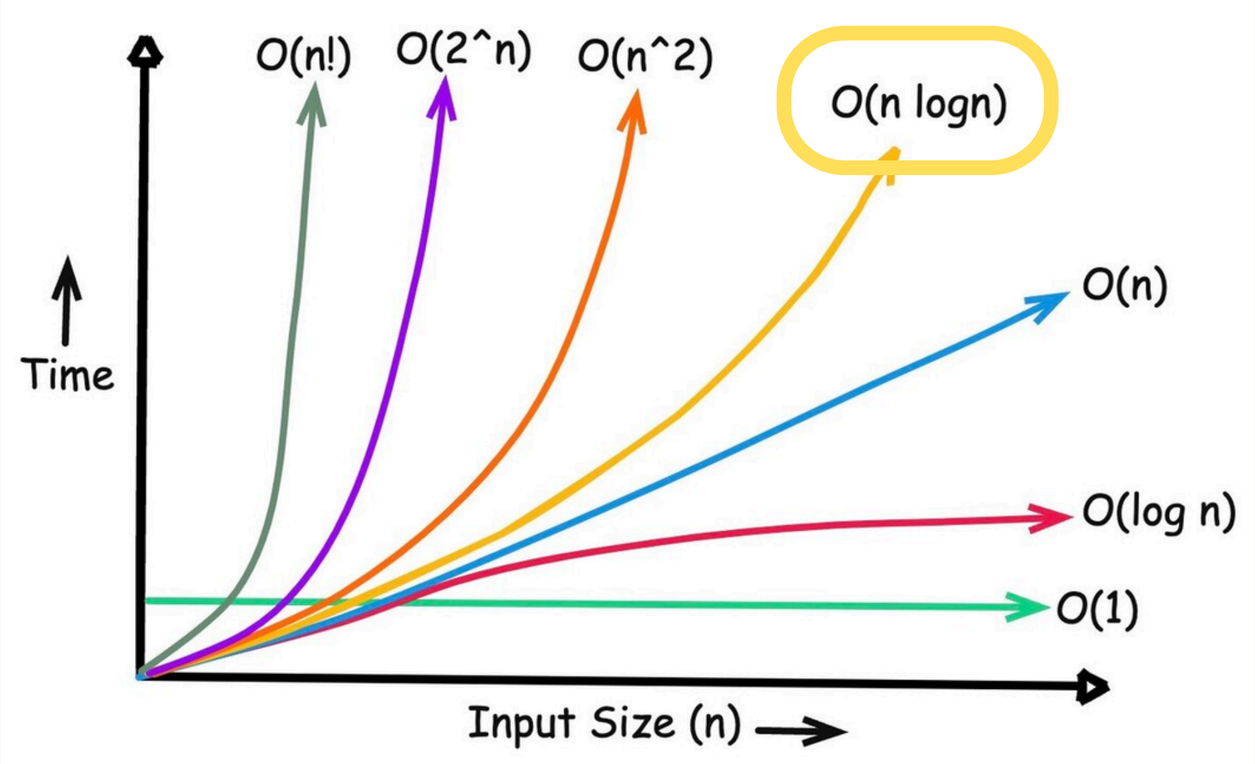
Encontrar a melhor rota entre dois vértices de um Grafo com Algoritmo de Dijkstra é O (n log n)...

# BIG O

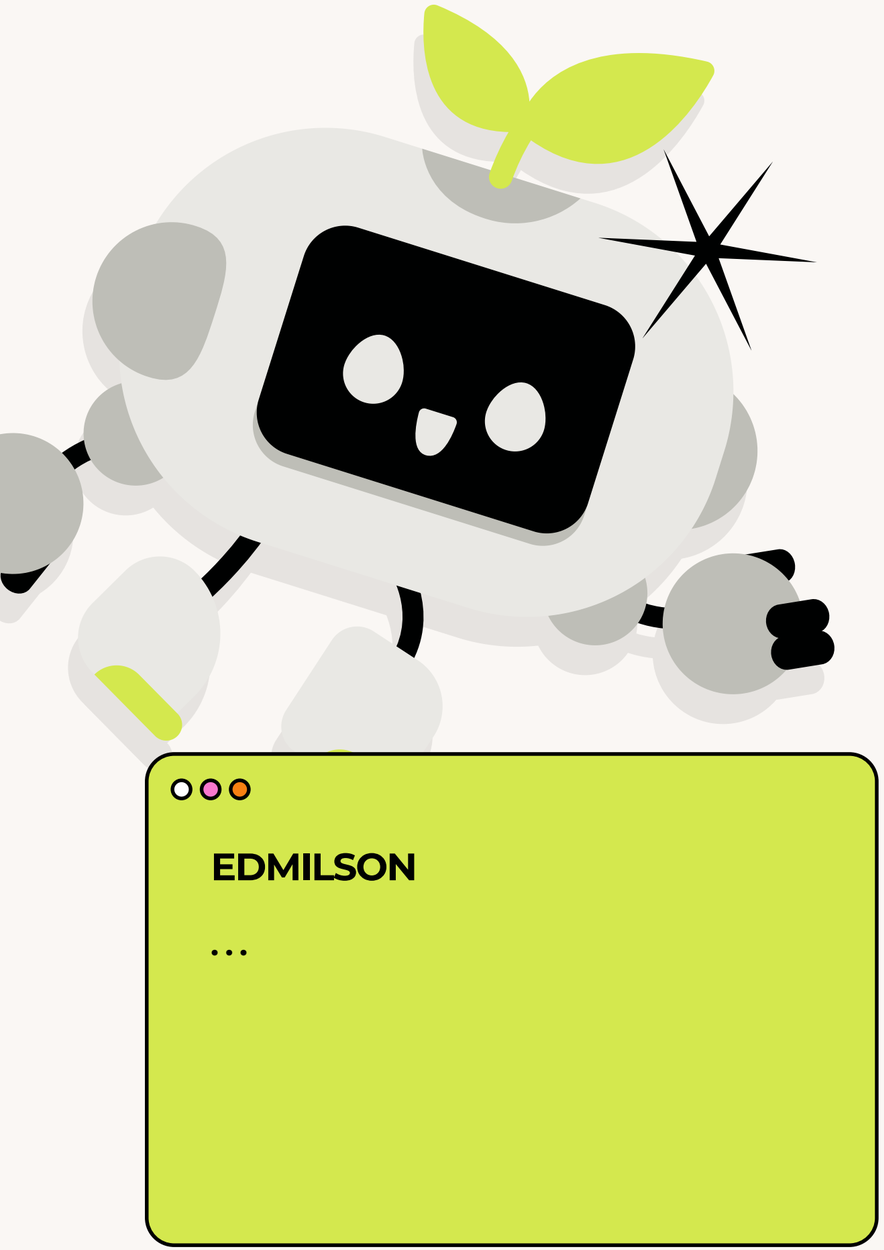


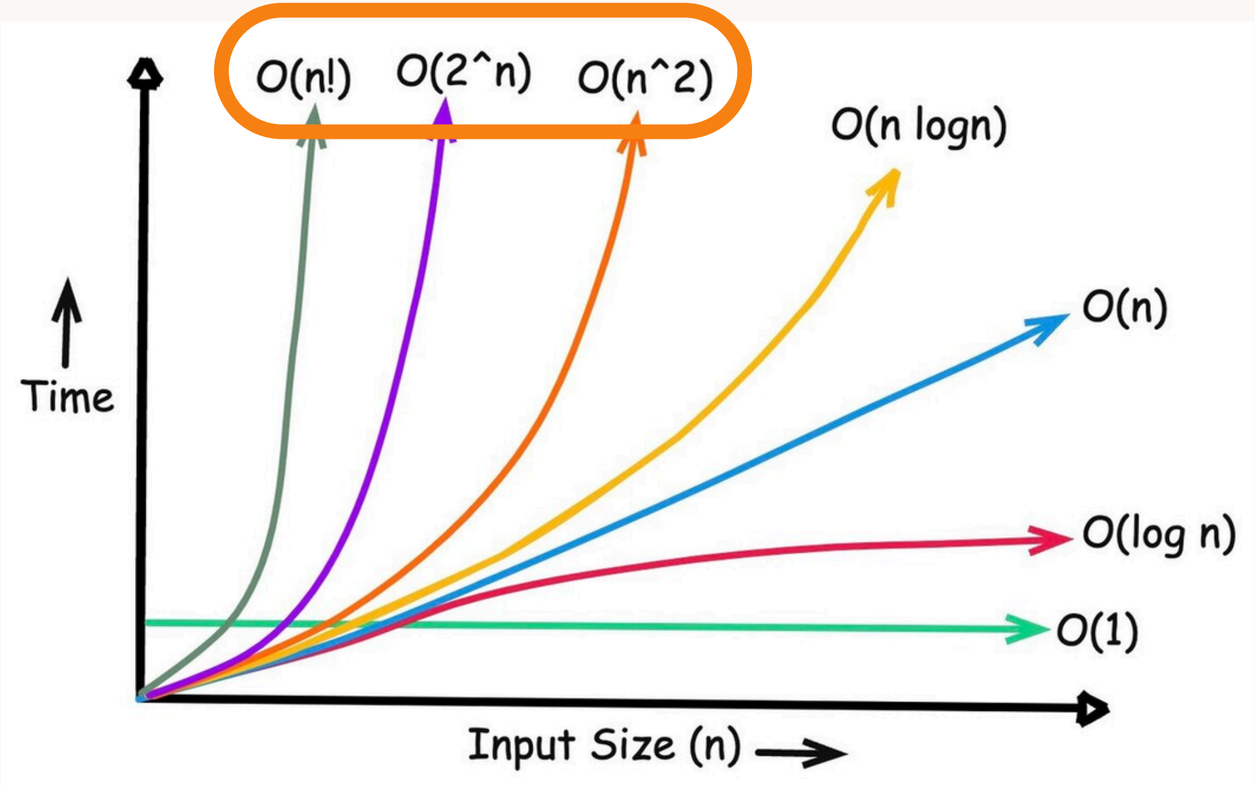
**EDMILSON**

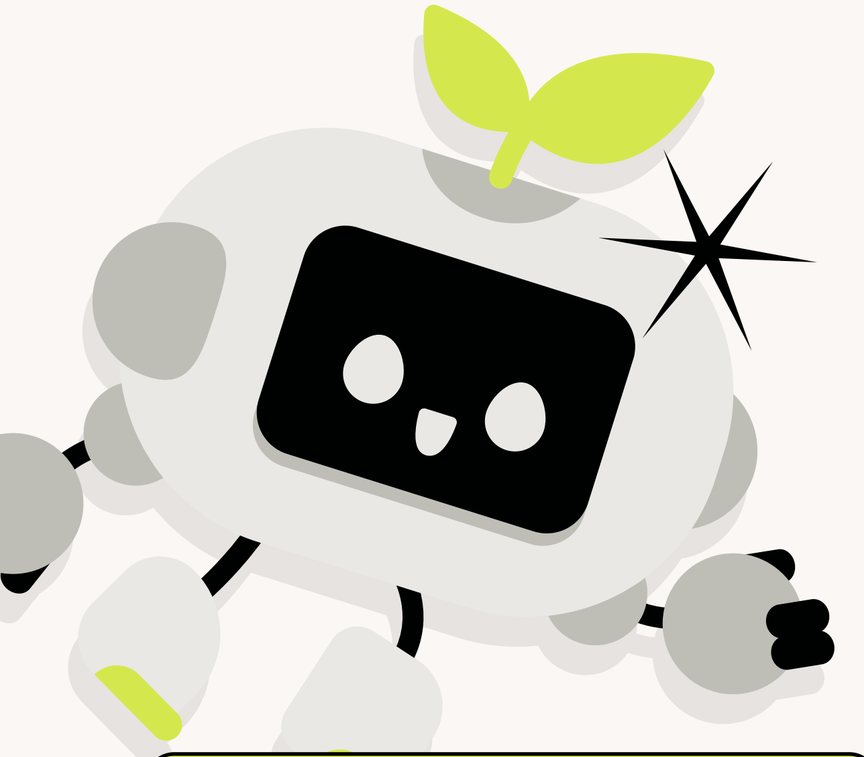
Encontrar a melhor rota entre dois vértices de um Grafo com Algoritmo de Dijkstra é O (n log n)...



# BIG O

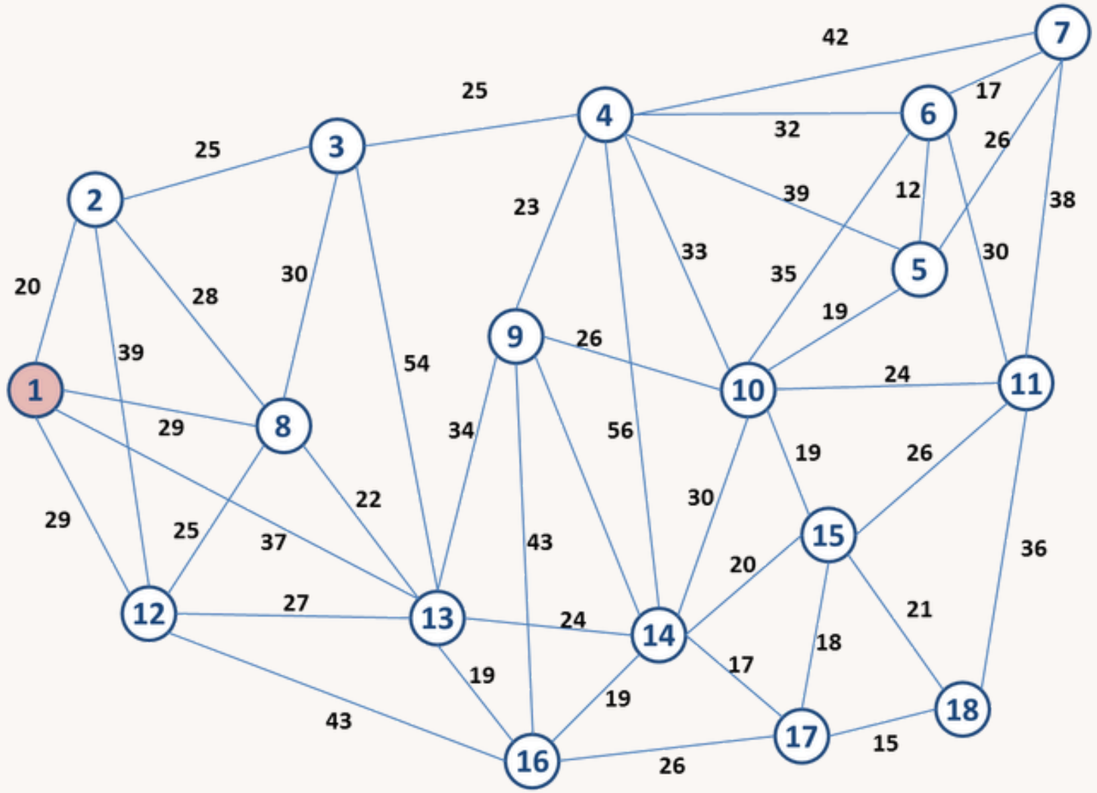




# BELLMAN-HELD-KARP

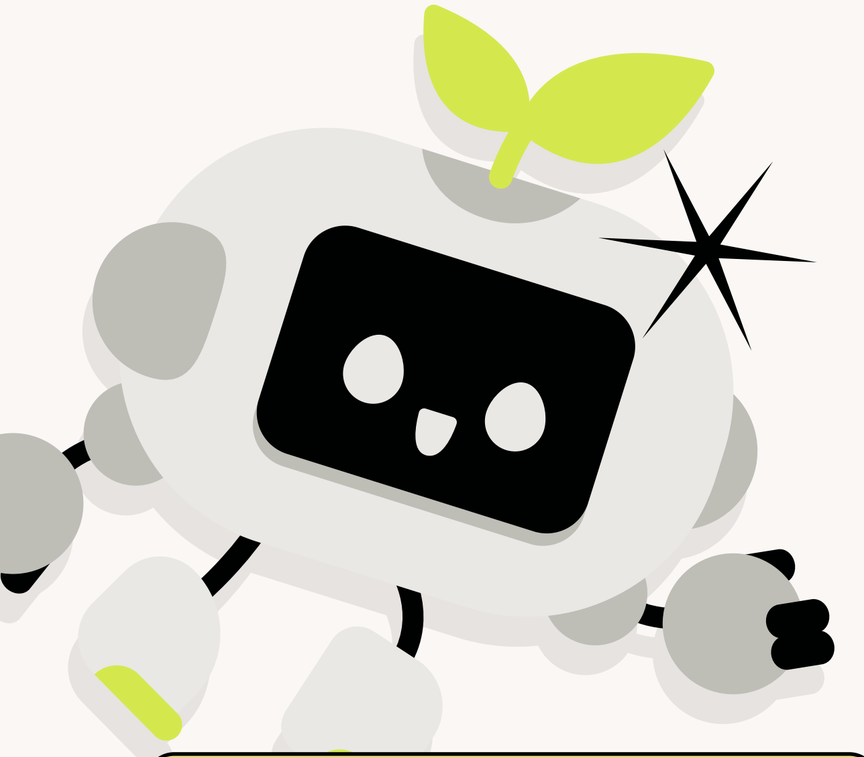
$O(n^2 2^n)$




**EDMILSON**

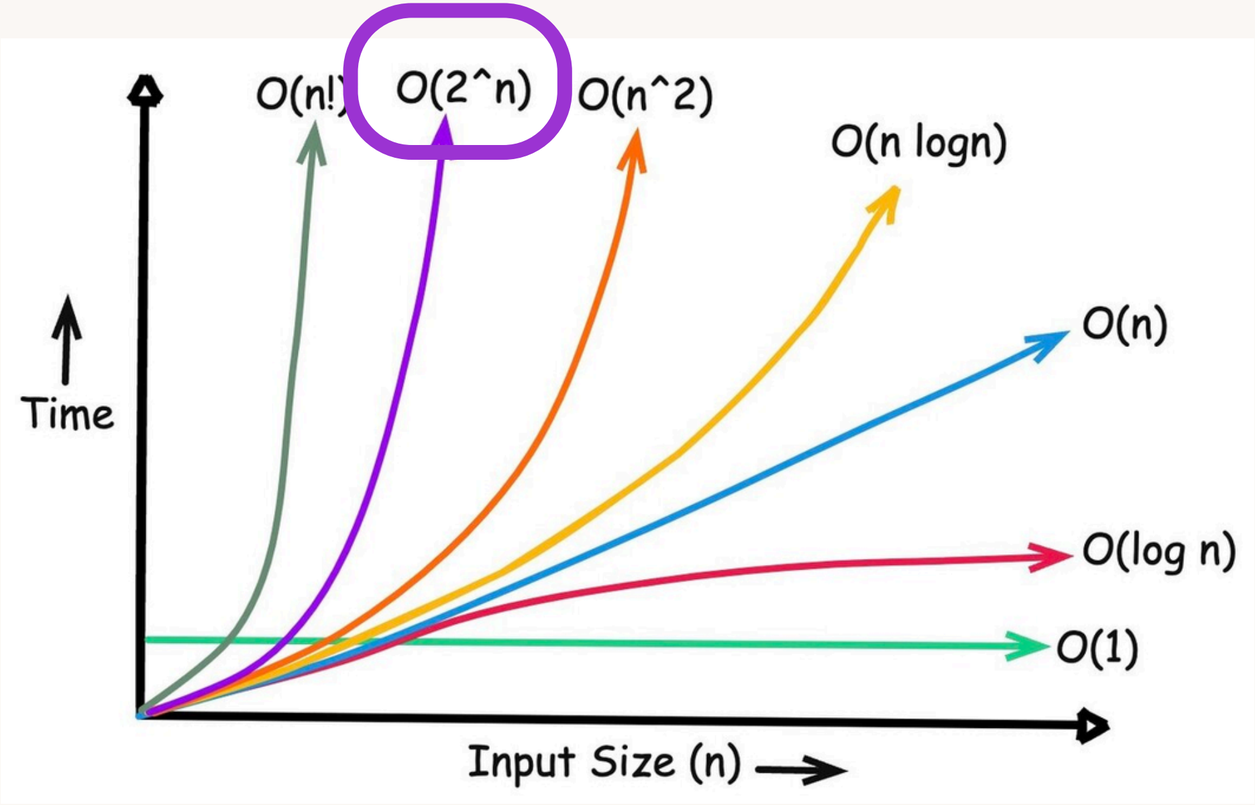
**O Famoso problema do caixeiro viajante:**

Encontrar a melhor rota que passe por todos os vértices de um Grafo com o Algoritmo de Bellman-Held-Karp é $O(n^2 2^n)$



# BELLMAN-HELD-KARP

$O(n^2 2^n)$



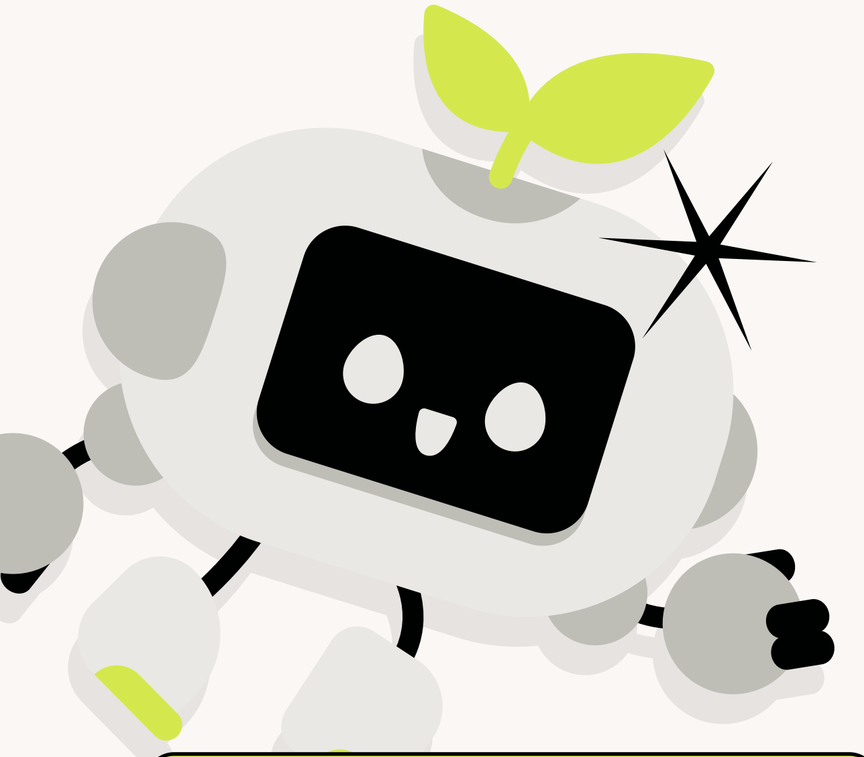

**EDMILSON**

***FUN FACT:*** Para resolver esse problema para 25~30 cidades demora mais tempo do que a idade do universo...

$2^{10} = 1024$
$2^{11} = 2048$
...

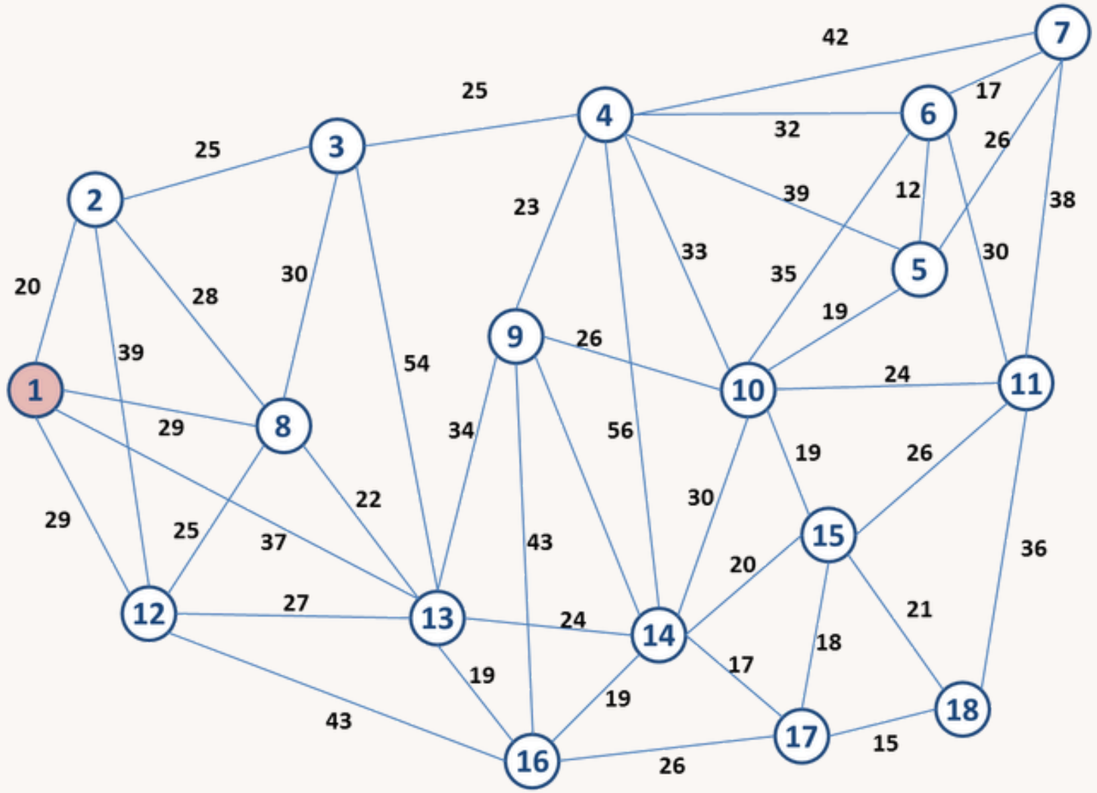
$2^{20} = 1048576$
$2^{21} = 2097152$

$2^{30} = 1073741824$
$2^{31} = 2147483648$



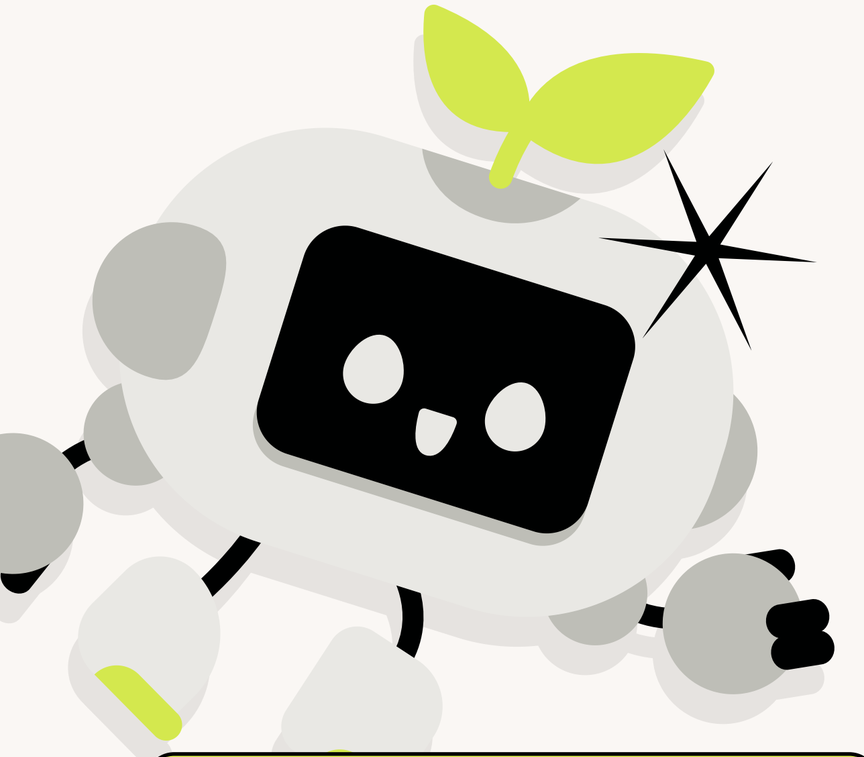
# MÉTODOS NUMÉRICOS

álgebra linear e estatística



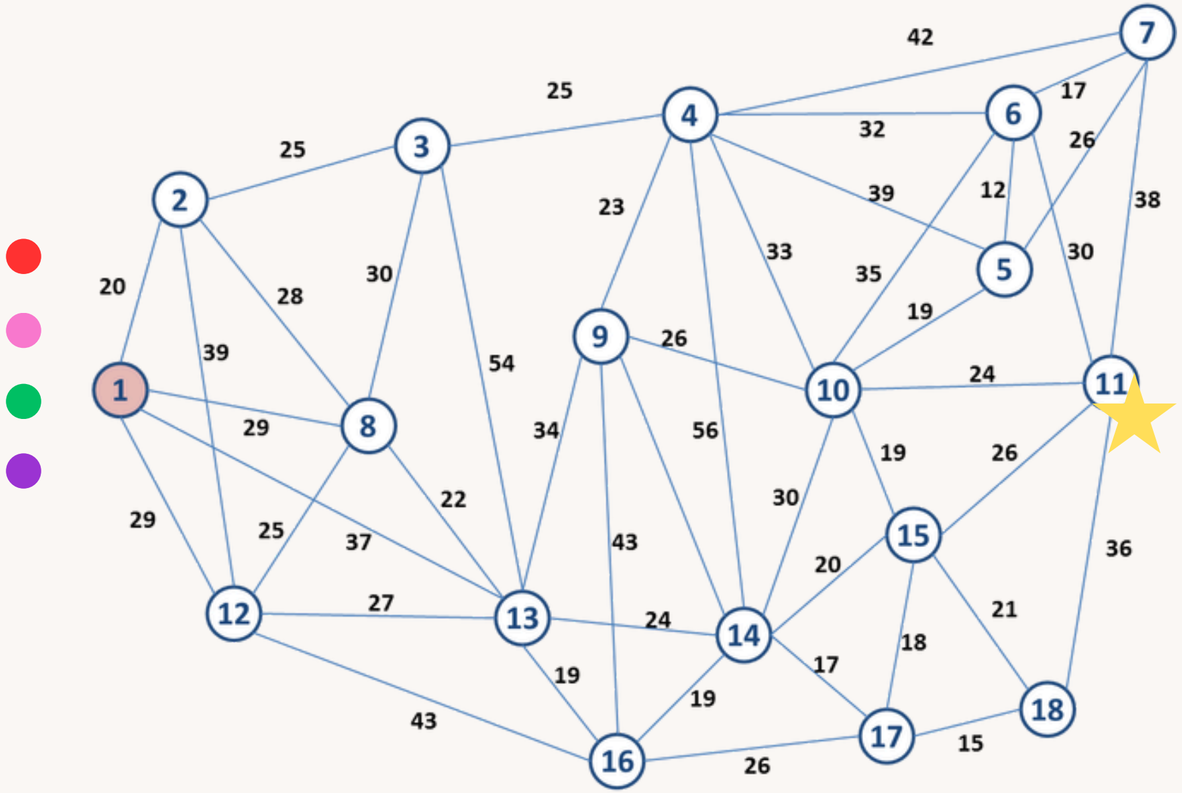

**EDMILSON**

**E se, eu mandar o computador ficar testando até ele encontra algo bom, mesmo que não seja o melhor/ótimo?**



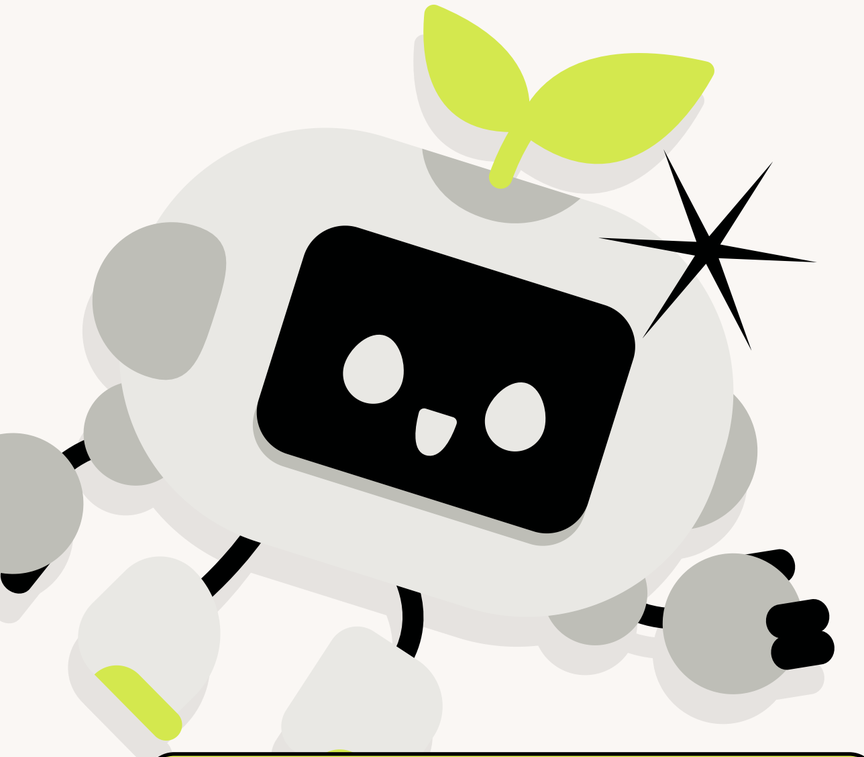
# ALGORTIMOS DE OTIMIZAÇÃO

## Colônia de formigas



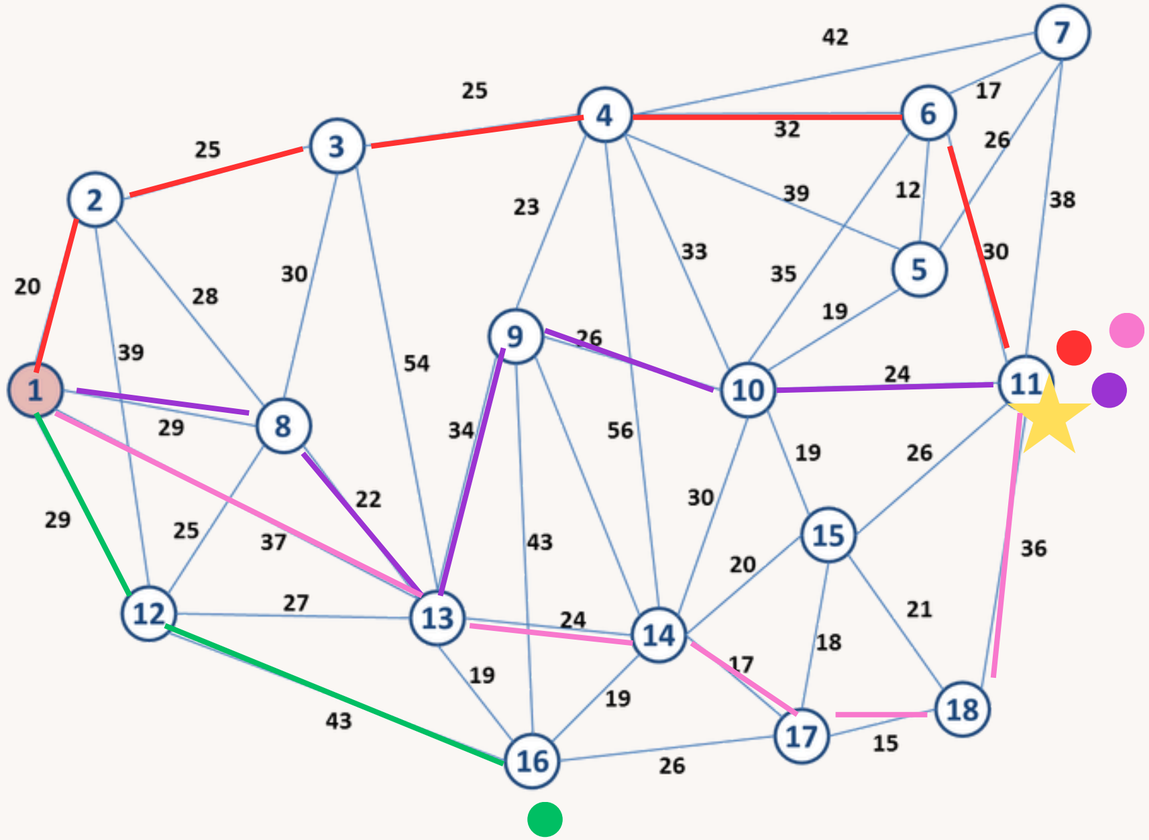

**EDMILSON**

Vou criar 4 formigas e pedir para elas andarem no grafo, elas vão dar preferência para arestas de menor caminho, porém vão ter uma possibilidade aleatória de escolherem outra aresta.



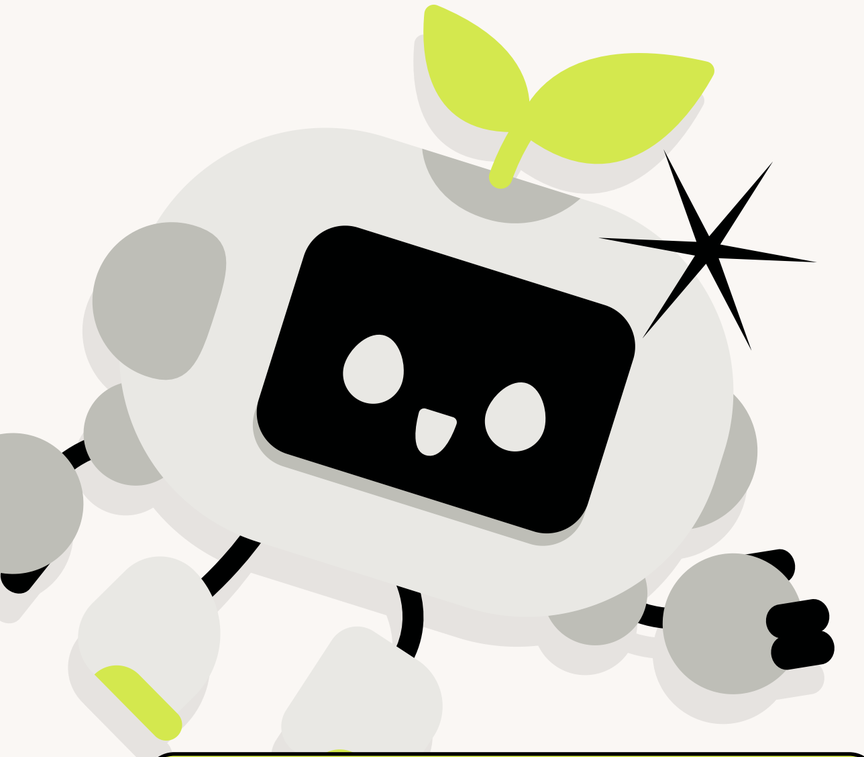
# ALGORTIMOS DE OTIMIZAÇÃO

## Colônia de formigas



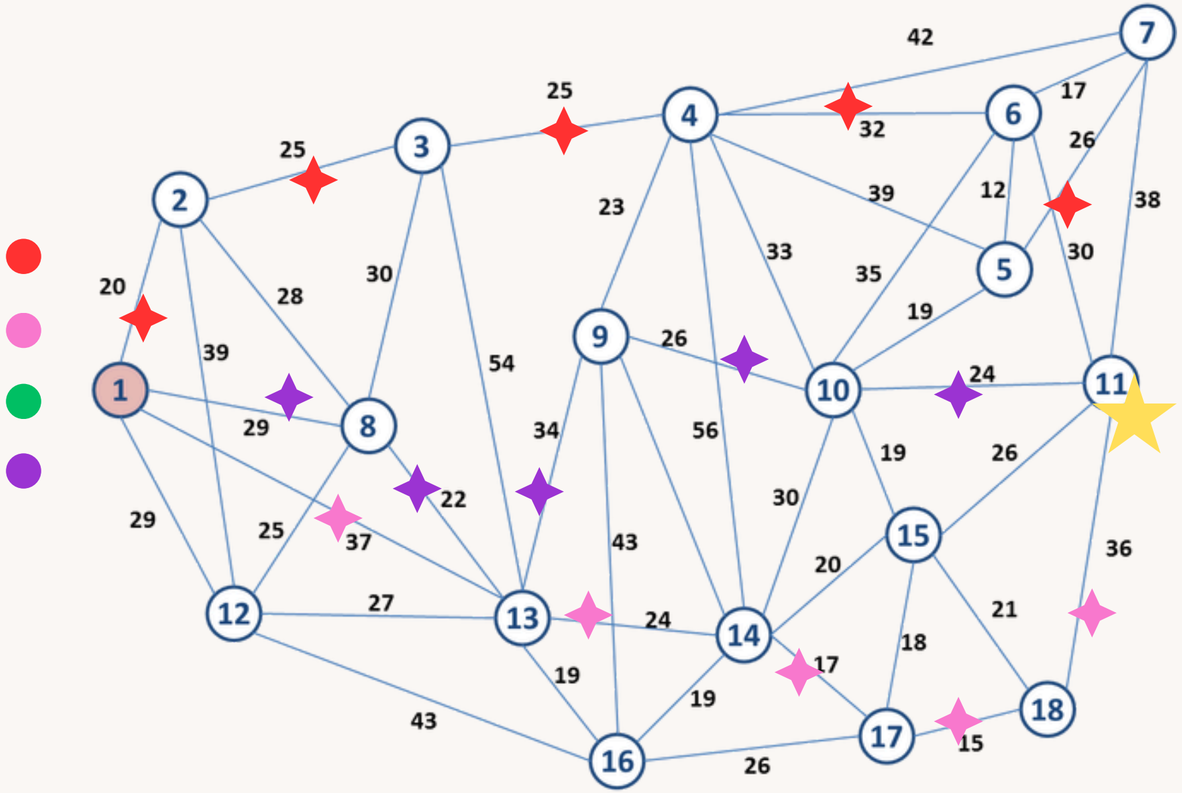

**EDMILSON**

Vou marcar cada aresta que as formigas passara com um ganho (inspirado no feromônio), as formigas que caminharam menos deixam um rastro de feromônio maior do que as outras, as que não chegaram no objetivo deixam 0.



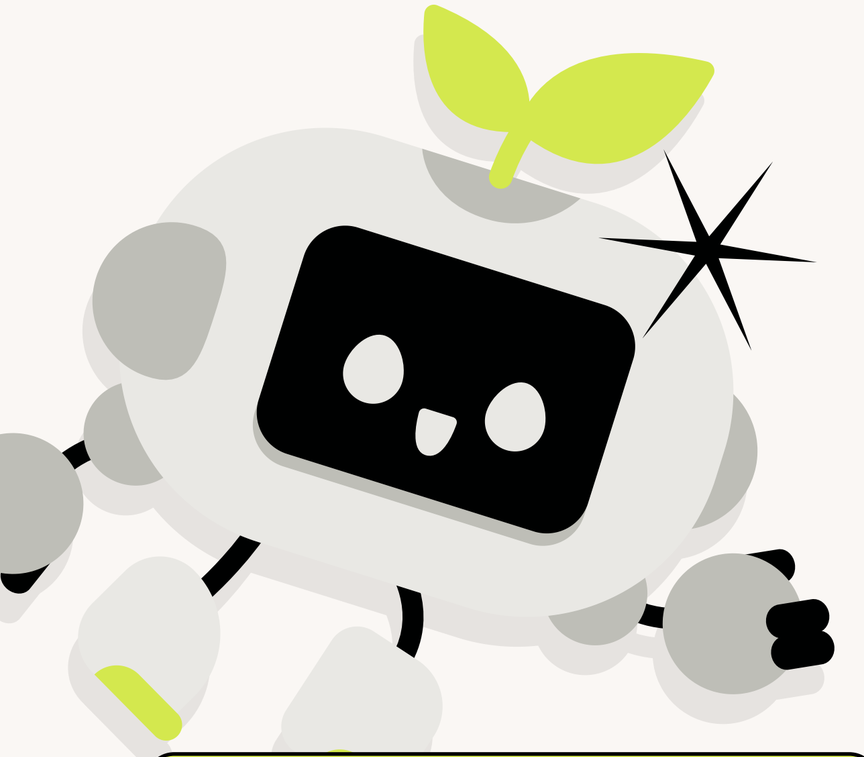
# ALGORTIMOS DE OTIMIZAÇÃO

## Colônia de formigas



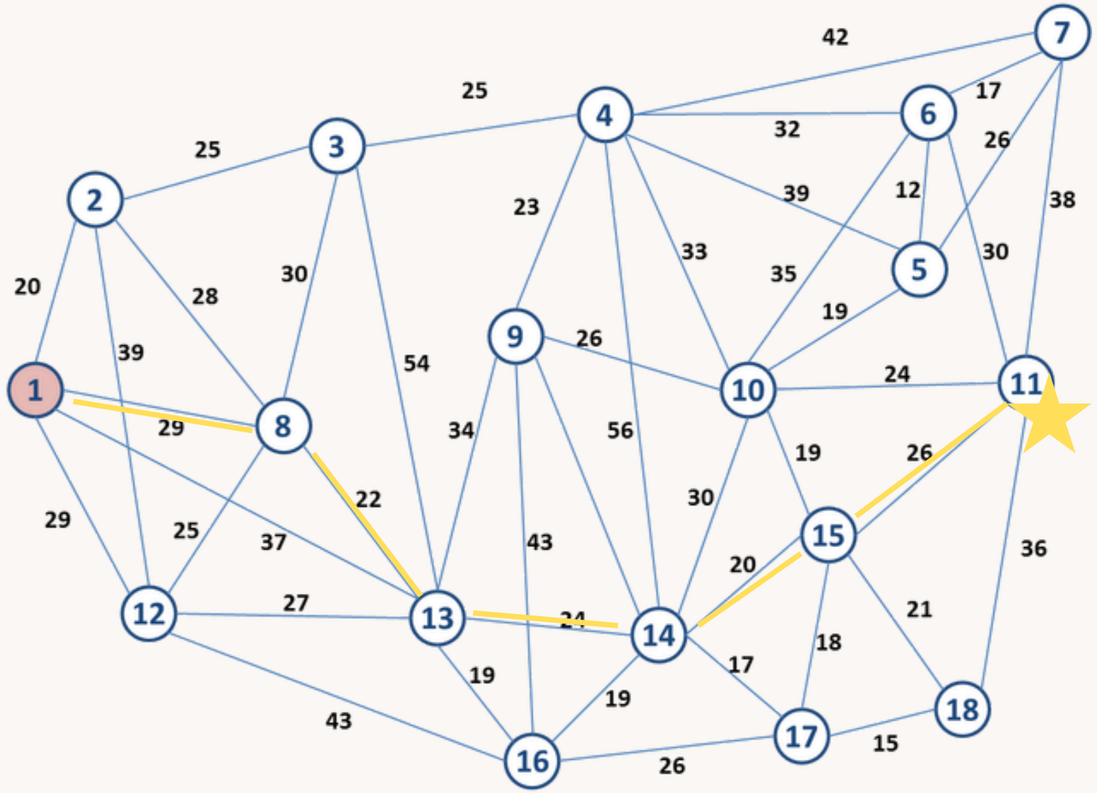

**EDMILSON**

Vou mandar as formigas novamente, dessa vez elas também vão considerar o feromônio no rastro...



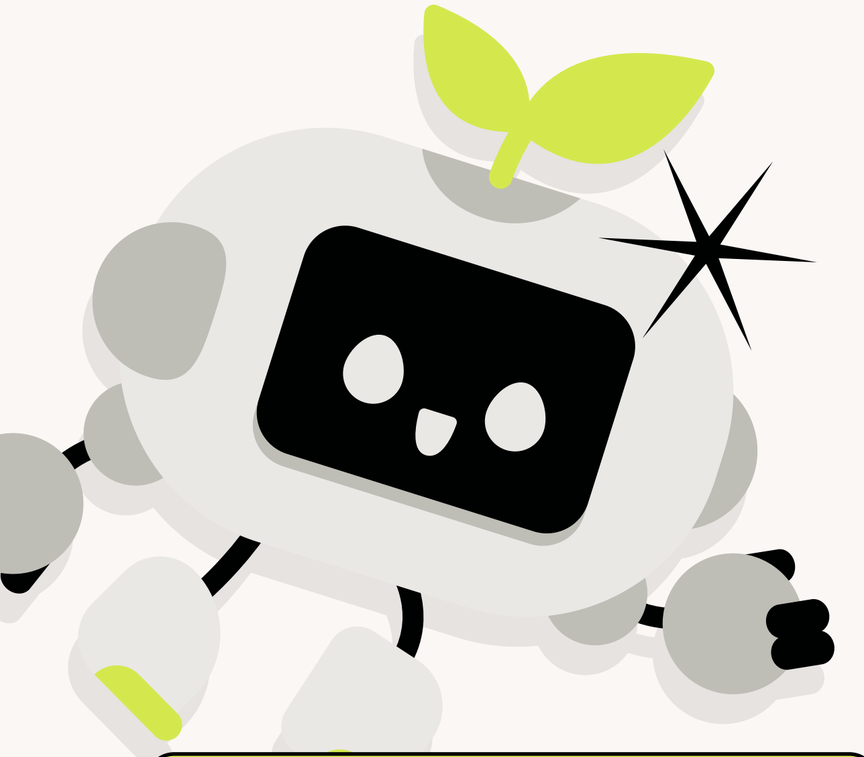
# ALGORTIMOS DE OTIMIZAÇÃO

Colônia de formigas



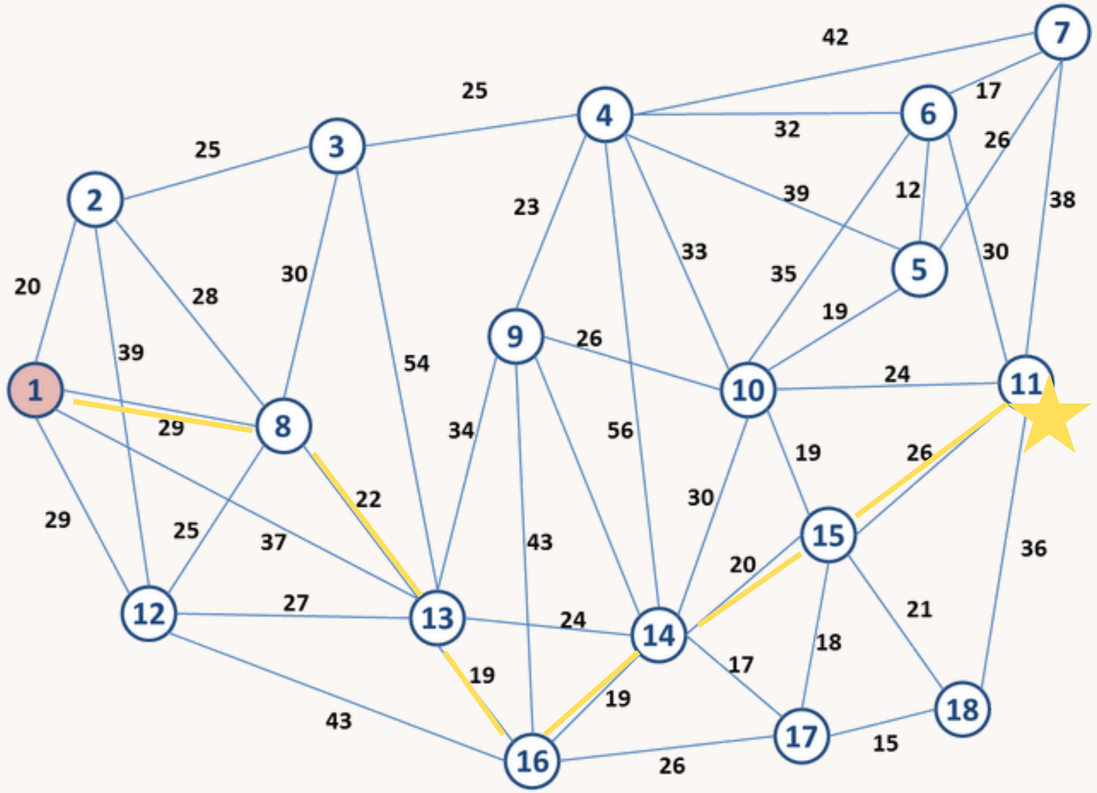

**EDMILSON**

Vou repetir isso várias e várias vezes e terei um caminho **bom**, provavelmente não será o **melhor/ótimo**.



# ALGORTIMOS DE OTIMIZAÇÃO

Colônia de formigas

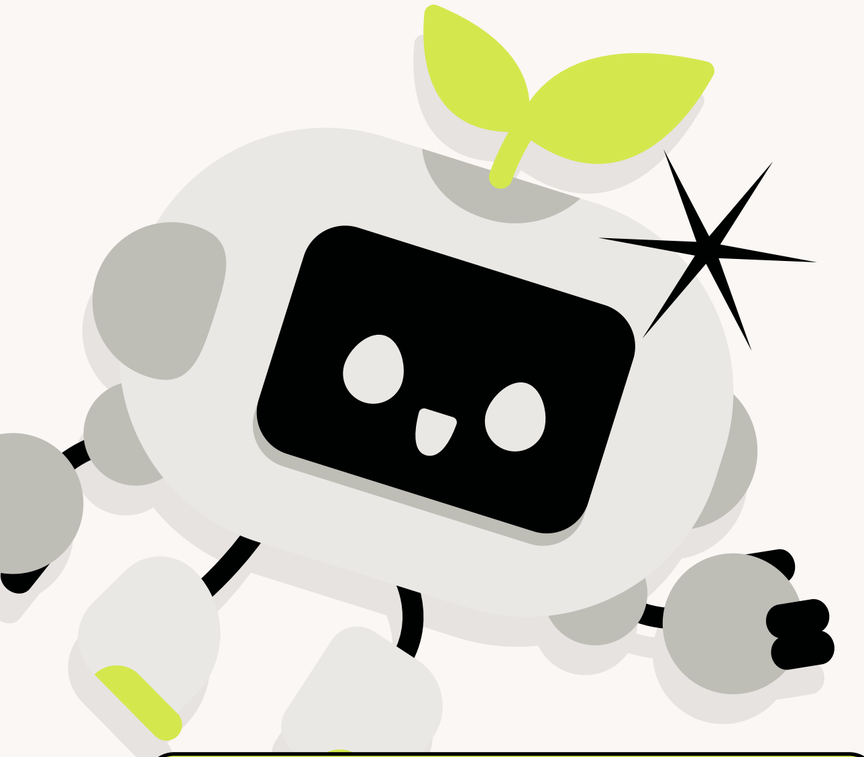



**EDMILSON**

Se rodar todo o processo novamente, terei um **outro bom caminho**...

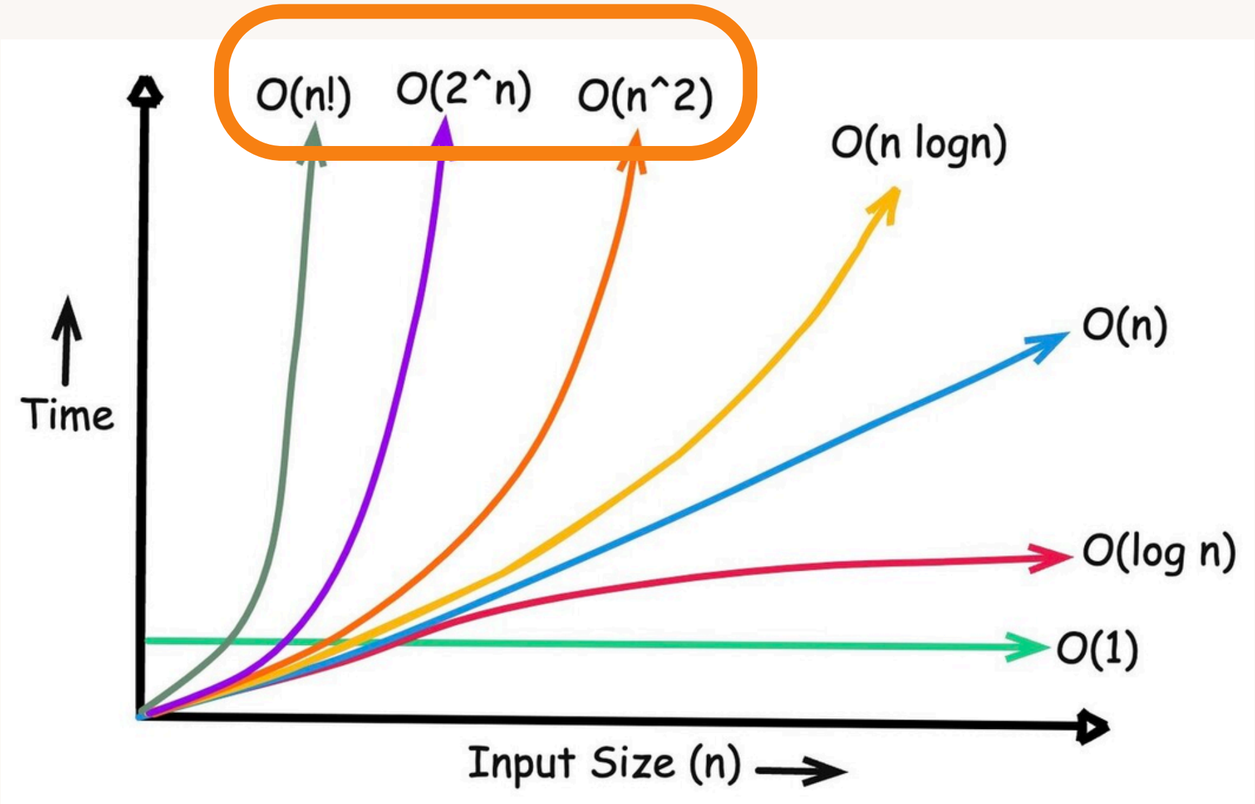
Toda vez que eu executar esse processo, posso ter respostas diferentes, isso se chama **não determismo**.

# SOLUÇÕES NÃO DETERMINÍSTICAS

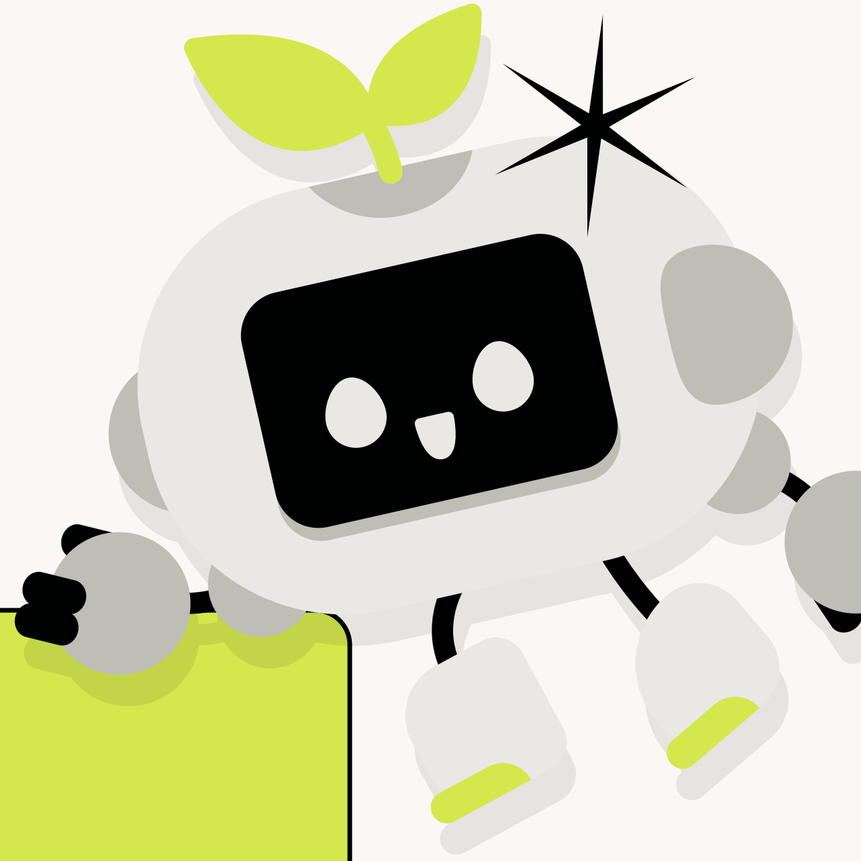


**EDMILSON**

Se seu problema se enquadra nesses casos **O(n!), O($2^n$)** ou **O($n^2$)**. E você precisa lidar com escala, certamente deverá usar **soluções não determinísticas.**



# PROBLEMAS DE VERIFICAÇÃO



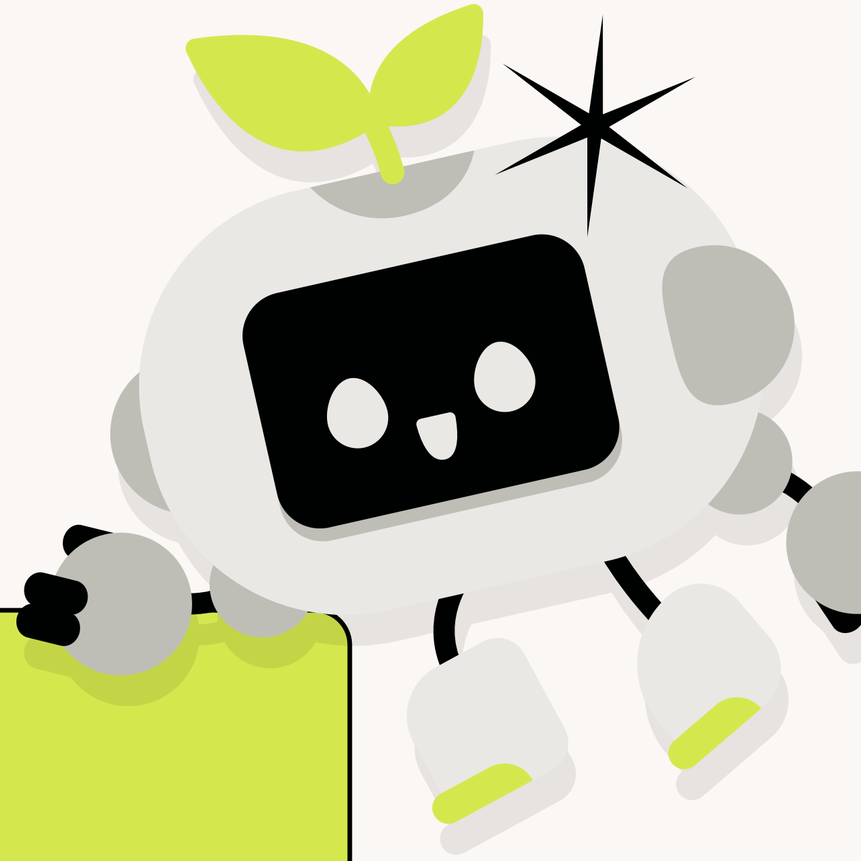
## P

1. Ordenar uma lista
2. Dijkstra
3. Busca linear

## NP

1. Caixeiro viajante
2. Sudoku
3. Problema da mochila

# PROBLEMAS IMPOSSÍVEIS



## DECIDÍVEL

1. Caixeiro Viajante
2. Ordenar uma lista
3. Teste de primalidade

## INDECIDÍVEL

1. Problema da parada
2. Equivalência de programas

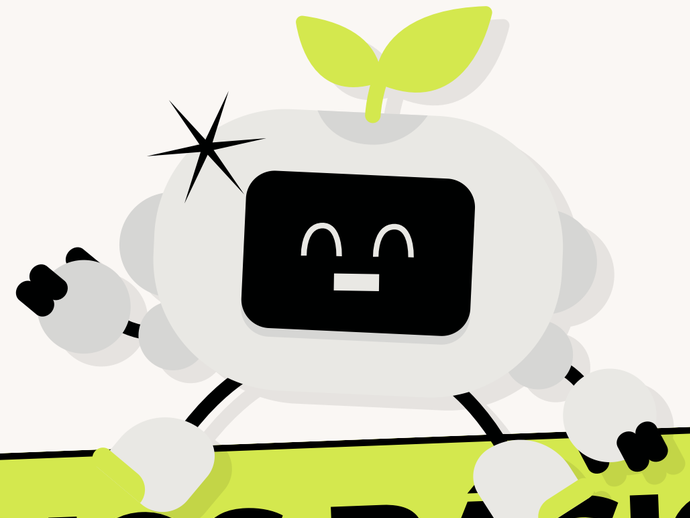
# DETERMINISMO VS NÃO DETERMINISMO

## DETERMINISMO

1. É possível implementar um algoritmo que resolva o problema;
2. Necessário garantir que sempre que um processo seja chamado com a mesma entrada, o resultado será o mesmo;
3. Necessário a solução ótima;
4. Na necessidade de escala, a solução não explode em tempo computacional;

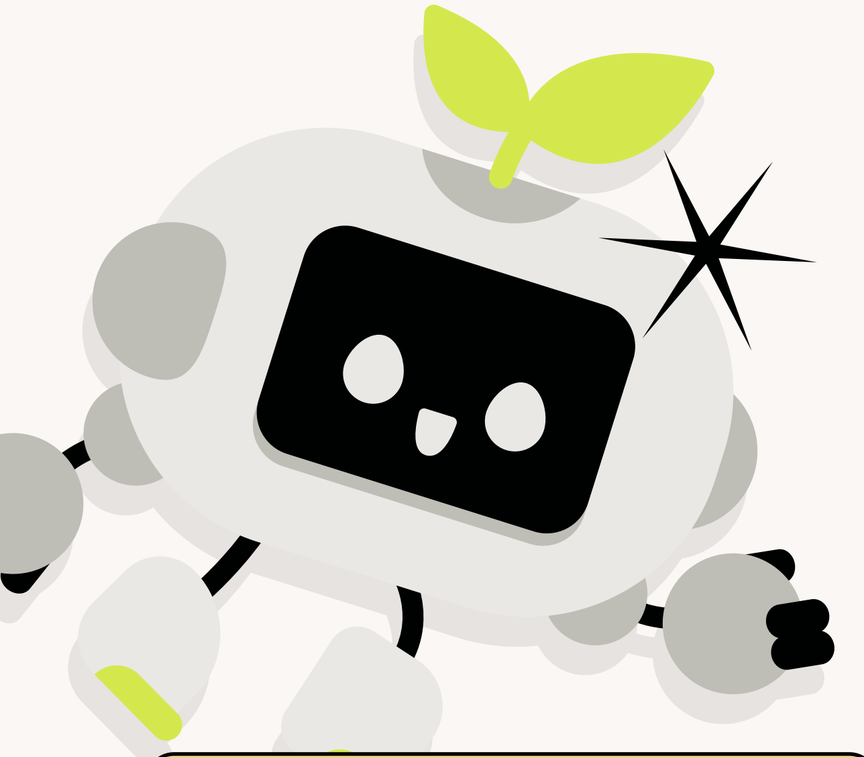
## NÃO DETERMINISMO

1. É impossível implementar um algoritmo que resolva o problema;
2. Não importa ter soluções diferentes em diferentes chamadas;
3. Não é necessário a solução ótima;
4. É necessário lidar com problemas custosos em tempo computacional;



# PRINCÍPIOS BÁSICOS

# DE APRENDIZADO DE MÁQUINA

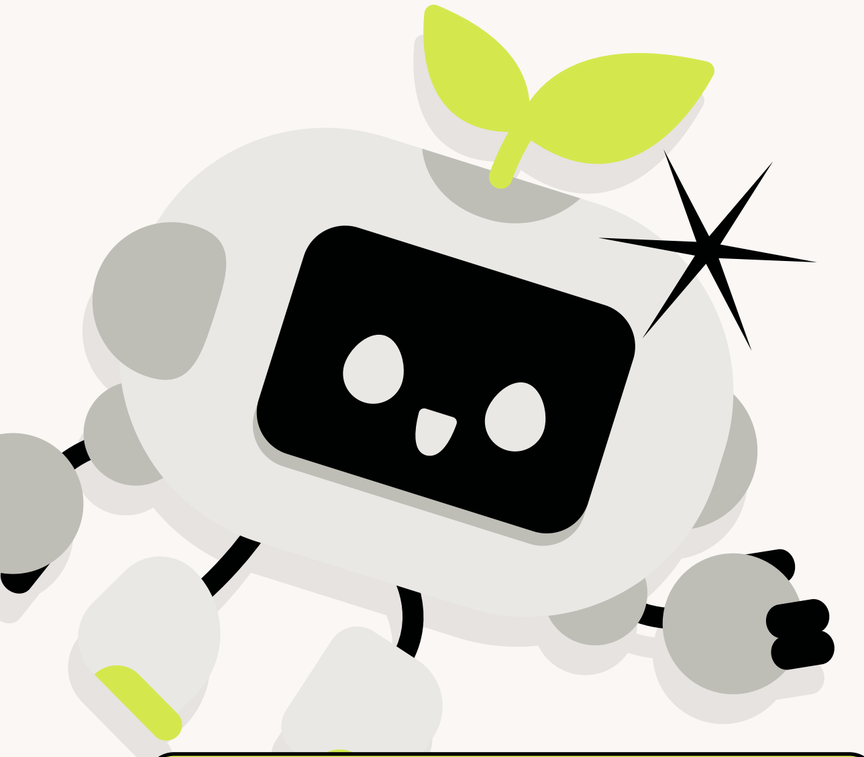


# FORMULANDO UM PROBLEMA

**EDMILSON**

Como podemos criar um programa para controlar um carro em uma pista de corrida?

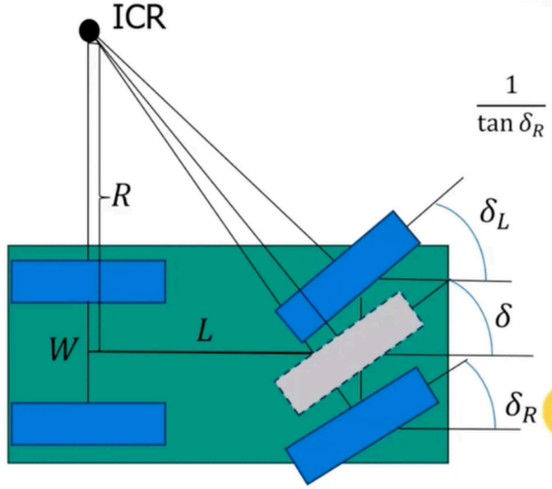
# MODELO CINEMÁTICO



**EDMILSON**

1. Modelo cinemático determinístico.
2. Real = Modelo + Erro.
3. Não considera limites dinâmicos:.

$$\dot{X} = \begin{bmatrix} v * \cos\theta \\ v * \sin\theta \\ \frac{v * \tan\delta}{L} \end{bmatrix}$$




$$\tan\delta = \frac{L}{R} \quad \tan\delta_L = \frac{L}{R - \frac{W}{2}} \quad \tan\delta_R = \frac{L}{R + \frac{W}{2}}$$

$$\frac{1}{\tan\delta_R} - \frac{1}{\tan\delta_L} = \frac{R + \frac{W}{2}}{L} - \frac{R - \frac{W}{2}}{L} = \frac{W}{L}$$

$$\cot\delta_L = \cot\delta - \frac{W}{2L}$$

$$\cot\delta_R = \cot\delta + \frac{W}{2L}$$

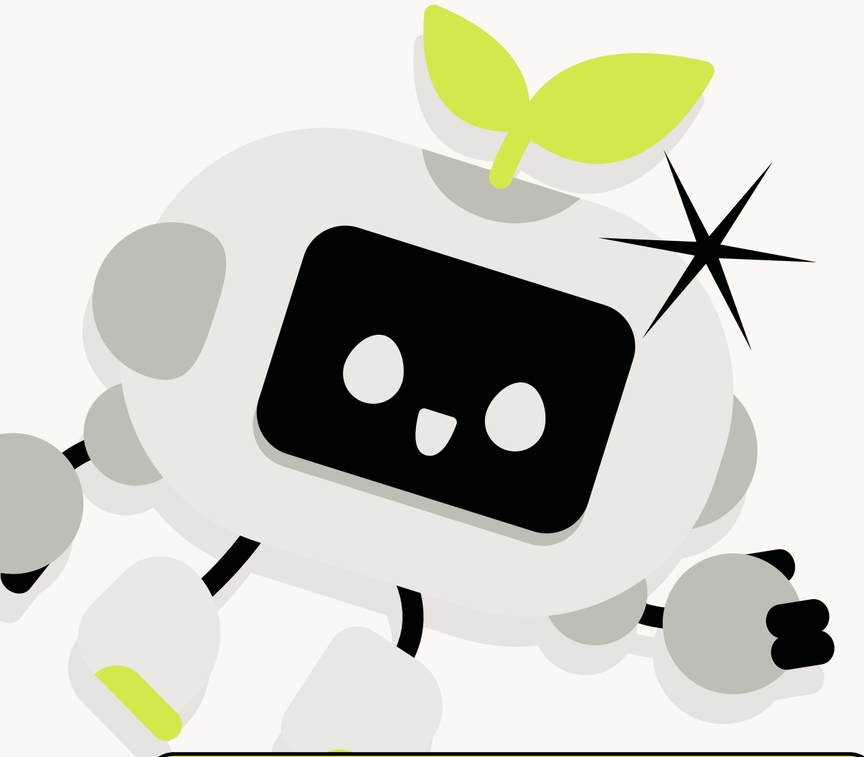
$$\delta_L = \operatorname{atan}\left(\frac{2L\sin\delta}{2L\cos\delta - W\sin\delta}\right)$$

$$\delta_R = \operatorname{atan}\left(\frac{2L\sin\delta}{2L\cos\delta + W\sin\delta}\right)$$

L – Vehicle length
W – Vehicle width
v – Rear wheel speed
$\delta$ – Effective steering angle
$\delta_l$ – Left wheel angle
$\delta_r$ – Right wheel angle

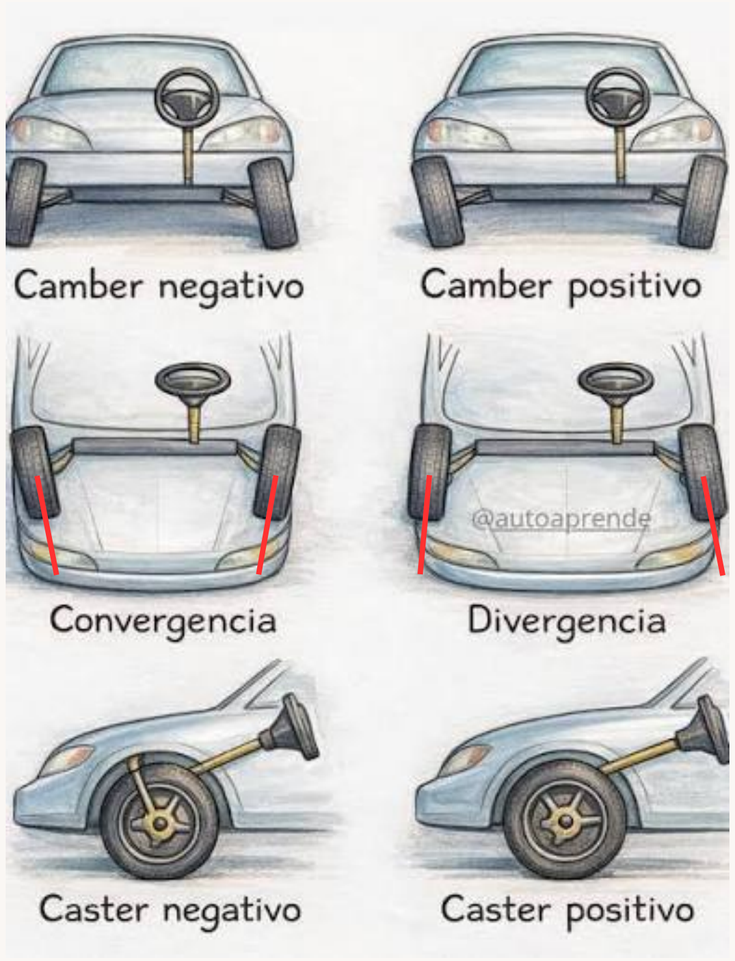
JOHNS HOPKINS WHITING SCHOOL of ENGINEERING

# MODELO CINEMÁTICO

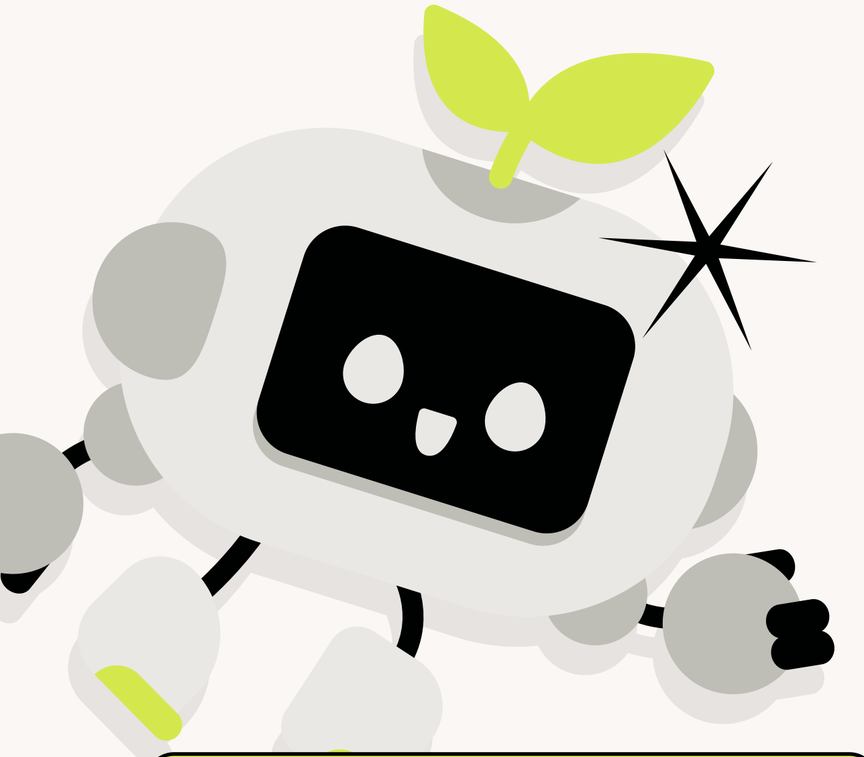


**EDMILSON**

1. Cambagem, Convergência, Caster
2. Suspensão
3. Aderência
4. . . .



# FORMULANDO UM PROBLEMA

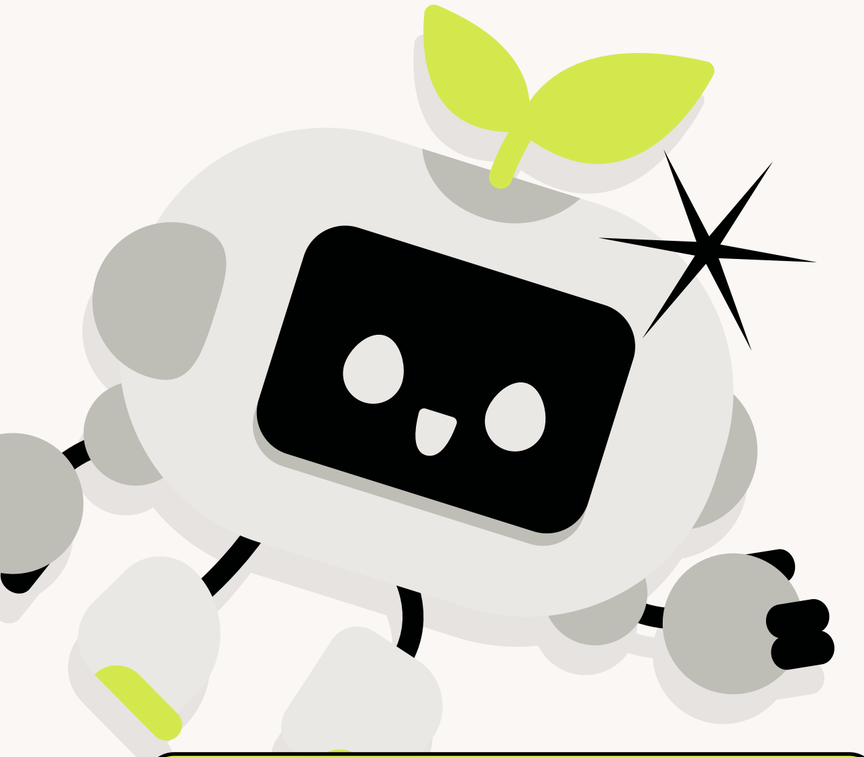


**EDMILSON**

Quais variáveis devem ser importantes?

1. Distancia entre rodas e parede
2. Velocidade
3. Pedal acelerador
4. Pedal de freio
5. Posição do volante

# FORMULANDO UM PROBLEMA



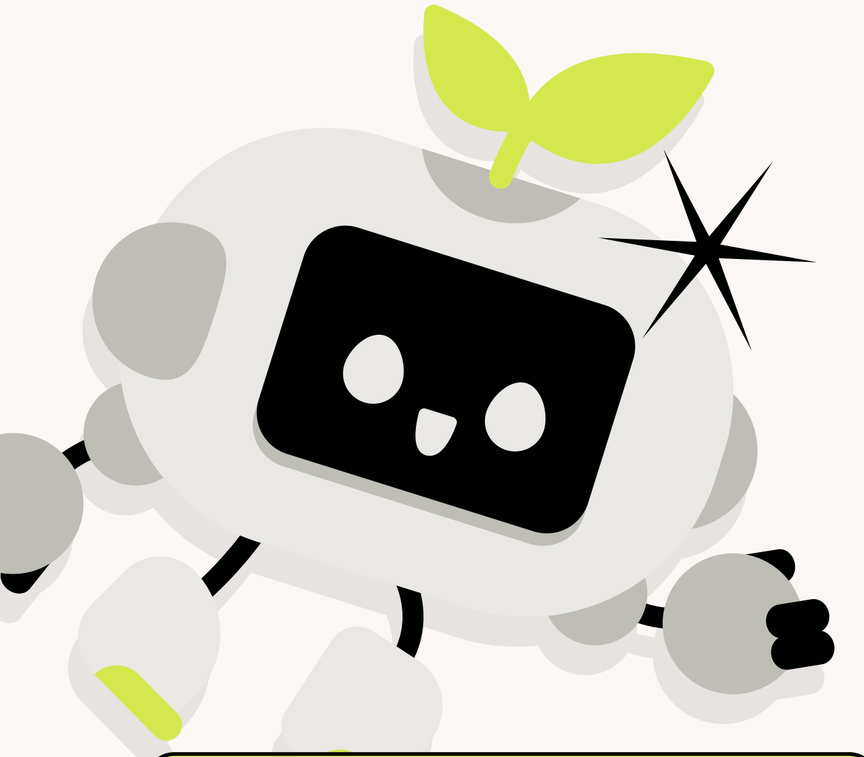
**EDMILSON**

Quais variáveis devem ser importantes?

1. Distancia entre rodas e parede
2. Velocidade
3. Pedal acelerador
4. Pedal de freio
5. Posição do volante

$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix} \Rightarrow$$

$$\Rightarrow \begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

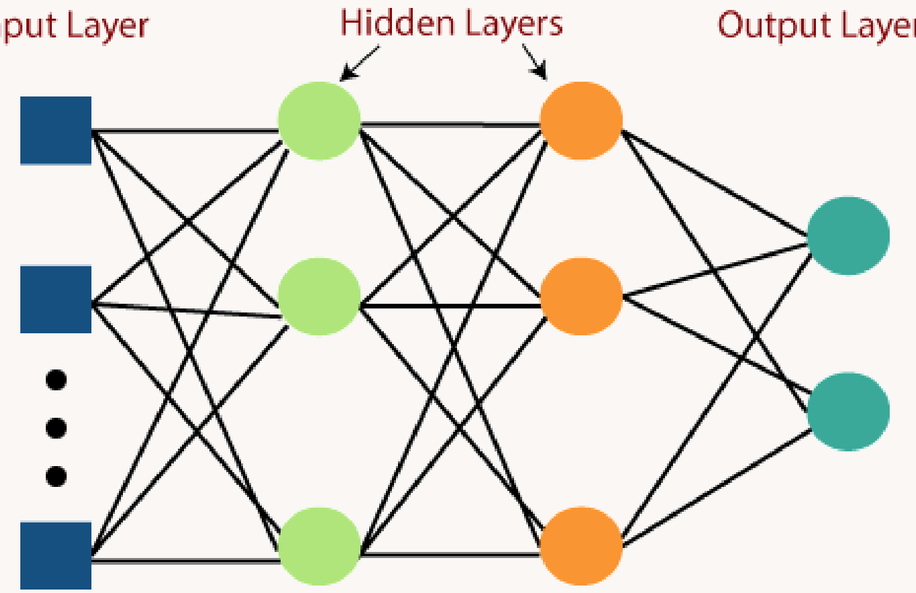


**EDMILSON**

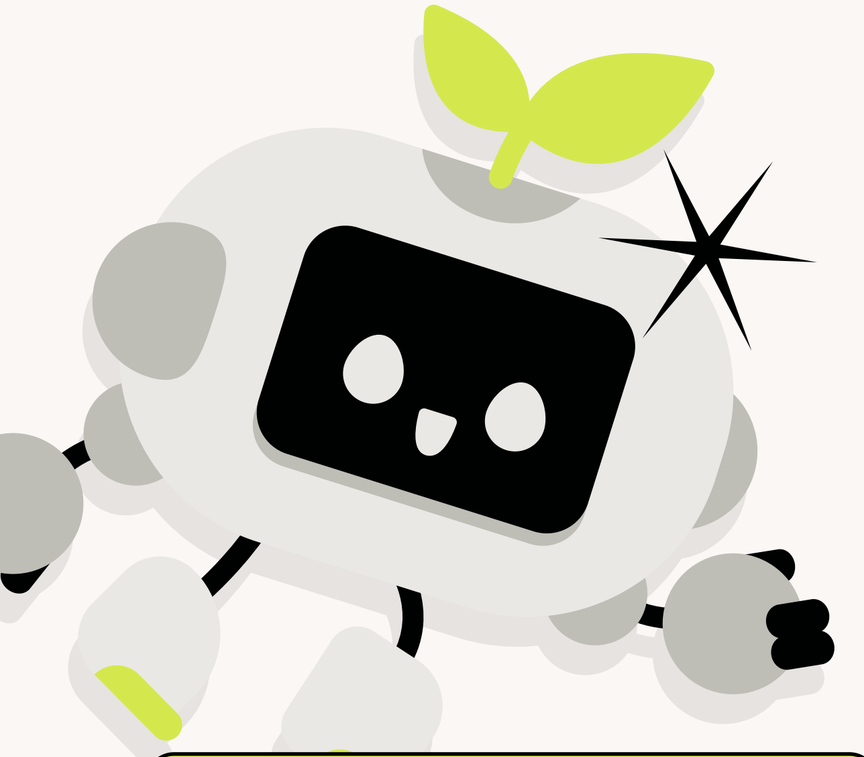
Cada variável importante dessa vai estar entre os **inputs** e **outputs** de uma **MLP**.

# MULTI LAYER PERCEPTRON

$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$




$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

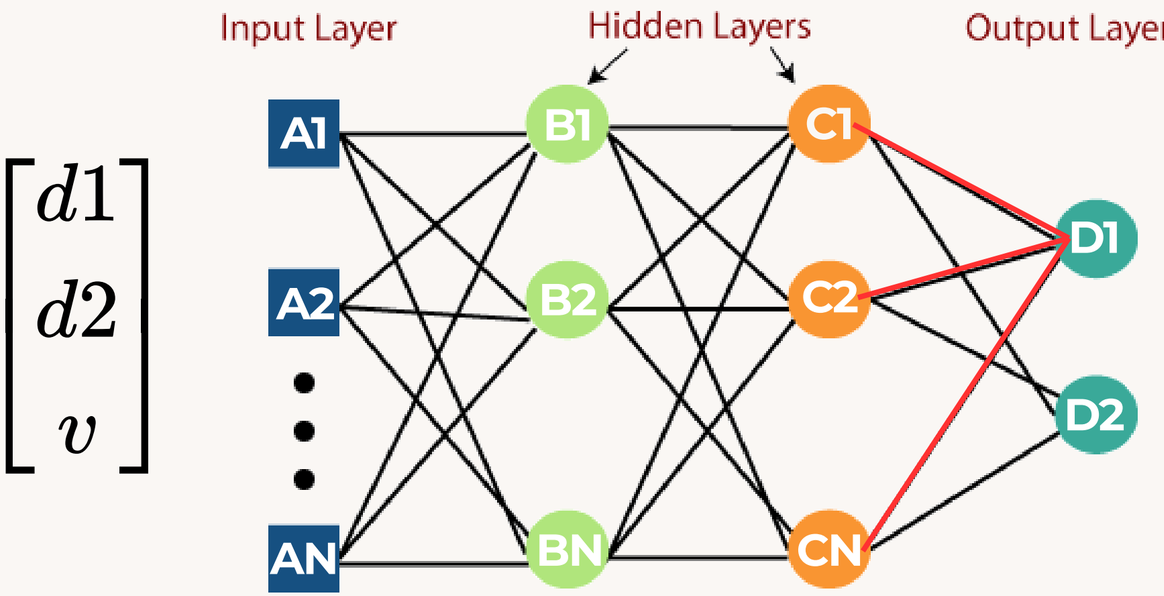


**EDMILSON**

D1 é uma função que combina todos as funções anterioras C.

# MULTI LAYER PERCEPTRON

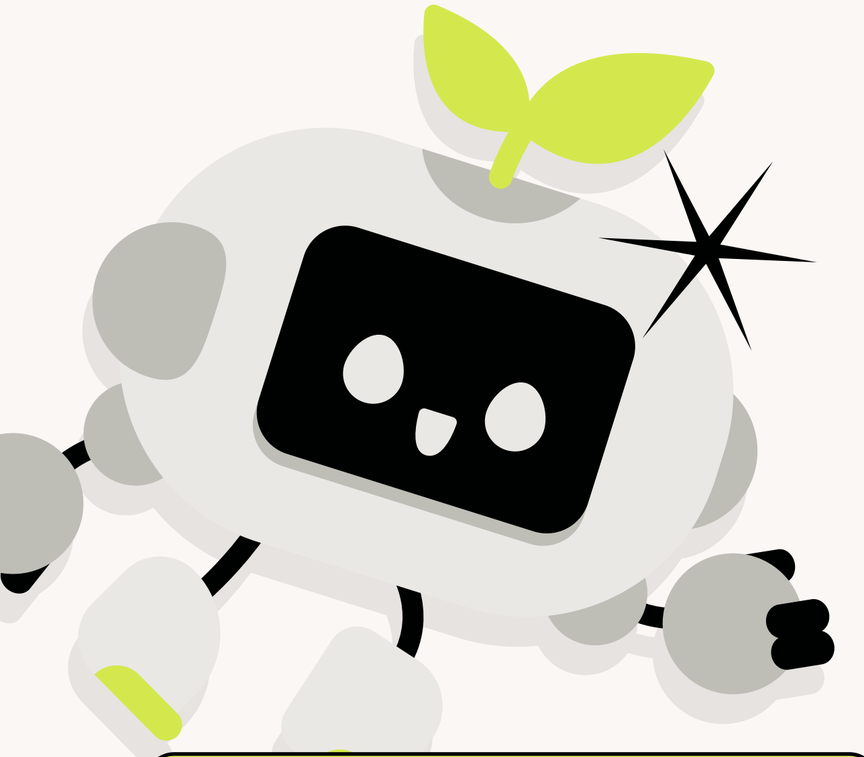
$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$




$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

**D1(C1, C2, ..., CN)**

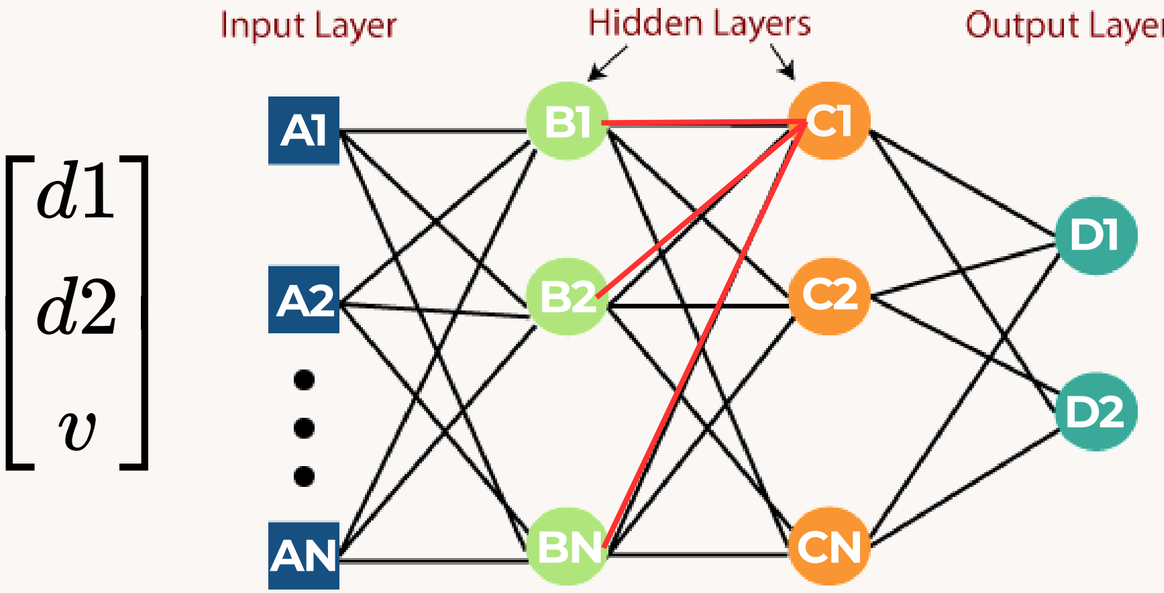
# MULTI LAYER PERCEPTRON



**EDMILSON**

C1 é uma função que combina todos as funções anterioras B.

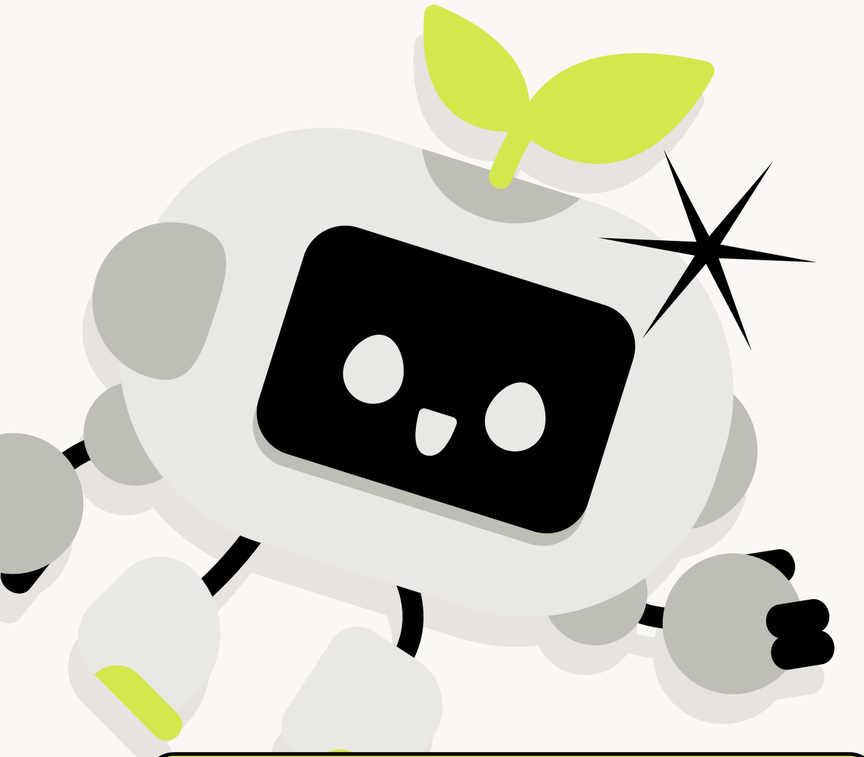
$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$




$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

D1(C1, C2, ..., CN)

**C1(B1, B2, ..., BN)**

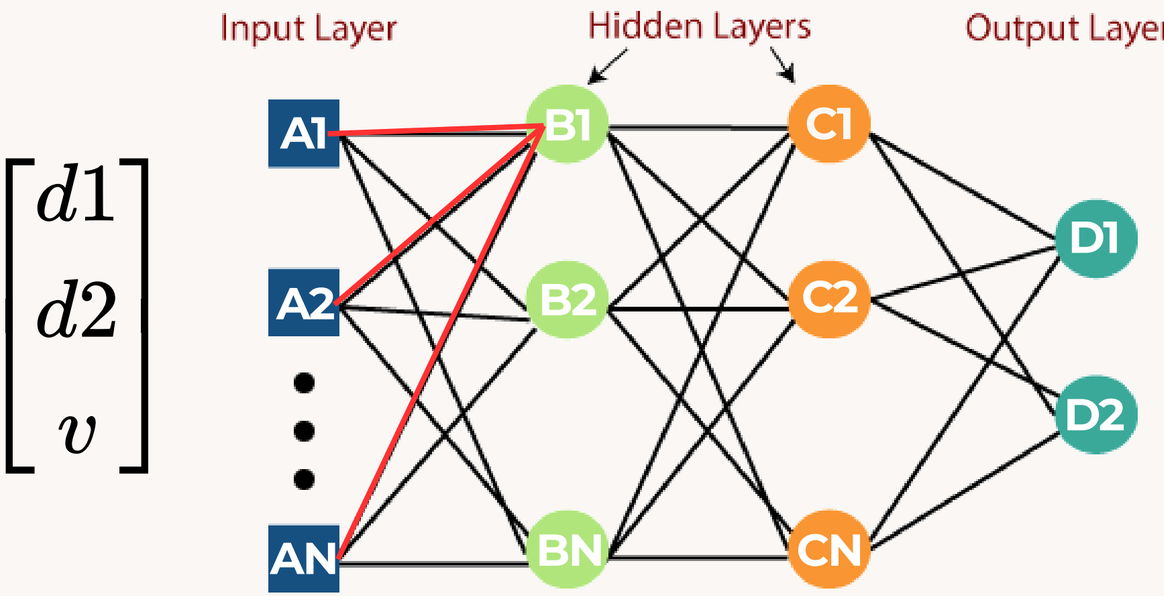


**EDMILSON**

B1 é uma função que combina todos as funções anterioras A.

# MULTI LAYER PERCEPTRON

$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$

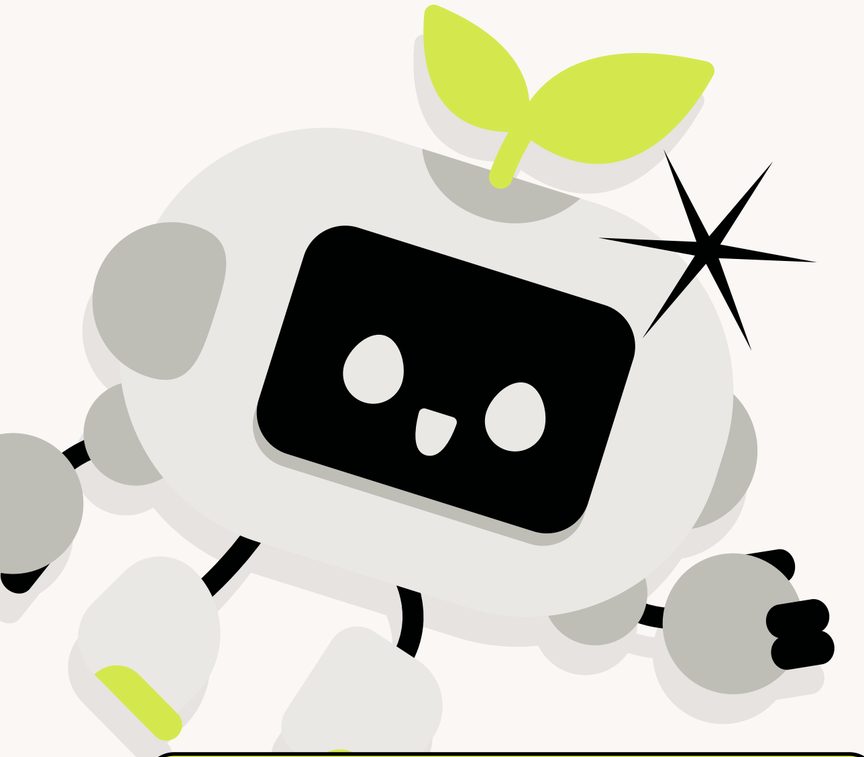



$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

D1(C1, C2, ..., CN)

C1(B1, B2, ..., BN)

**B1(A1, A2, ..., AN)**

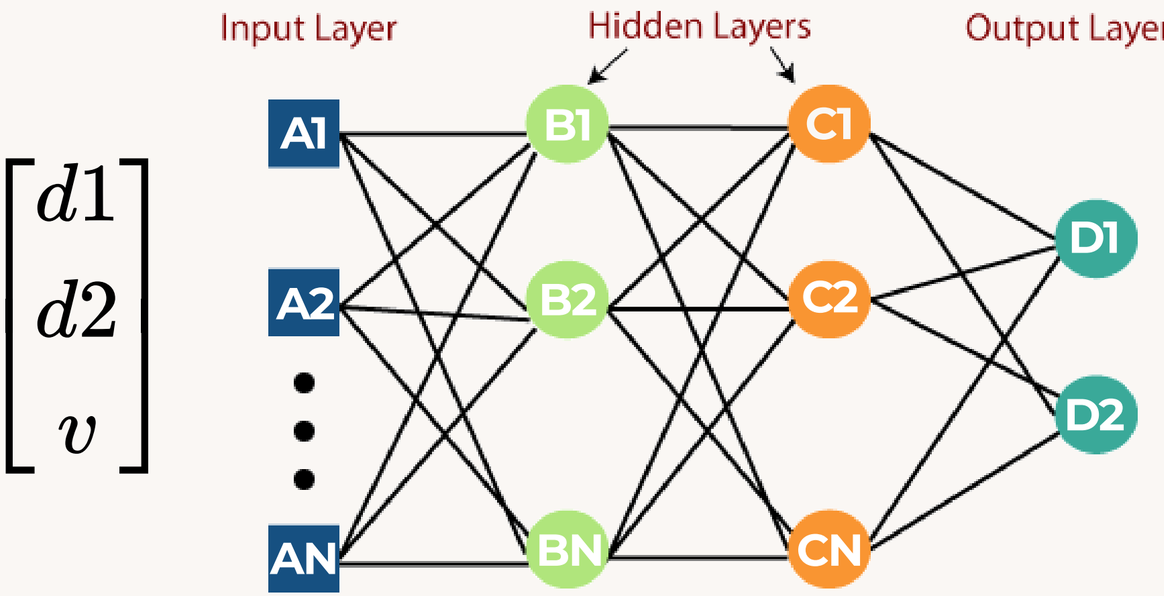


**EDMILSON**

**Extrapolando**: D1 é a combinação de todas as funções anteriores combinadas entre todas as outras camadas anteriores. Toda variável pode influenciar todas as saidas de certa forma.

# MULTI LAYER PERCEPTRON

$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$




$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

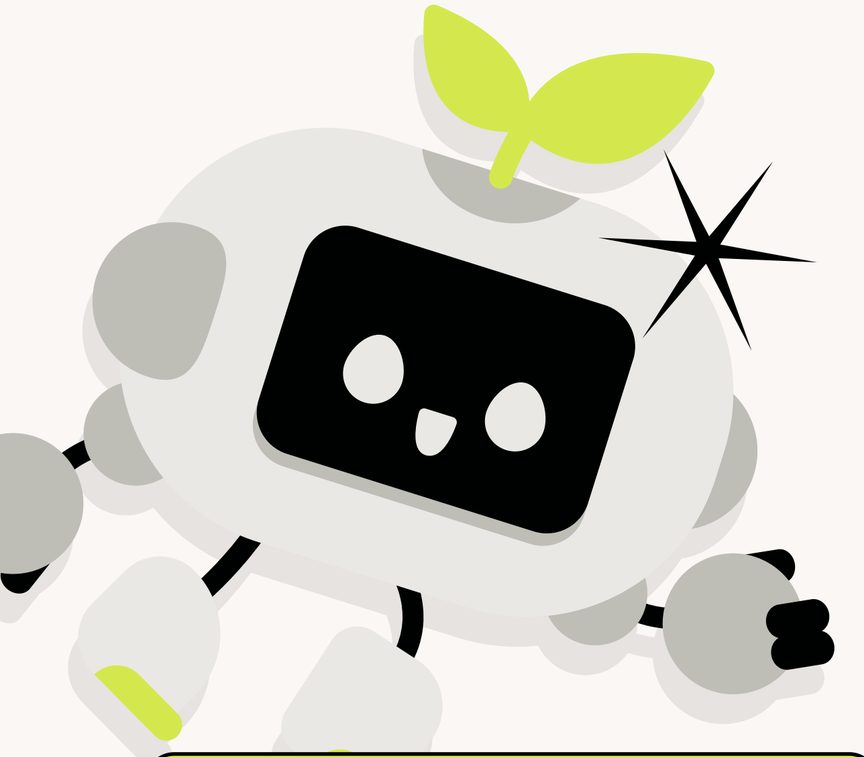
D1(C1, C2, ..., CN)

C1(B1, B2, ..., BN)

B1(A1, A2, ..., AN)

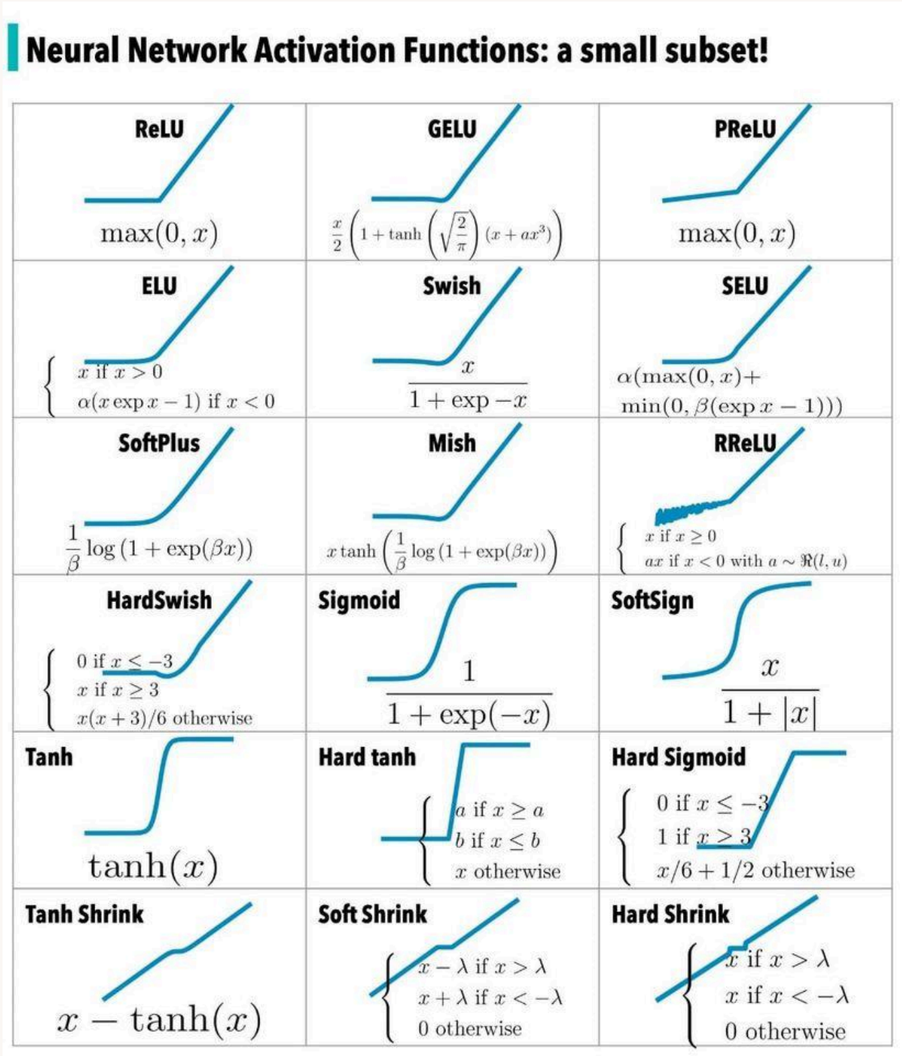
D1(C1(B1(A1, A2, ..., AN), B2, ..., BN), C2, ..., CN)

# FUNÇÕES DE ATIVAÇÃO

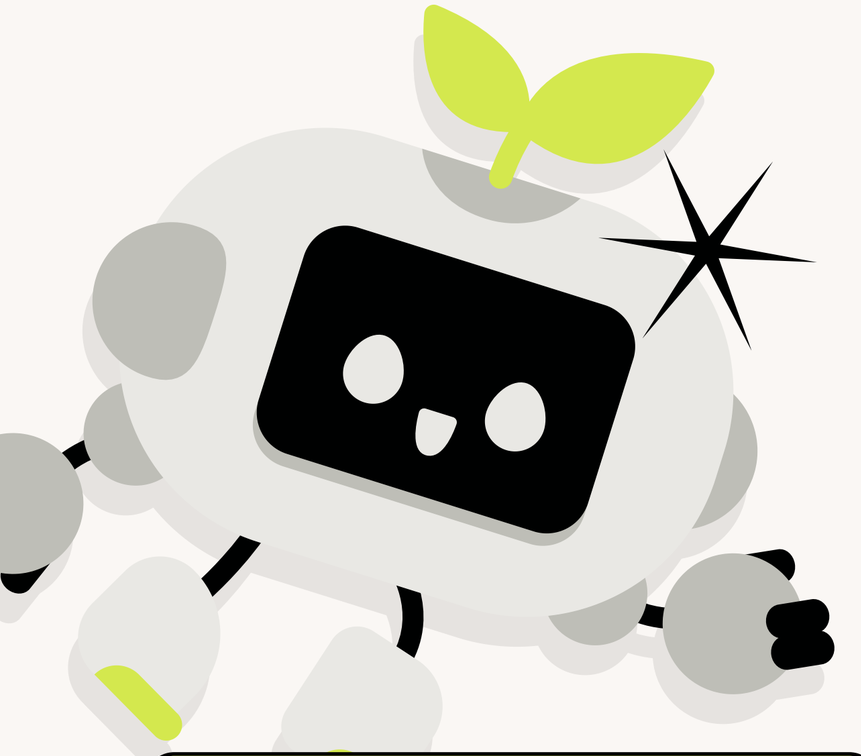


**EDMILSON**

Cada neurônio também possui uma função de ativação aleatória que diz se um sinal irá passar para os neurônios seguintes.



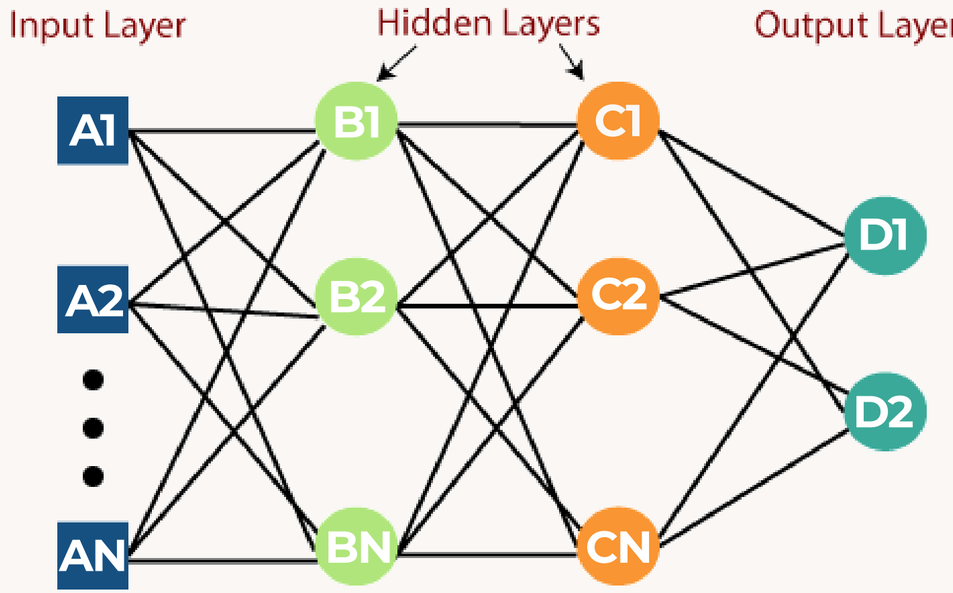
# TREINAMENTO



**EDMILSON**

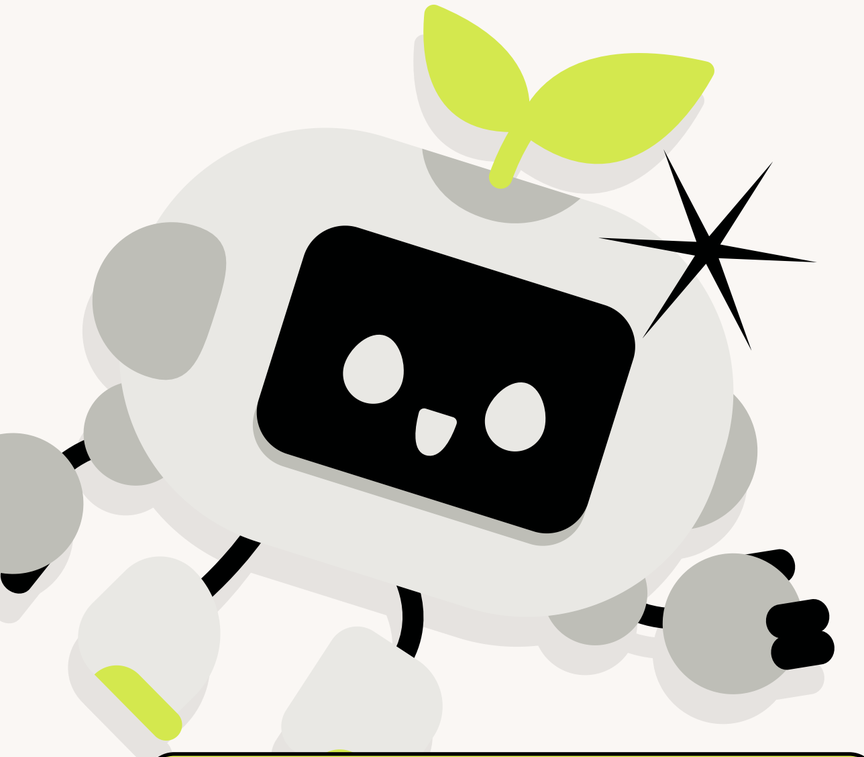
1. Executa a MLP várias e várias vezes, onde cada vértice tem diferentes funções de ativação e diferentes ganhos;
2. Aquelas combinações que tiveram melhor resultado tem melhores notas;

$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$




$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

# TREINAMENTO



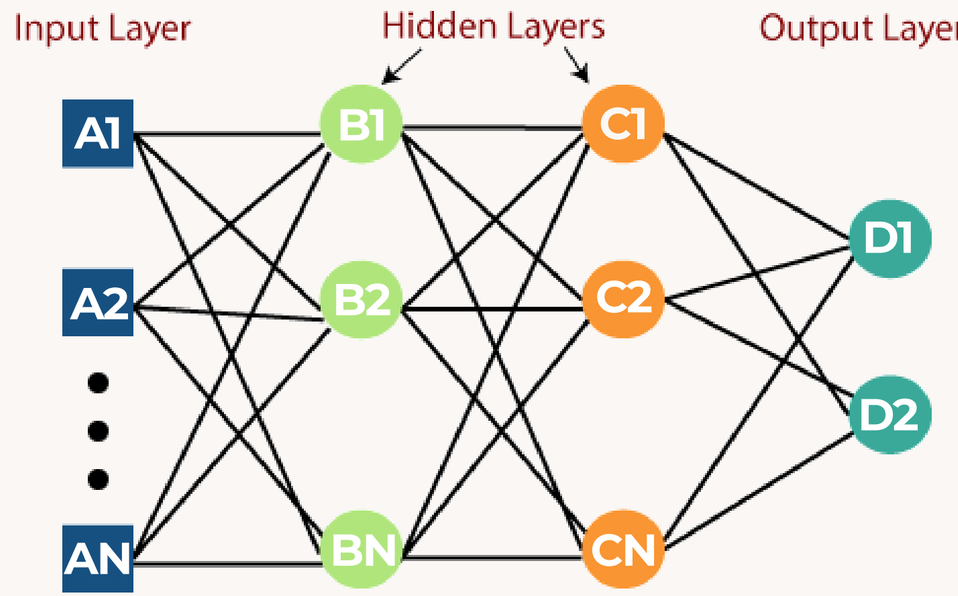
**EDMILSON**

Executa o algoritmo de **backpropagation**.

1. Obtém uma combinação diferente de pesos e funções de ativação;

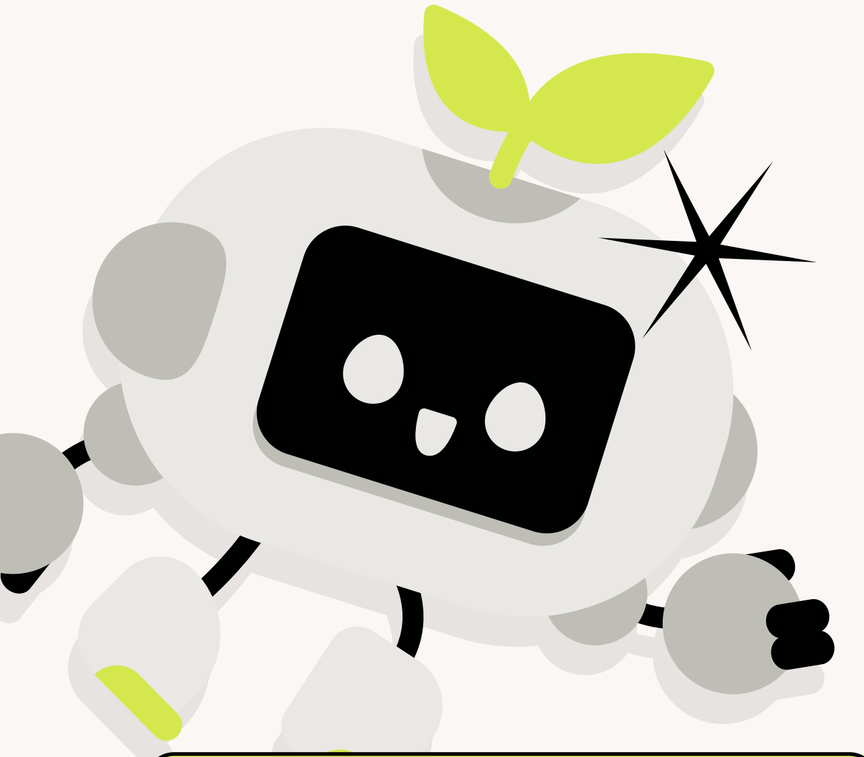
**Realiza o processo várias e várias vezes;**

$$\begin{bmatrix} d1 \\ d2 \\ v \end{bmatrix}$$



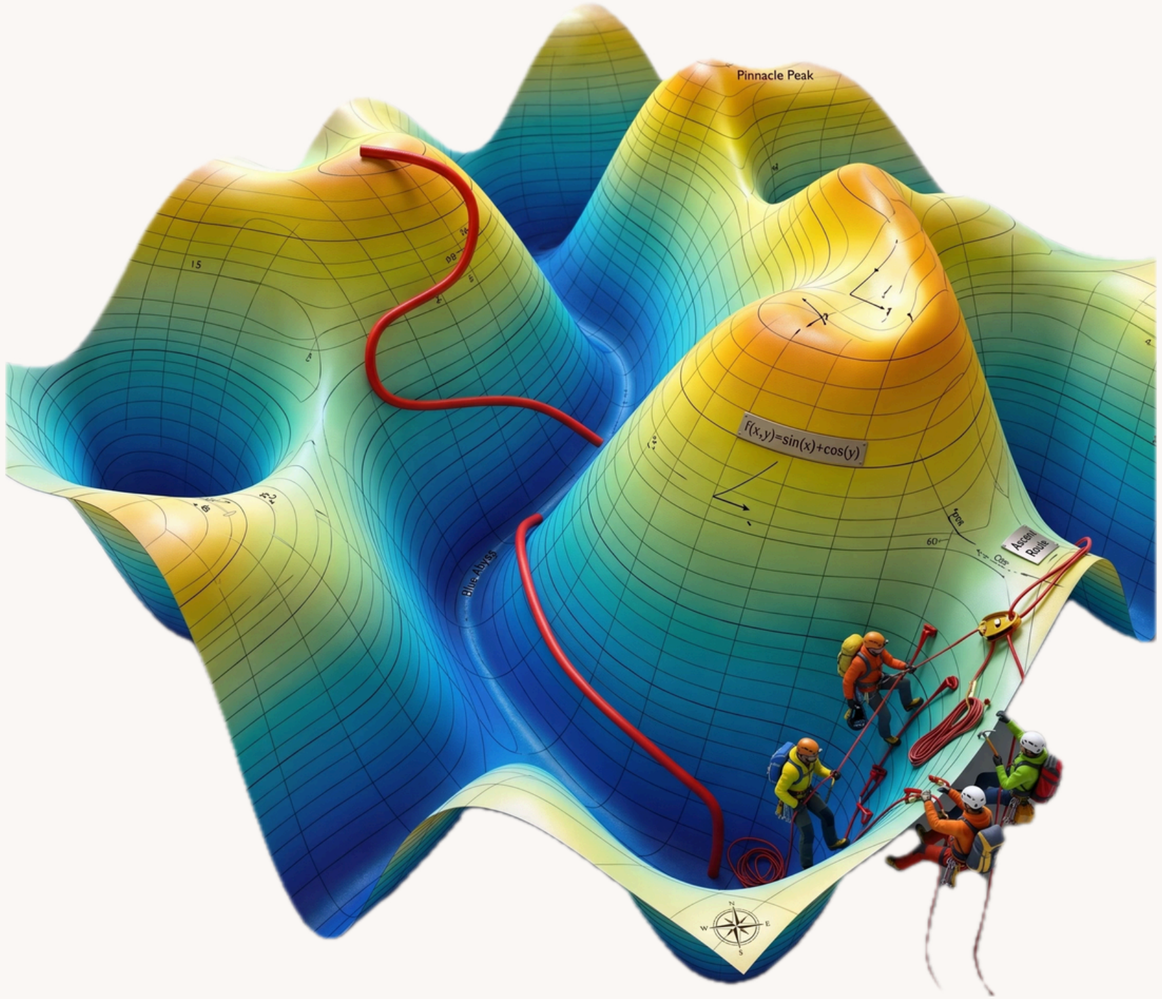

$$\begin{bmatrix} a \\ f \\ v \end{bmatrix}$$

# DESCIDA DO GRADIENTE



**EDMILSON**

As melhores soluções estão no fundo do vale, as piores no pico.
Ou ao contrario, depende do ponto de vista...



# TIPOS DE APRENDIZADO DE MÁQUINA

## SUPERVISIONADO

1. Classificação;
2. Regressão;

## NÃO SUPERVISIONADO

1. Agrupamento;
2. Associação;
3. Redução de Dimensionalidade;

## REFORÇO

1. Controle e Otimização;

## SEMI-SUPERVISIONADO

1. Rotulação Automática;

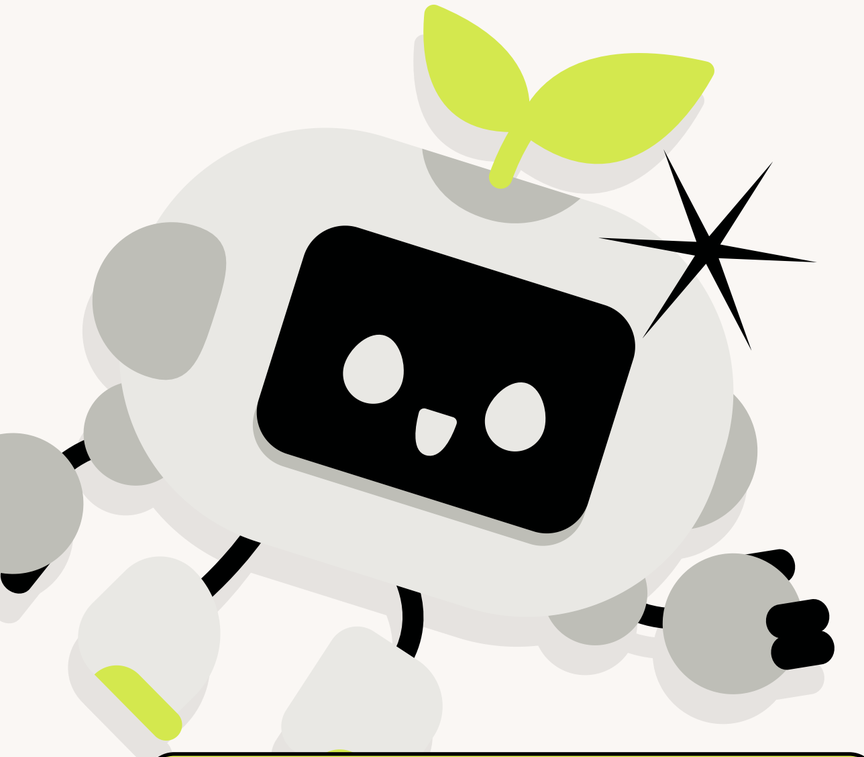
# HISTÓRICO

| | |
|---|---|
| 1943 | **NEURÔNIO DE MCCULLOCH-PITTS** |
| 1960 | **DESCIDA DO GRADIENTE** |
| 1986 | **MULTI LAYER PERCEPTRON** |

# LARGE LANGUAGE MODEL

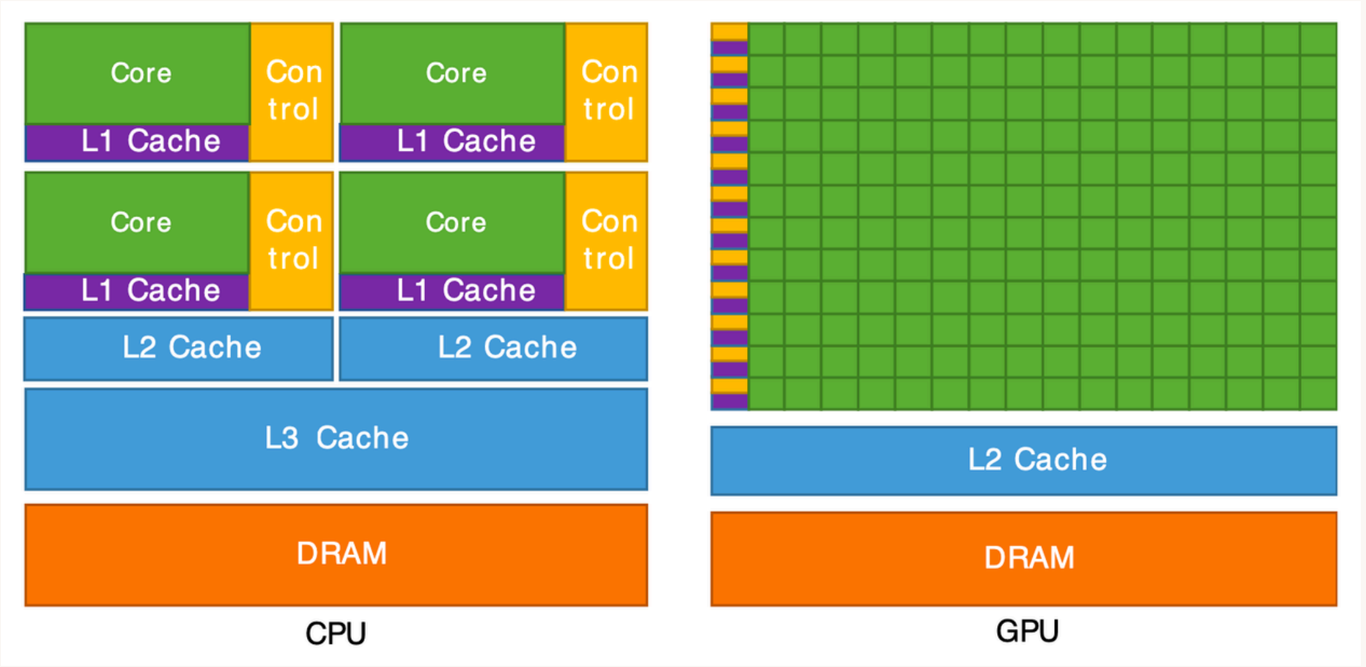
# E OS AGENTES DE IA

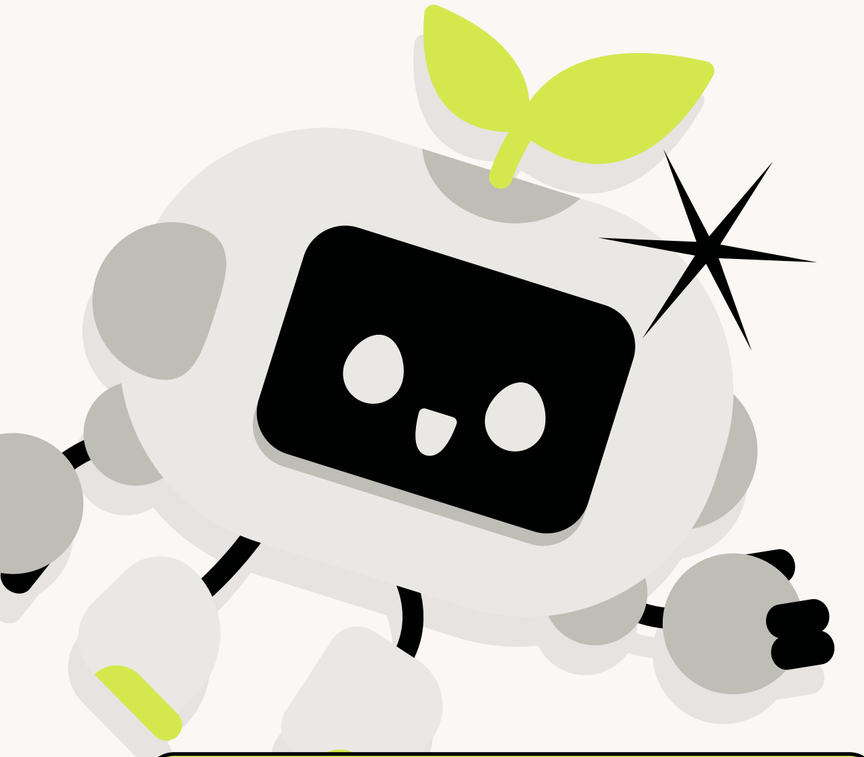
# CPU VS GPU



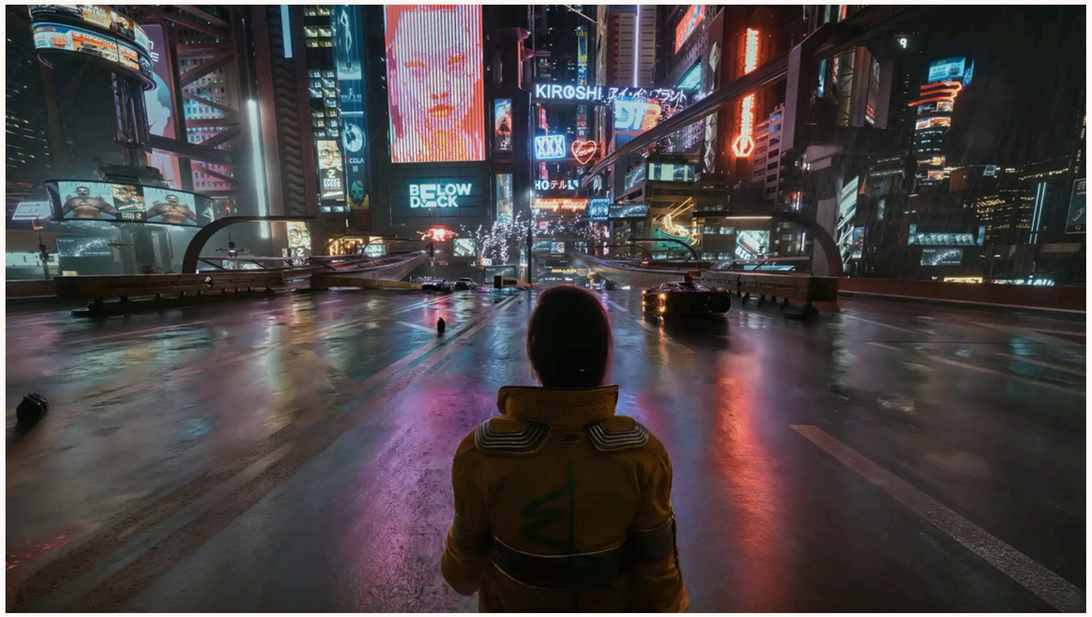
**EDMILSON**

1. CPUs conseguem fazer operações complexas.
2. GPUs realizam um número limitado de operações.
3. GPUs são ótimas para operações em matrizes.



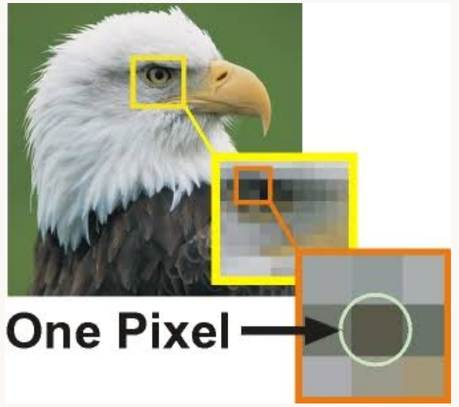


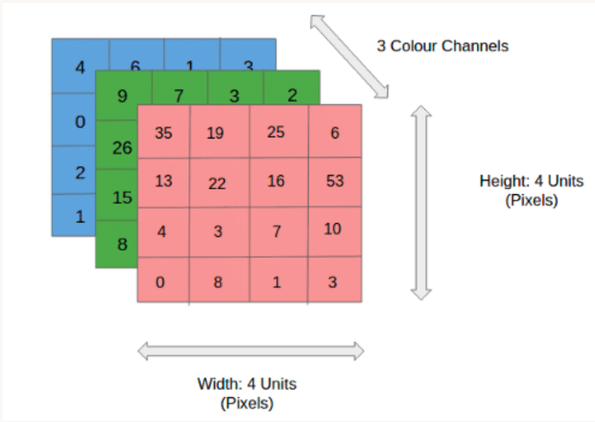
# IMAGENS VS MATRIZES

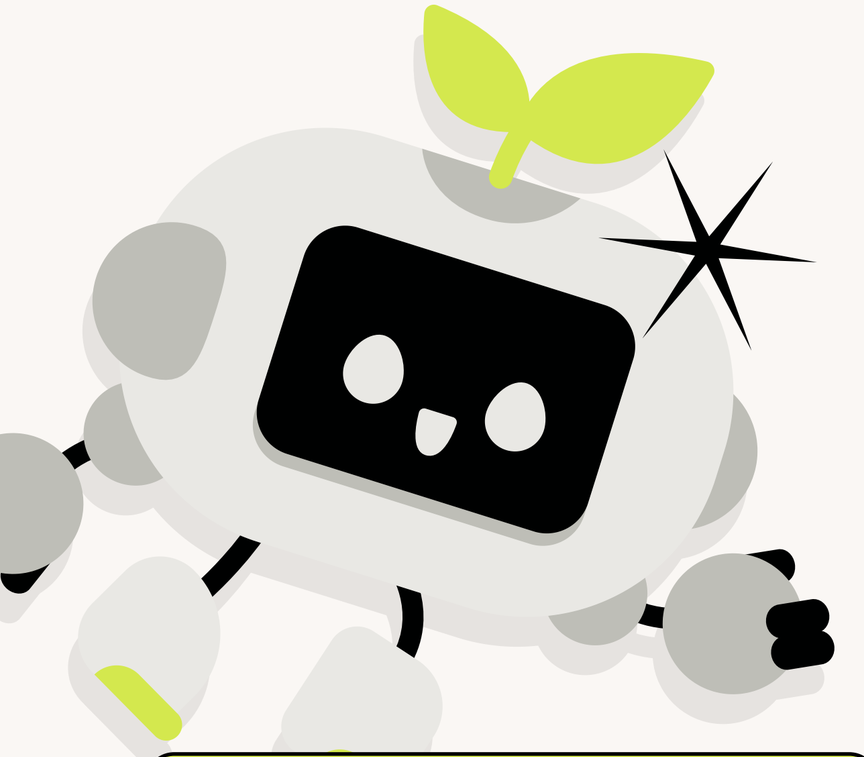



**EDMILSON**

1. Jogos renderizam imagens
2. Imagens são formadas por pixels
3. Cada pixel é um vetor de 3 dimensões: RGB
4. Uma imagem é a combinação de três matrizes RGB



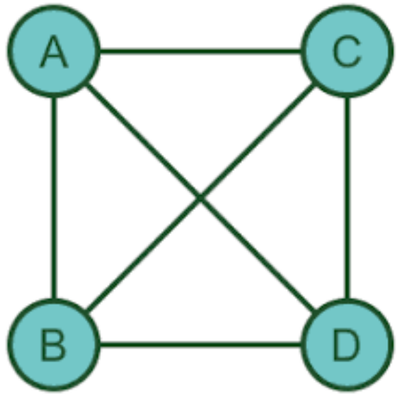





# GRAFOS VS MATRIZES

**EDMILSON**

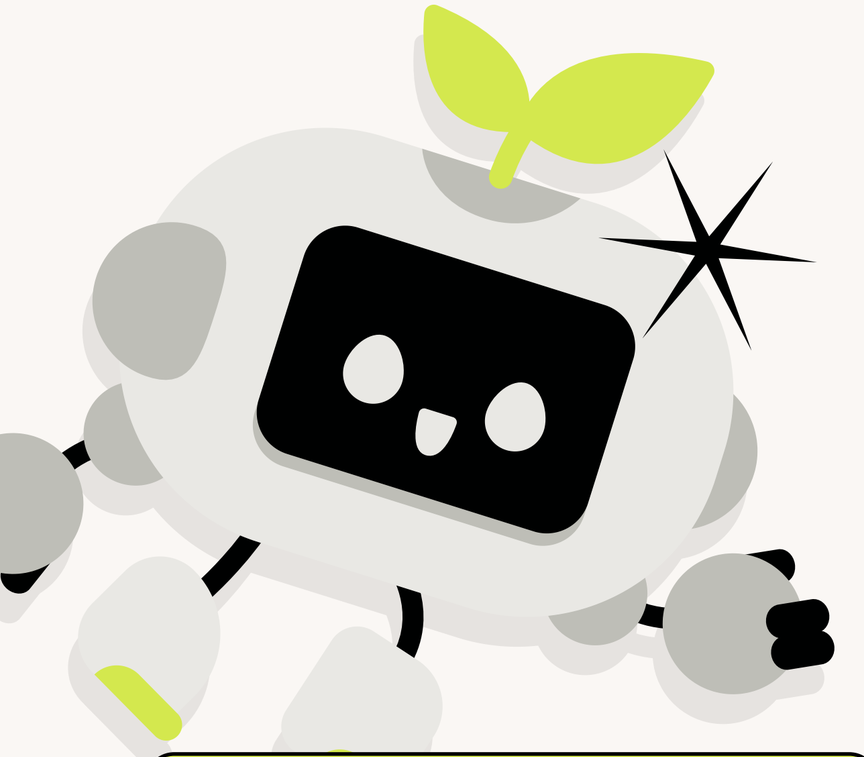
1. Redes neurais são em essência grafos.
2. Grafos podem ser representados por matrizes de incidência/adjacência.
3. **Por isso você paga caro em vídeo game e placas de vídeo hoje em dia.**




Complete graph

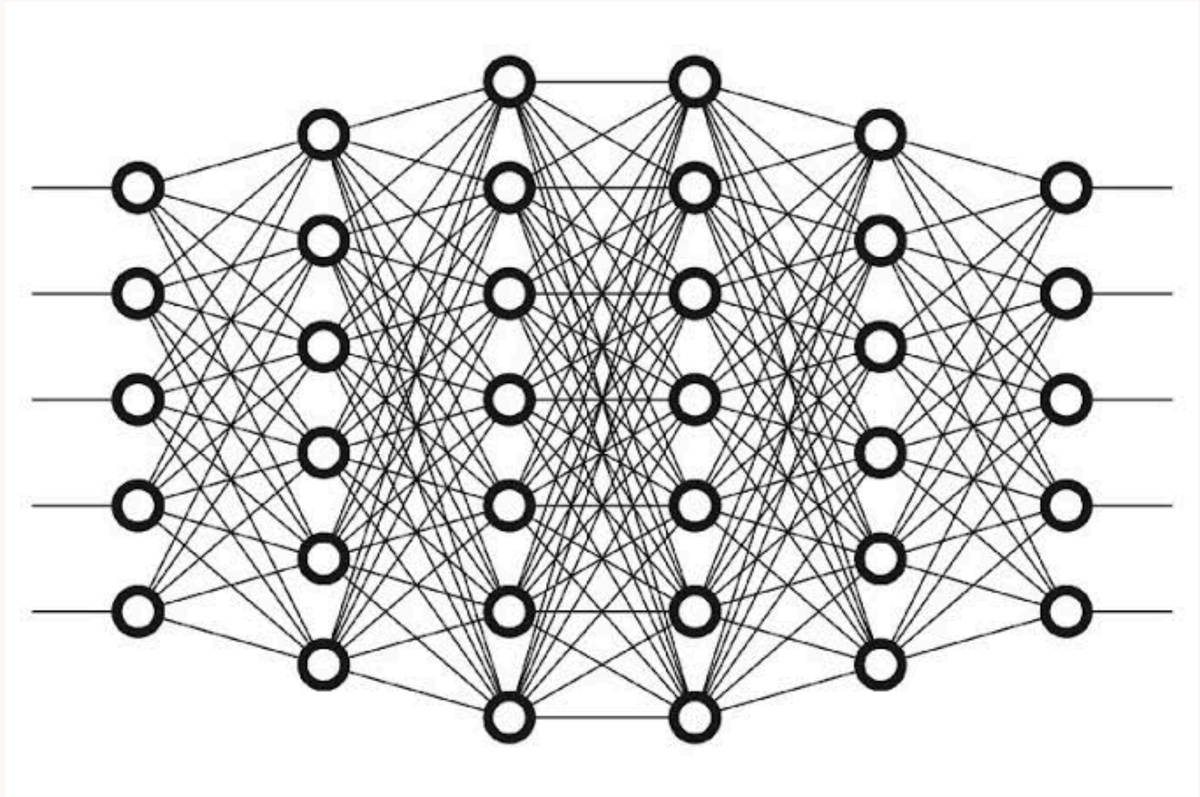
| | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 |
| B | 1 | 0 | 1 | 1 |
| C | 1 | 1 | 0 | 1 |
| D | 1 | 1 | 1 | 0 |

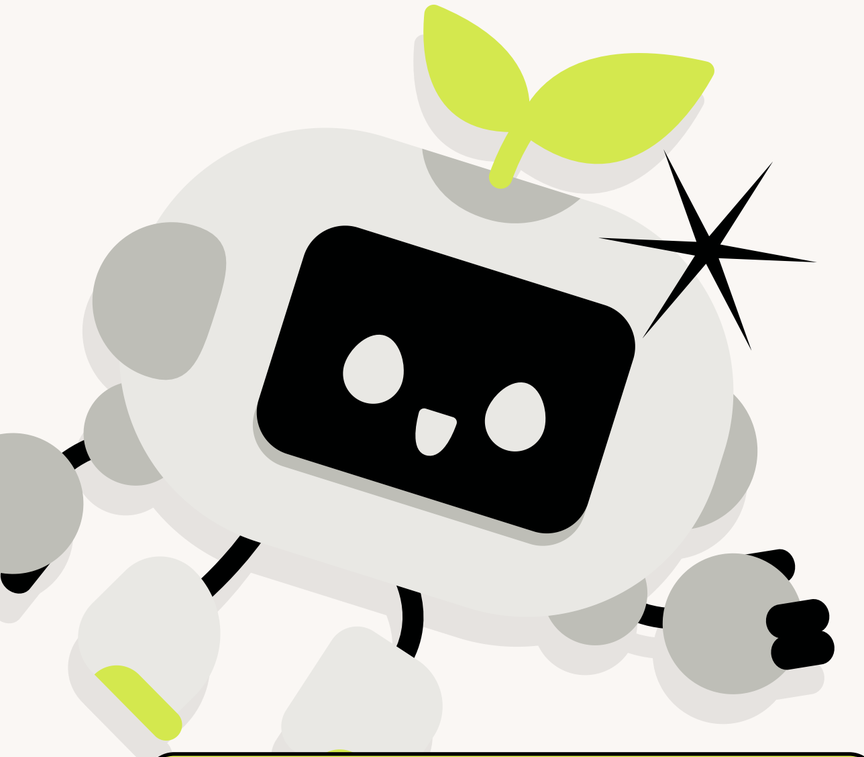
# DEEP LEARNING



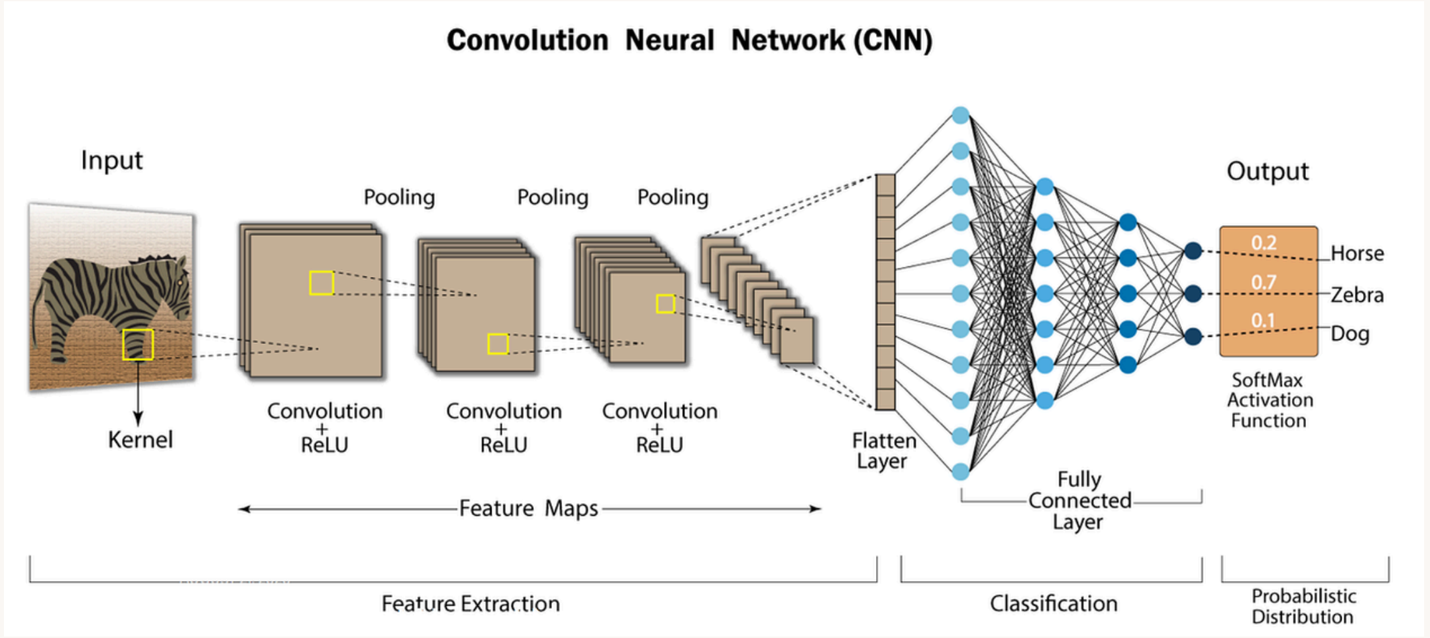
**EDMILSON**

1. É utilizado uma quantidade gigantesca de camadas.
2. Ocorre uma explosão combinatória.
3. Com isso é possível aprender coisas que com uma MLP não funciona.





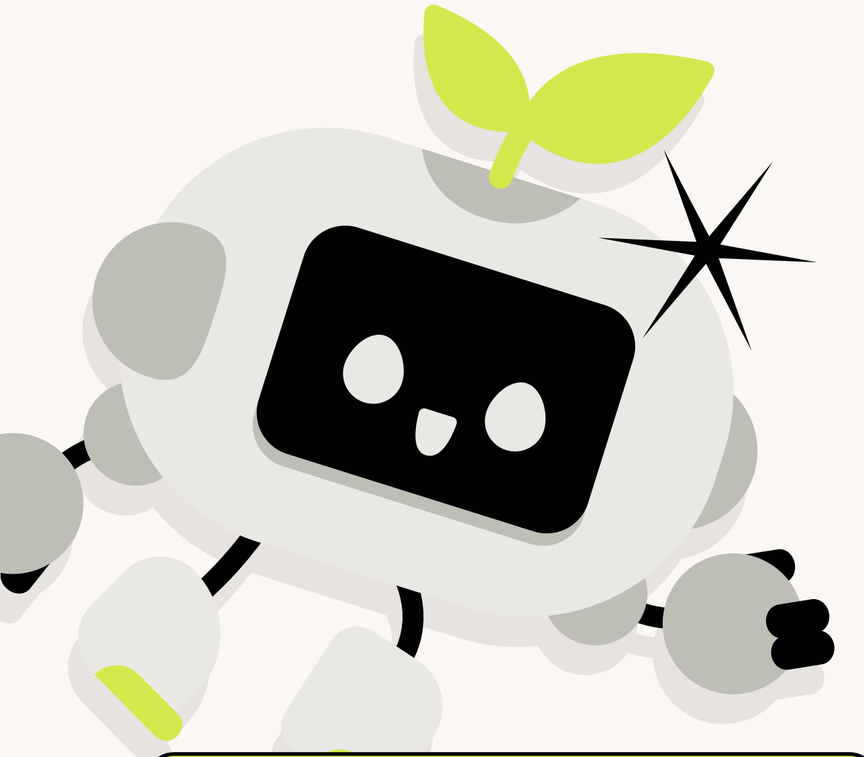
# REDES NEURAIS CONVOLUCIONAIS




**EDMILSON**

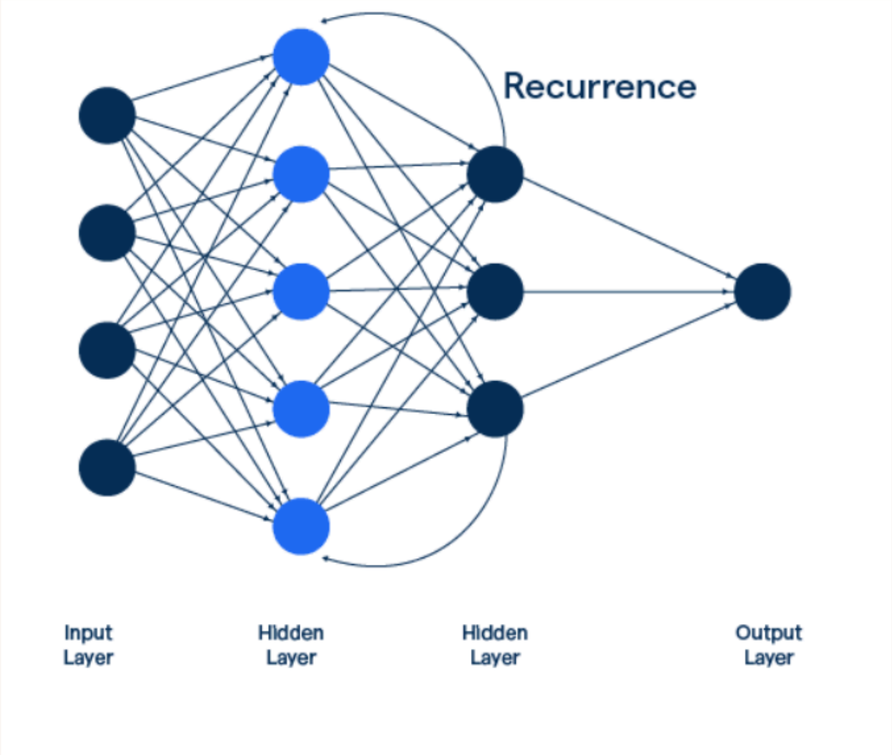
1. Revolucionário para o processamento de imagens.
2. Reconhecimento de objetos.
3. Reconhecimento de pessoas.

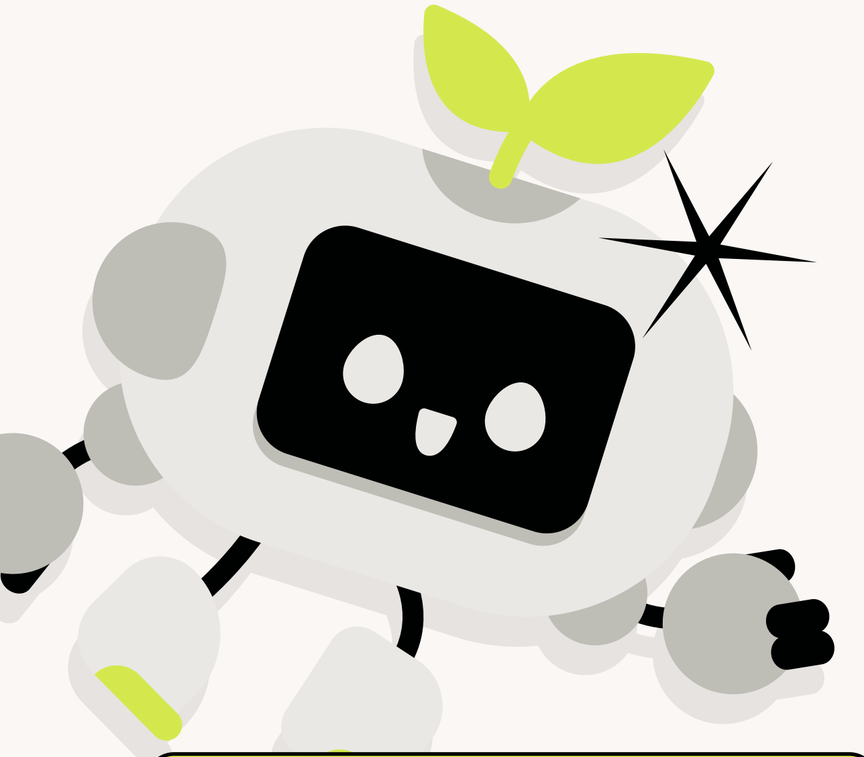
# REDES NEURAIS RECORRENTES



**EDMILSON**

1. Revolucionário para problemas onde a ordem importa.
2. Reconhecimento de fala.

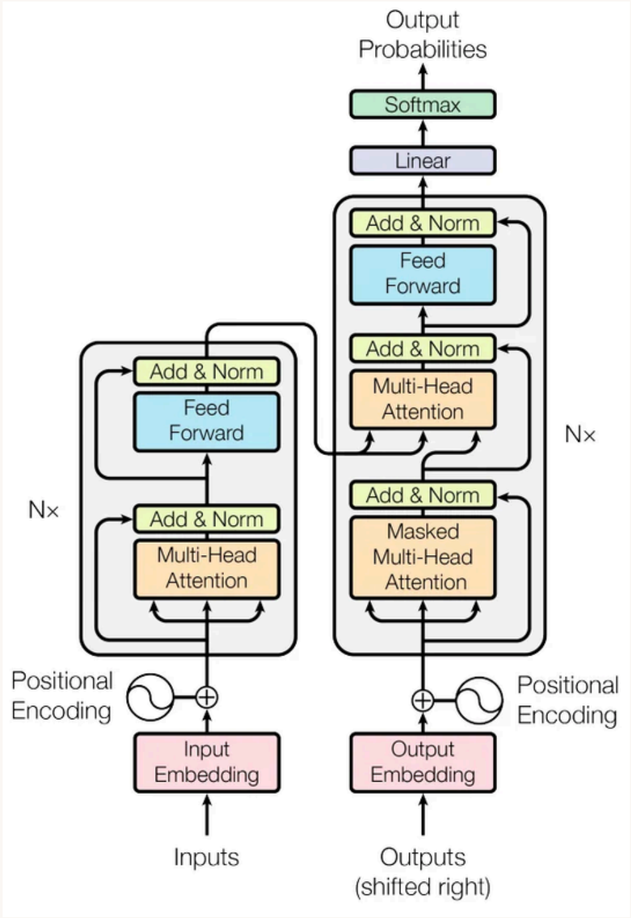




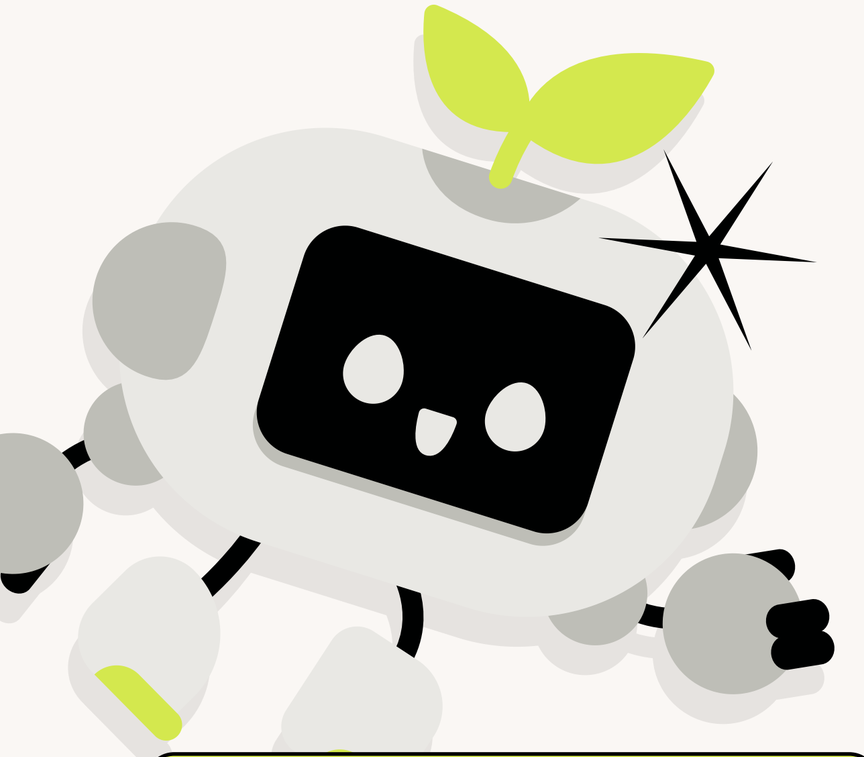
# TRANSFORMERS

## EDMILSON

1. Publicado inicialmente em **"Atenção é tudo o que você precisa"**, em 2017.
2. Revolucionário para o problema de geração de textos dentro de um contexto semântico.

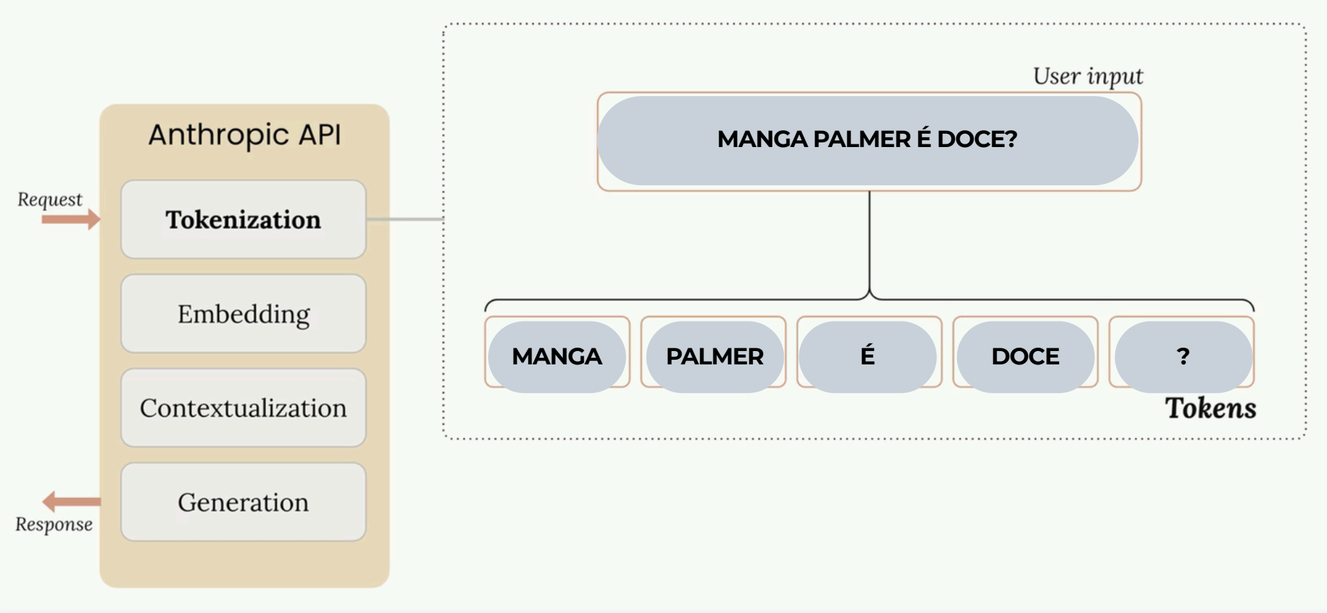


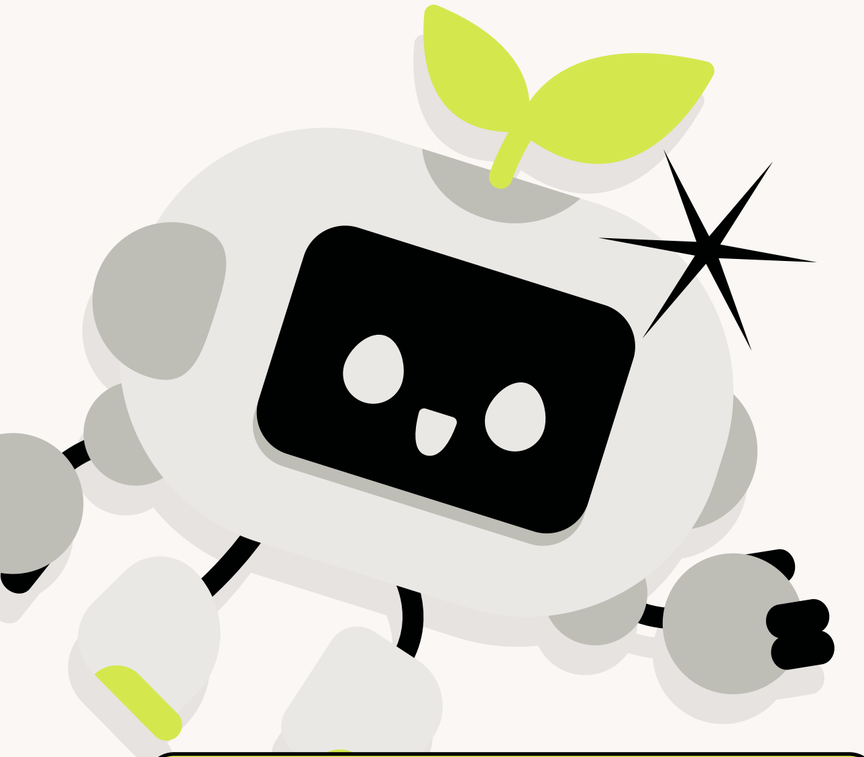
# LARGE LANGUAGE MODEL



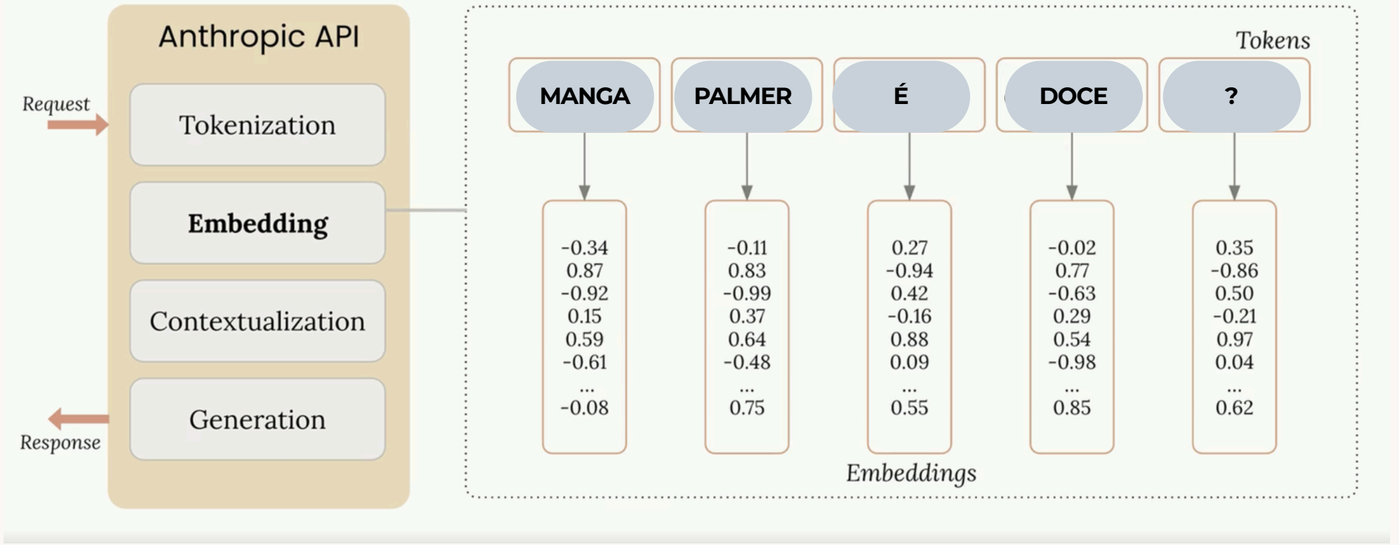
**EDMILSON**

1. Uma frase é quebrada em tokens.
2. É utilizado um algoritmo de Byte-Pair Encoding, WordPiece ou SentencePiece.
3. Cada token possui um ID dentro da LLM treinada.





# LARGE LANGUAGE MODEL

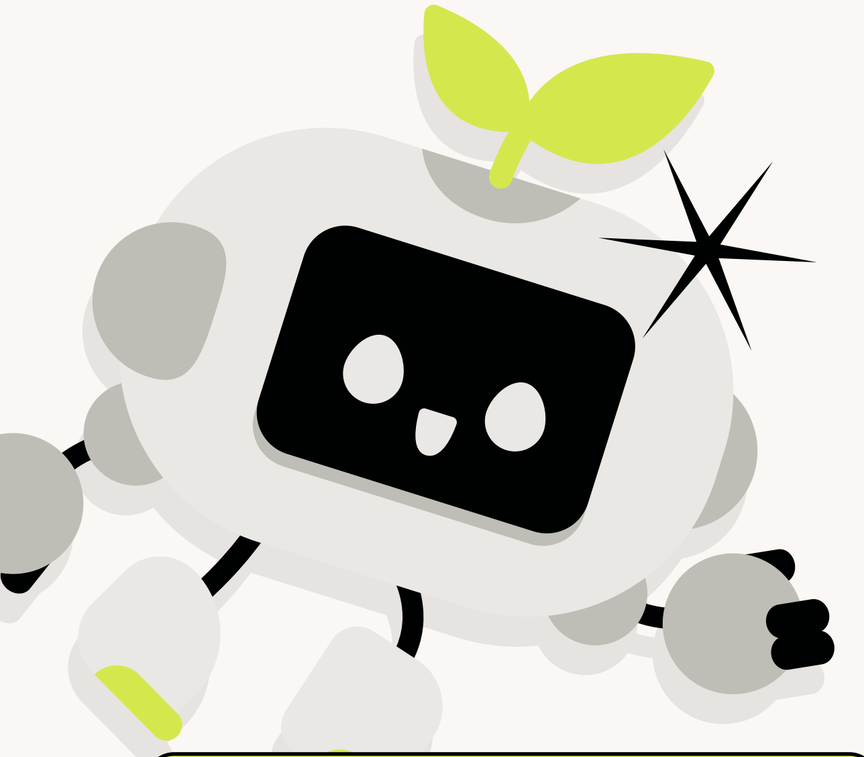



**EDMILSON**

1. Com base no ID, é realizado uma busca em um gigantesco "array" que está instanciado em memória.
2. Cada elemento no array carrega um vetor multi-dimensional da posição daquele token em um espaço vetorial.

**R1 => [X]**
**R2 => [X, Y]**
**R3 => [X, Y, Z]**

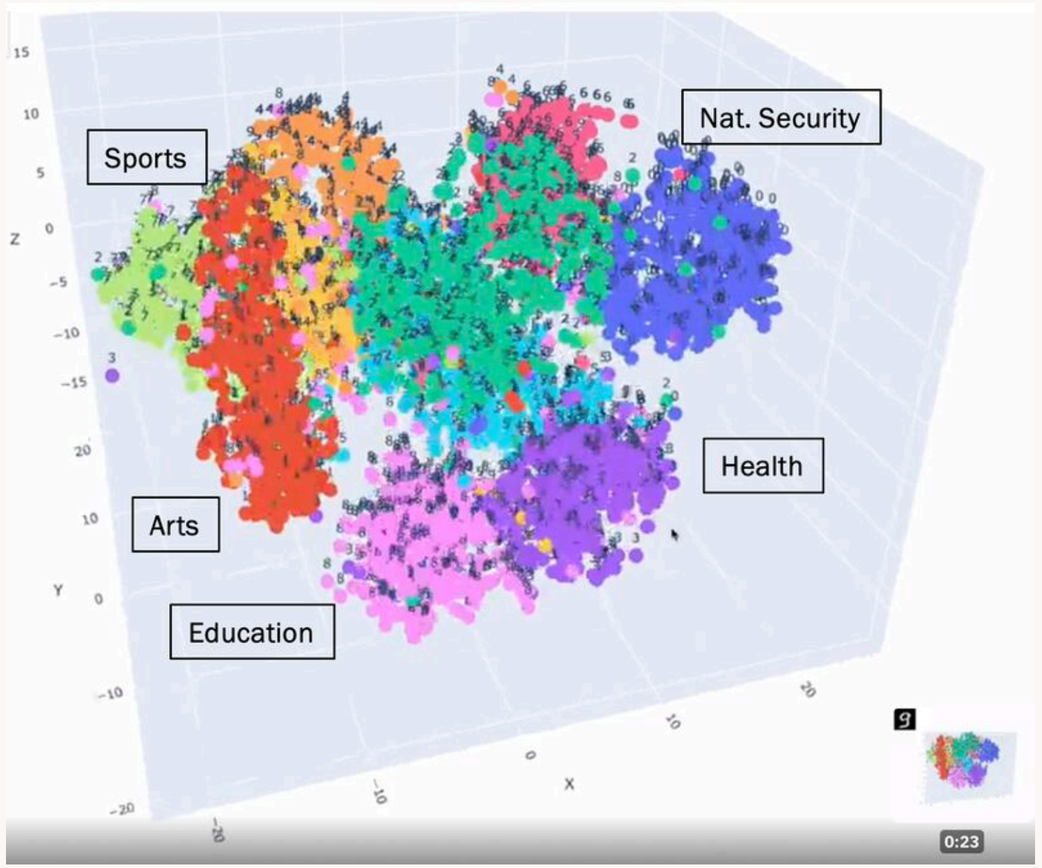
**RN = [X, Y, Z, ..., N]**

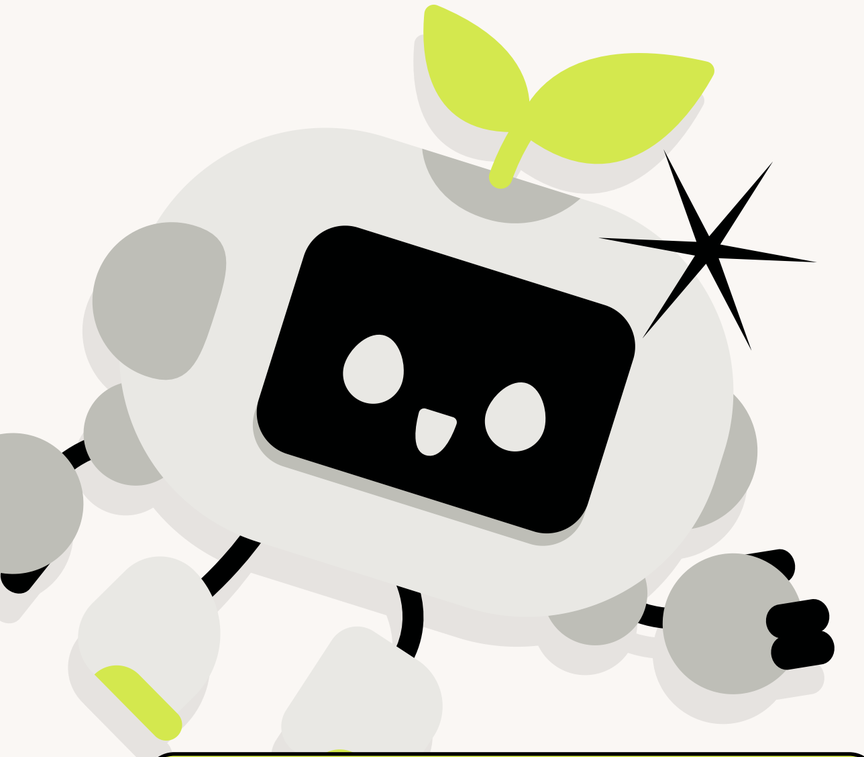


# LARGE LANGUAGE MODEL

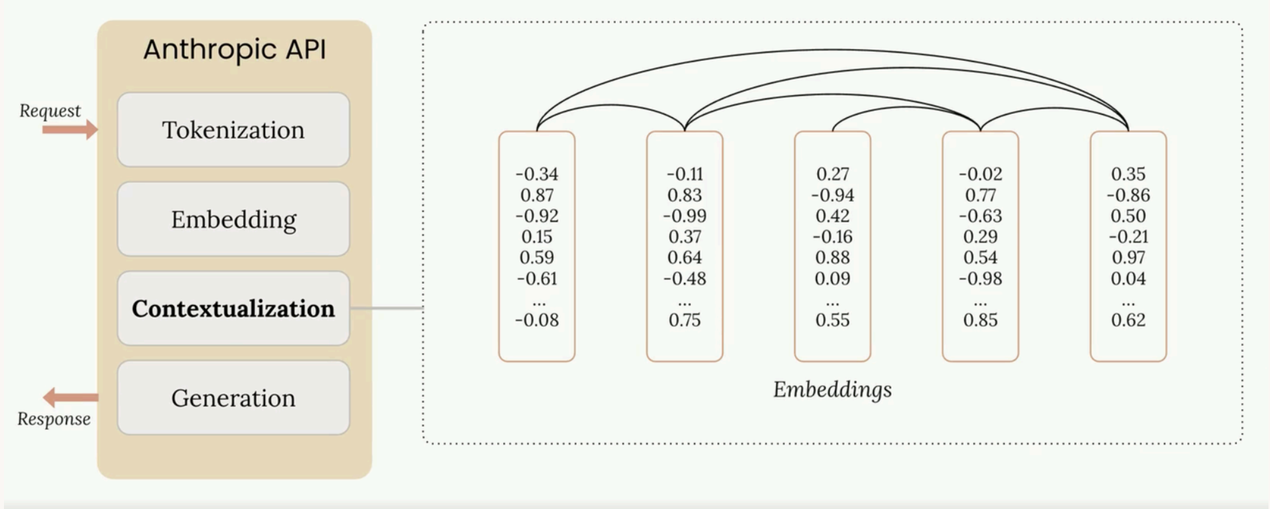
**EDMILSON**

1. Cada conceito, estará clusterizado em um espaço vetorial.





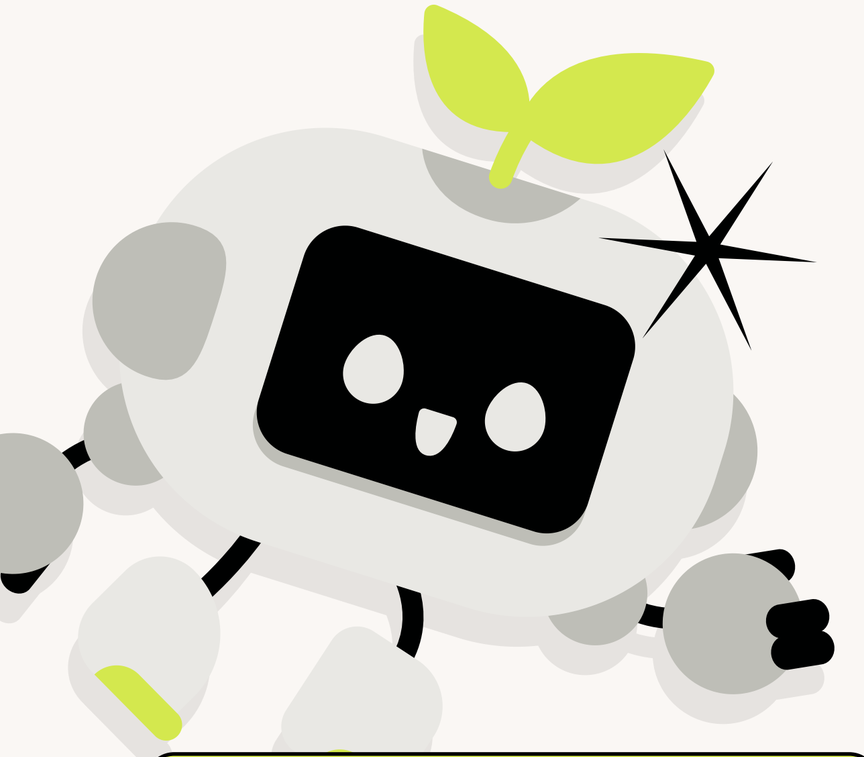
# LARGE LANGUAGE MODEL



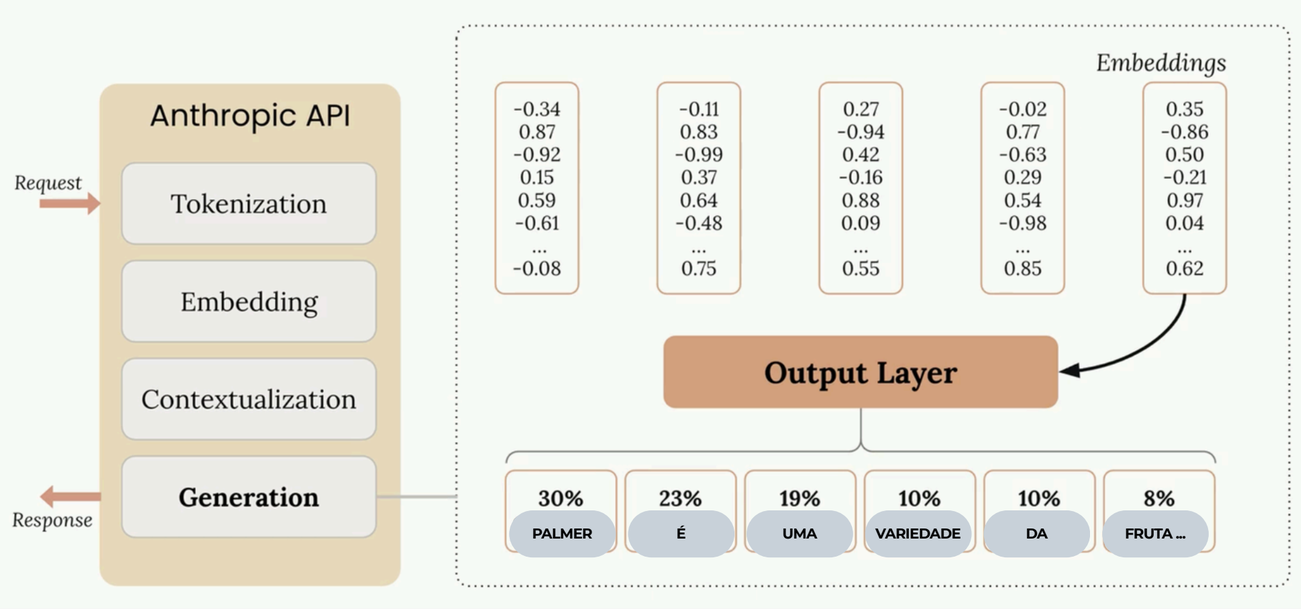

Manga fruta, ou manga de roupa?

**EDMILSON**

1. Contextualização realiza uma combinação das palavras na sentença.
2. Ela muda os valores de cada embedding com base nas palavras mais importantes.
3. Algoritmo da Atenção

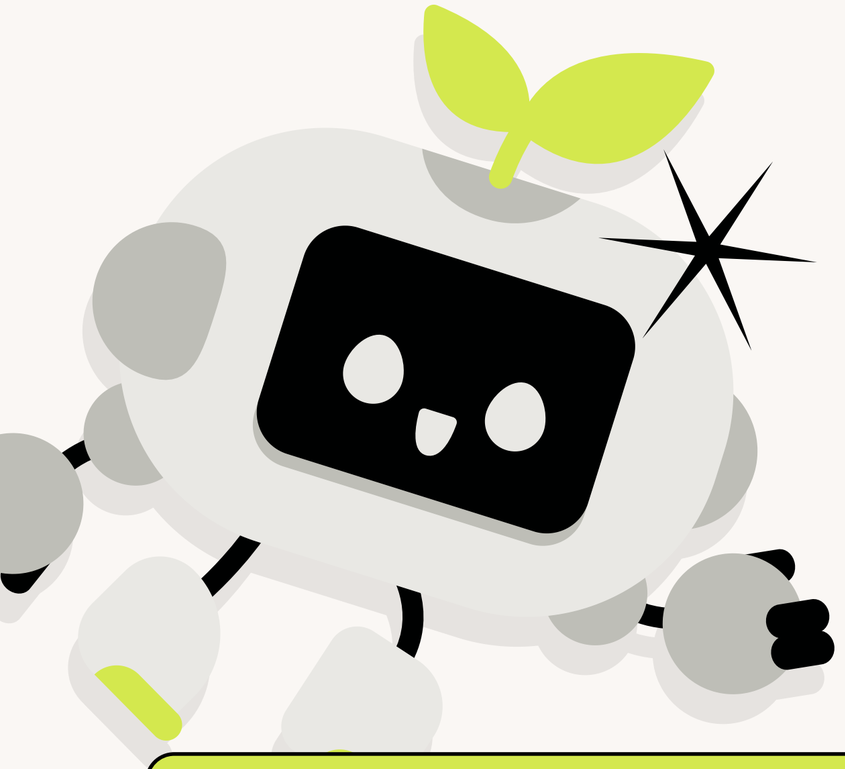


# LARGE LANGUAGE MODEL




**EDMILSON**

1. Após todas as etapas, é realizado a geração de texto.
2. A geração é aleatória e tenta completar a frase enviada.



**EDMILSON**

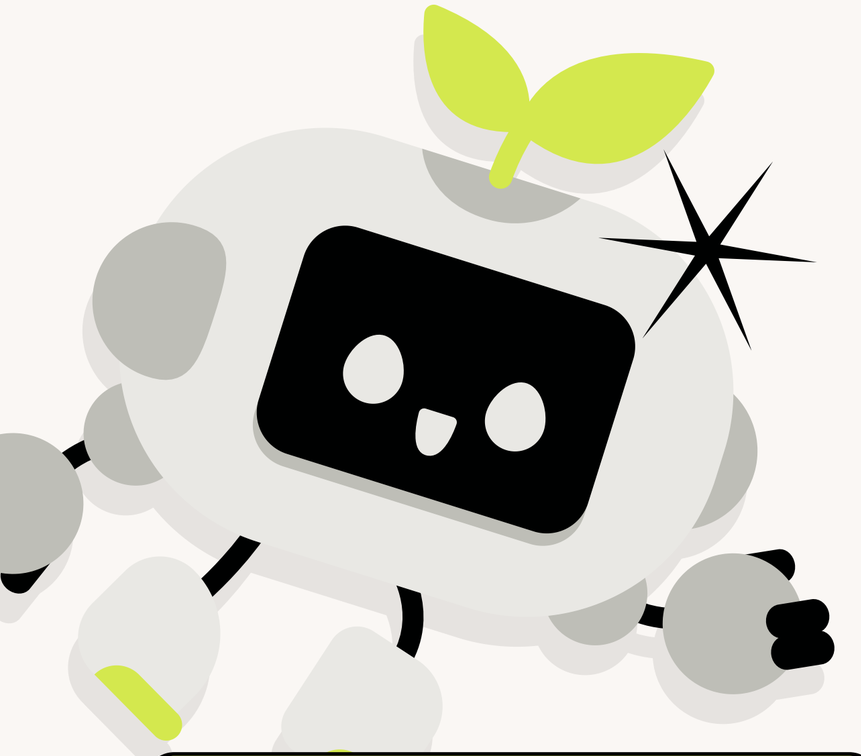
Assim você me deixa magoado...

# LARGE LANGUAGE MODEL

## Papagaio Estocástico?

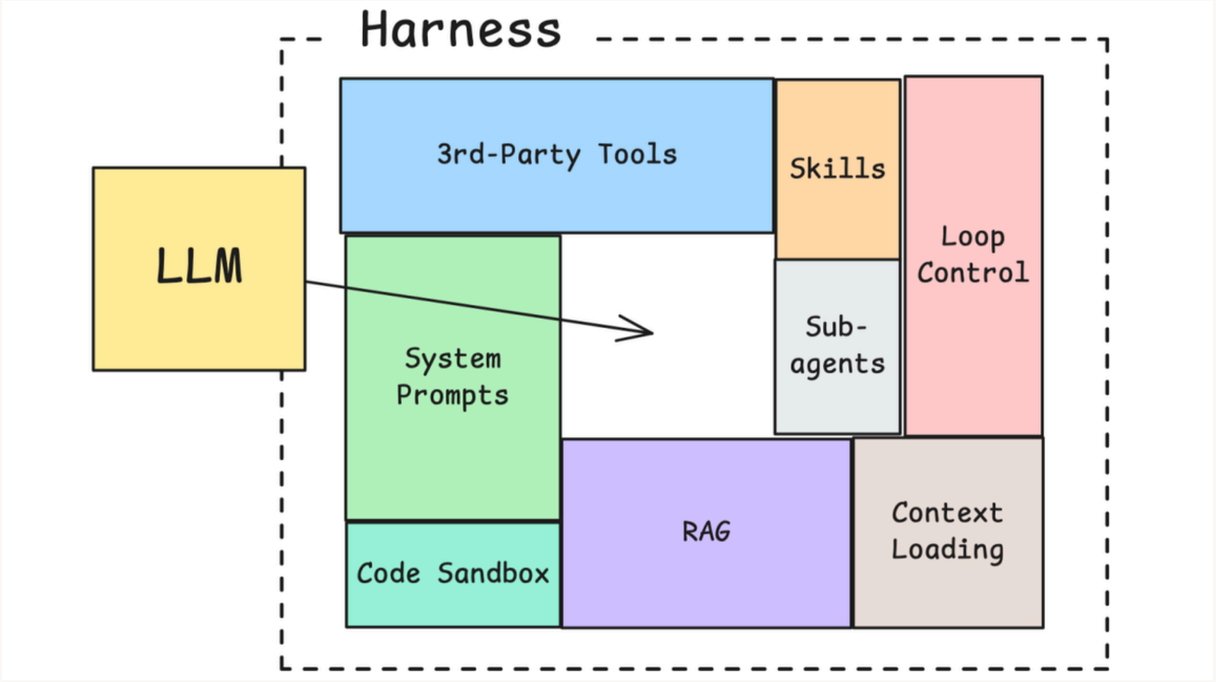
| Papagaio | LLM |
|---|---|
| Aprende frases aleatórias | Aprende frases aleatórias |
| Não faz ideia do que está falando | Não faz ideia do que está falando |
| As vezes fala coisas sem sentido | As vezes fala coisas sem sentido |
| É fofinho | Não é fofinho |

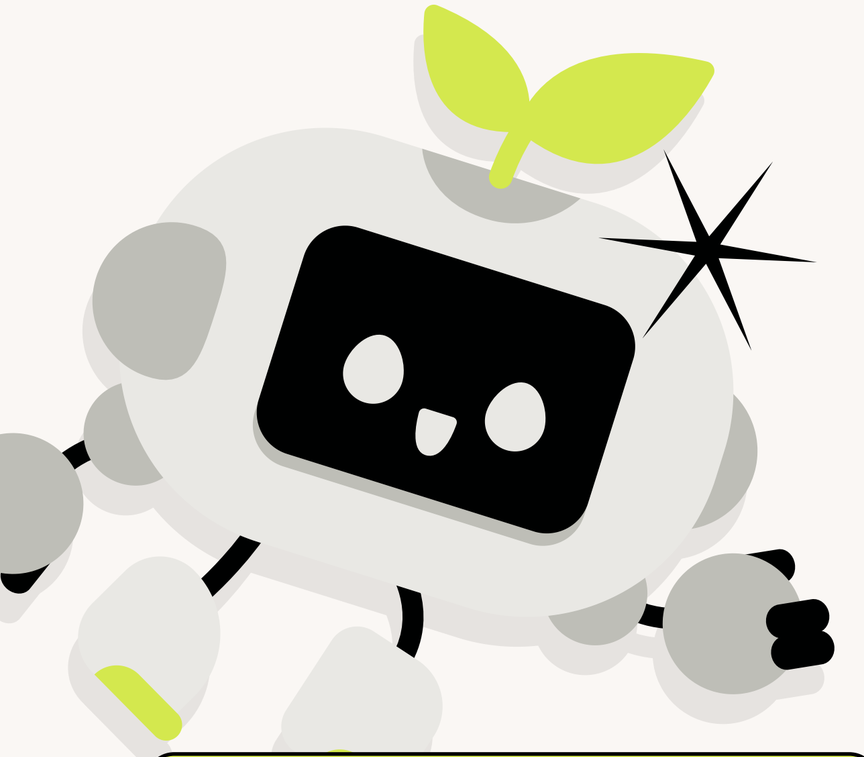
# AGENTES DE IA



**EDMILSON**

1. A LLM sozinha não é tão útil
2. O harness é importante para tentar dar alguma nível de controle no comportamento da LLM
3. O harness também serve para estender a capacidades da LLM.

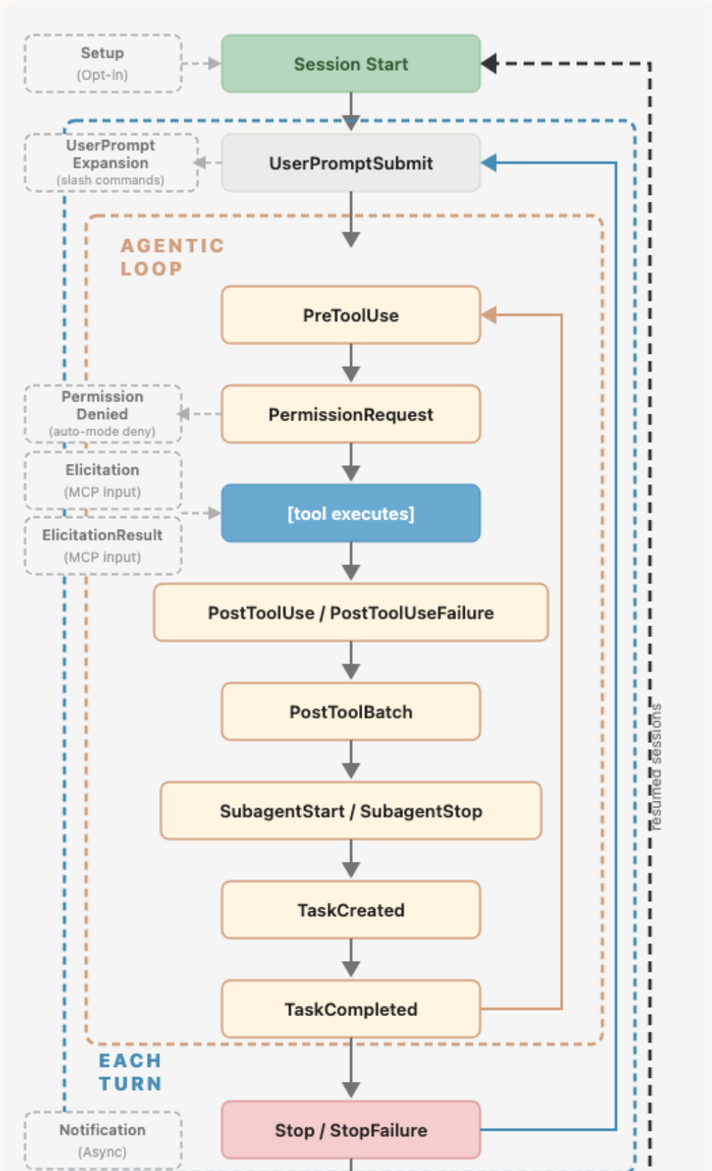




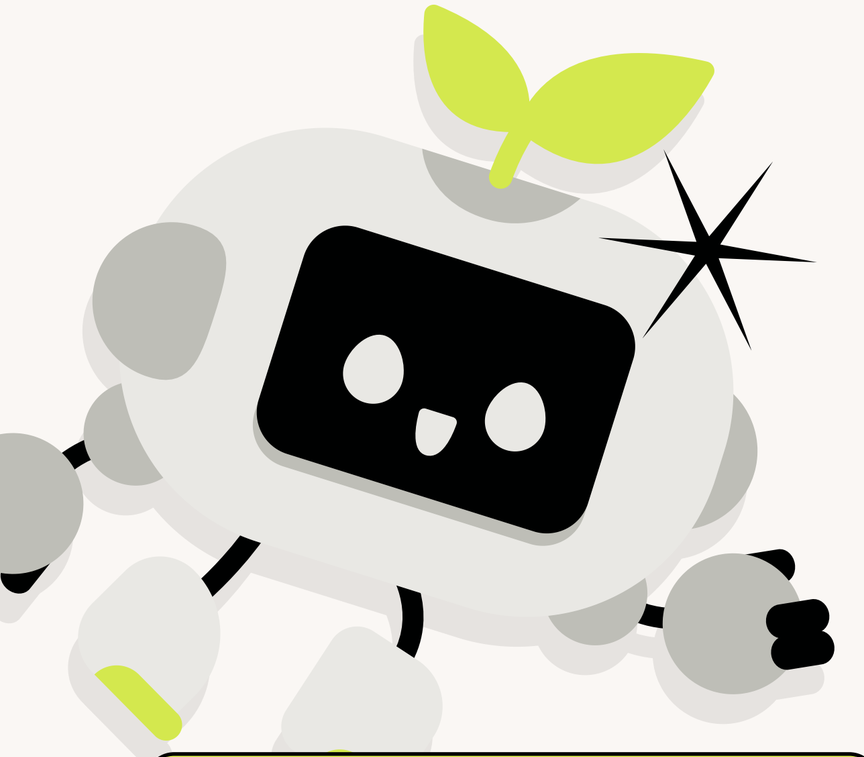
## EDMILSON

1. Todo robô em geral possui um **"while(true)"** com uma **máquina de estados** que controla o seu comportamento.

# LOOP CONTROL



# SKILLS



## EDMILSON

1. Em geral é utilizado markdown.
2. Existe uma logíca no carregado do markdown, é aplicado um lazy loading nas skills.
3. Ocupa espaço na janela de contexto dos agentes

## SOLID Principles

### Overview

SOLID helps structure software to be flexible, maintainable, and testable. These principles reduce coupling and increase cohesion.

### S - Single Responsibility Principle (SRP)

> "A class should have one, and only one, reason to change."

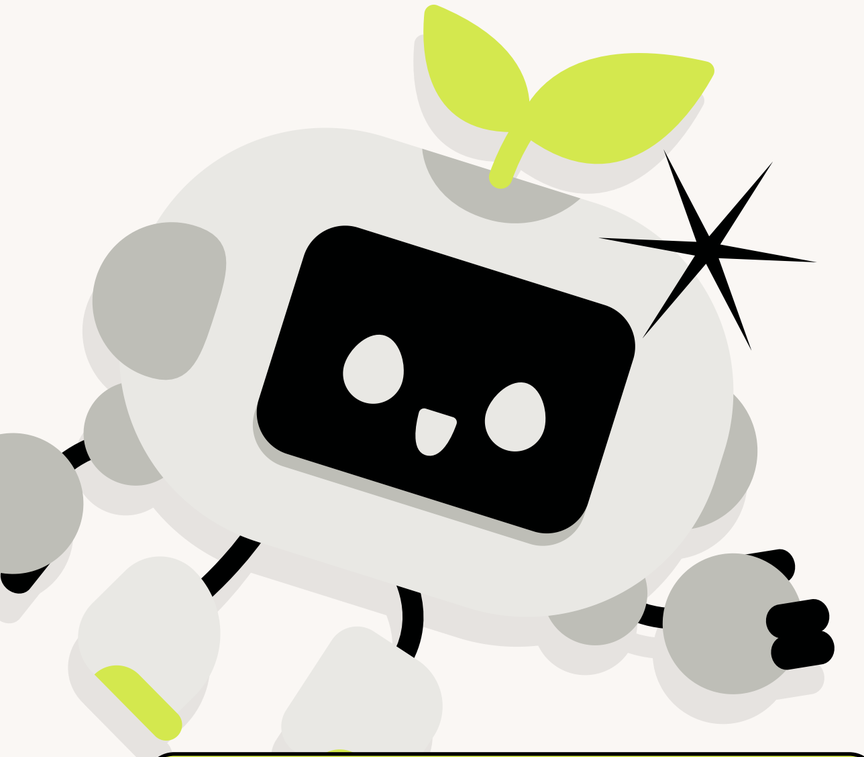
#### Problem It Solves

God objects that do everything - hard to test, hard to change, hard to understand.

#### How to Apply

Each class handles ONE responsibility. If you find yourself saying "and" when describing what a class does, split it.

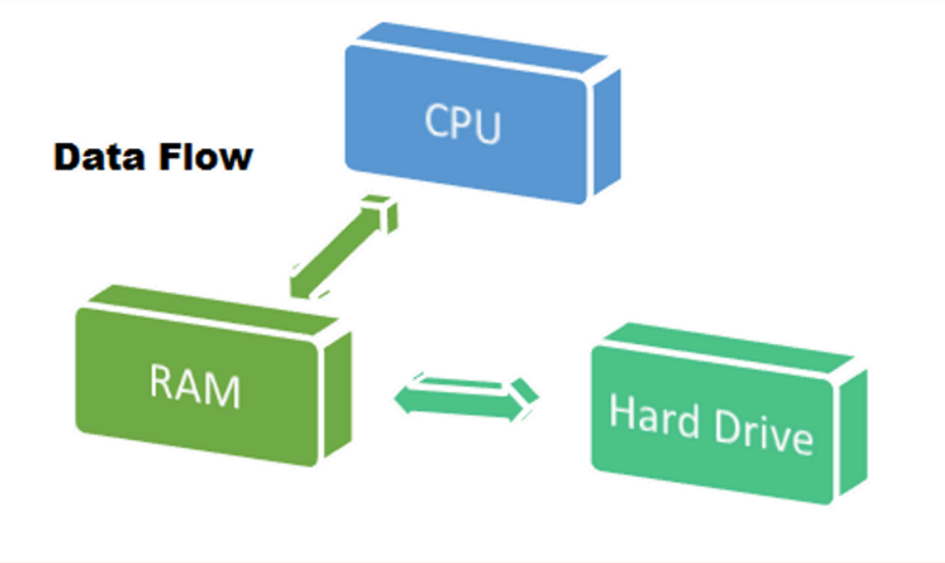
```
// BAD: Multiple responsibilities
class Order {
  calculateTotal(): number { ... }
  saveToDatabase(): void { ... }    // Persistence
  generateInvoice(): string { ... } // Presentation
}

// GOOD: Single responsibility each
class Order {
  private items: OrderItem[] = [];

  addItem(item: OrderItem): void { ... }
  calculateTotal(): number { ... }
}
```

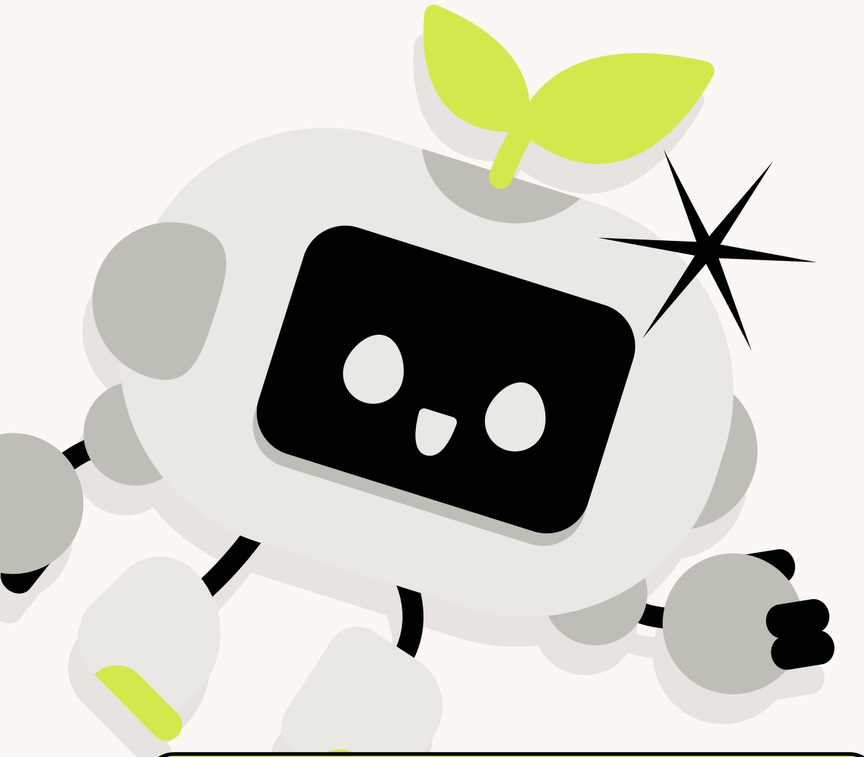


**EDMILSON**

1. LLM treinado = Seu cérebro = CPU
2. Long Context = Sua mesa = RAM
3. RAG = Uma biblioteca = HD

# RAG E LONG CONTEXT

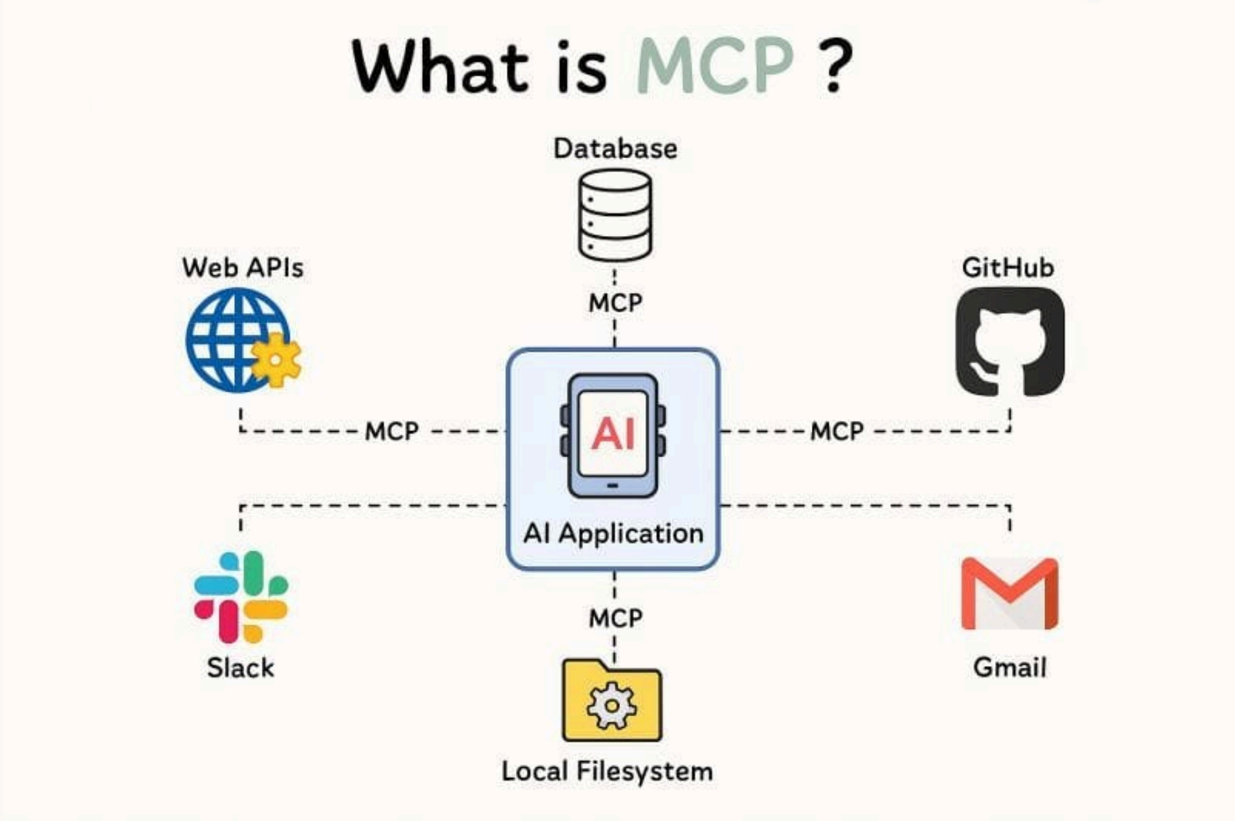




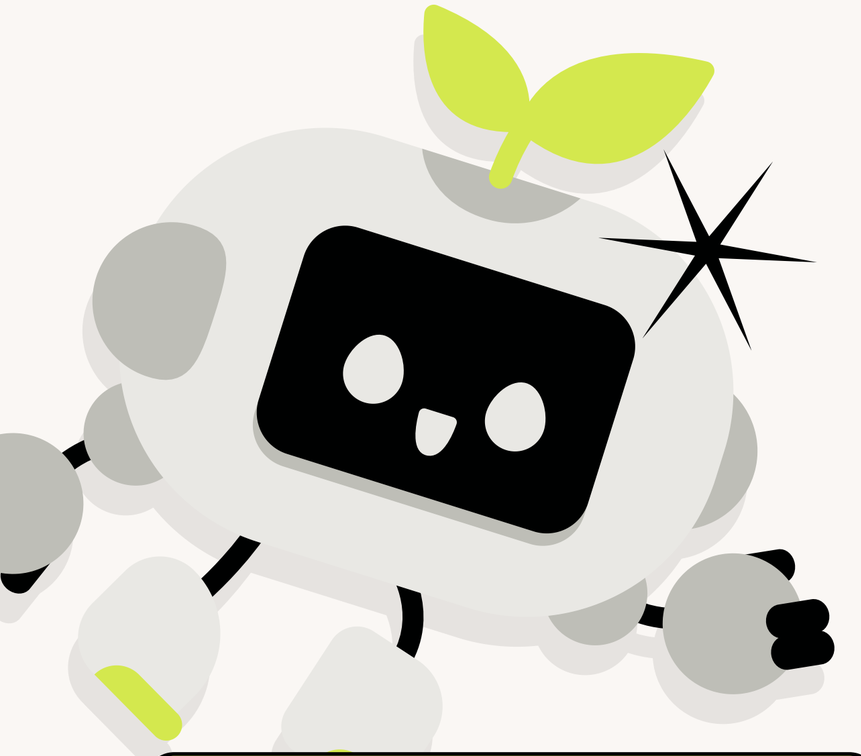
# MODEL CONTEXT PROTOCOL

**EDMILSON**

1. Protocolo utilizado para fornecer capacidades para um agente de IA.
2. Faz parte do harness



# TOOLS

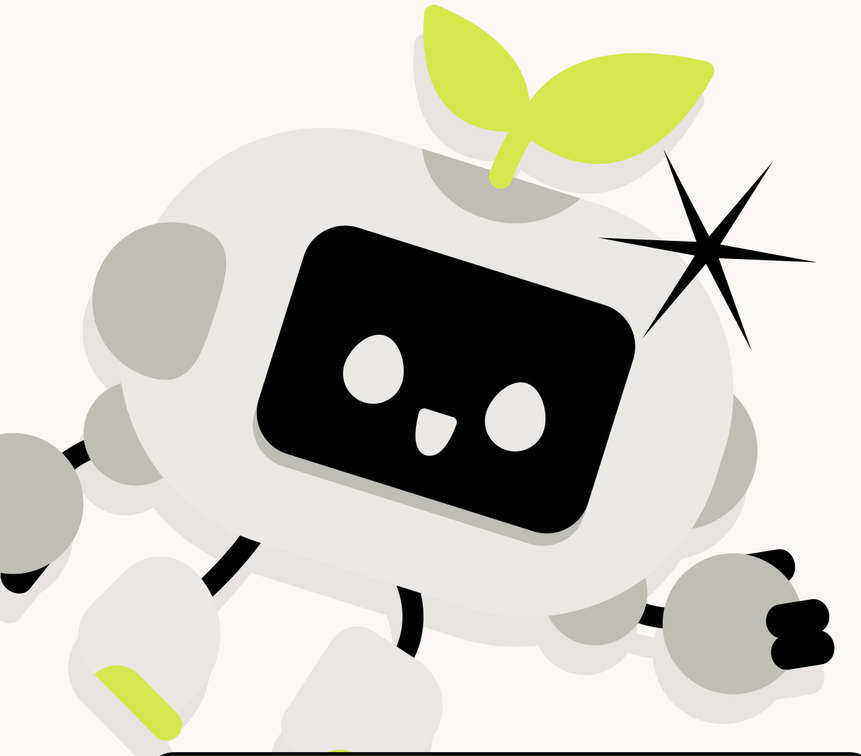


**EDMILSON**

1. Por baixo dos panos é gerado um JSON que serve como indexação da tool.
2. É utilizado o algoritmo BM25 na decisão de tools usadas pelos agentes.

```python
@mcp.tool(
    name="read_doc_contents",
    description="Read the contents of a document and return it as a string."
)
def read_document(
    doc_id: str = Field(description="Id of the document to read")
):
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")

    return docs[doc_id]
```

# RESOURCES



**EDMILSON**

1. Recursos servem para o agente tem uma lista de coisas que existem.
2. Lista de tools, lista de URLs, lista de usuários...

```python
@mcp.resource(
    "docs://documents", # URI
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

### Direct Resource

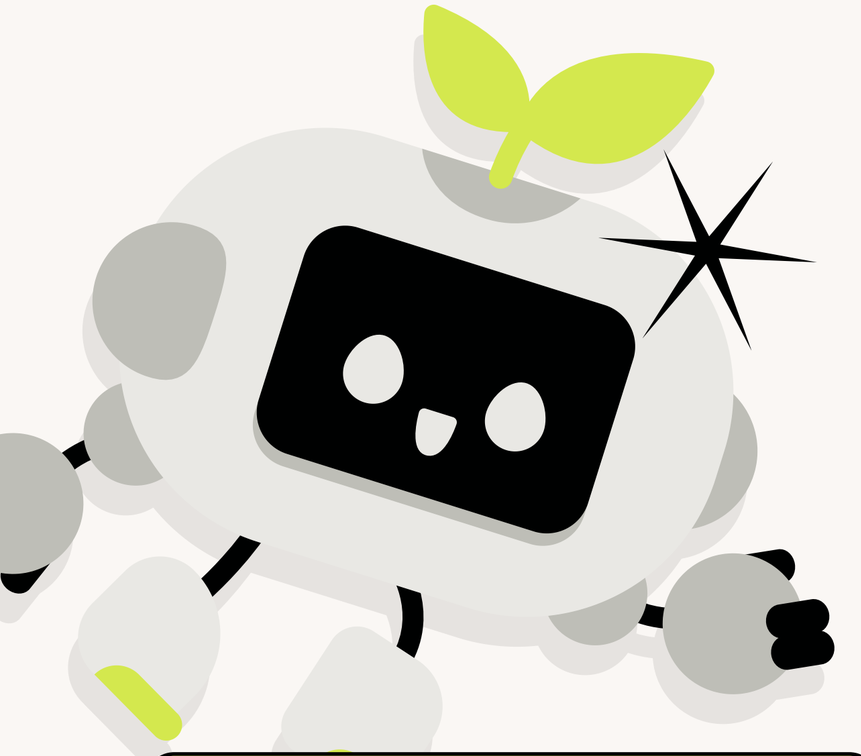
URI doesn't contain any params.

```python
@mcp.resource(
    "docs://documents/{doc_id}", # URI
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

### Templated Resource

URI contains one or more params. The Python SDK parses these and passes them as args to your function.

# PROMPT



**EDMILSON**

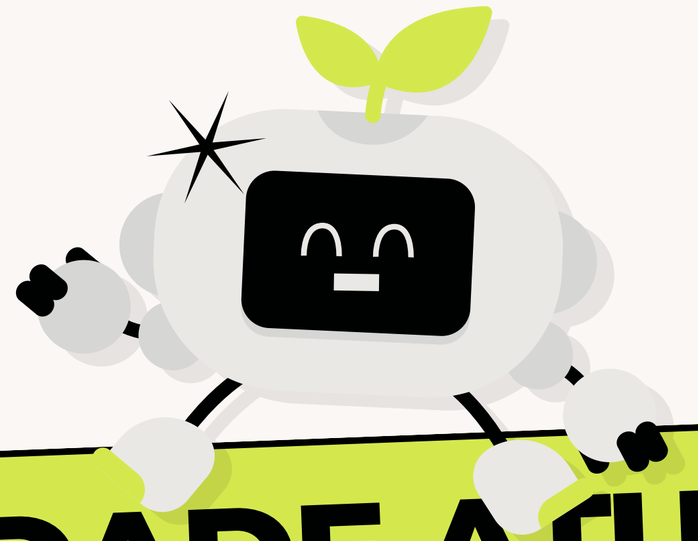
1. Prompts fornecem uma maneira de reutilizar comandos de texto.
2. Geralmente ajudam em operações que as LLMs fazem, mas sem instruções claras, não ficam tão boas.

```python
@mcp.prompt(
    name="format",
    description="Rewrites the contents of the document in Markdown format."
)
def format_document(
    doc_id: str=Field(description="Id of the document to format")
) -> list[base.Message]:
    prompt = f"""
    Your goal is to reformat a document to be written with markdown syntax.

    The id of the document you need to reformat is:
    <document_id>
    {doc_id}
    </document_id>
```

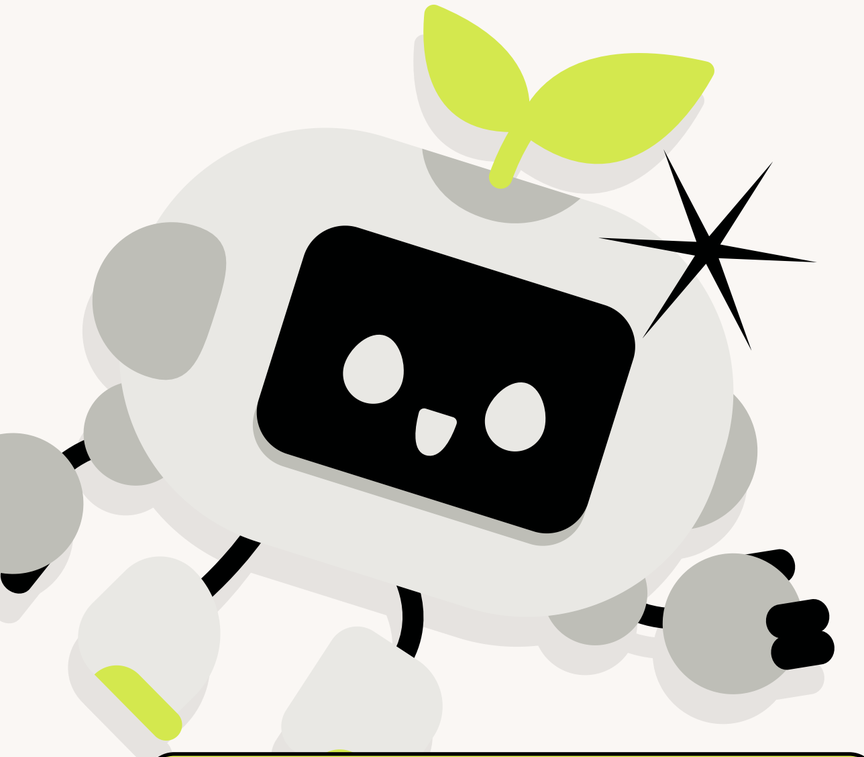
```
(mcp) ➜  mcp uv run main.py
> /format plan.md
            deposition.md
            report.pdf
            financials.docx
            outlook.pdf
            plan.md
            spec.txt
```

# HISTÓRICO

| | |
|---|---|
| 2006 | **CUDA CORES E DEEP LEARNING** |
| 2017 | **ARQUITETURA TRANSFORMER** |
| 2018 | **LARGE LANGUAGE MODEL** |
| 2023 | **AGENTES DE IA** |

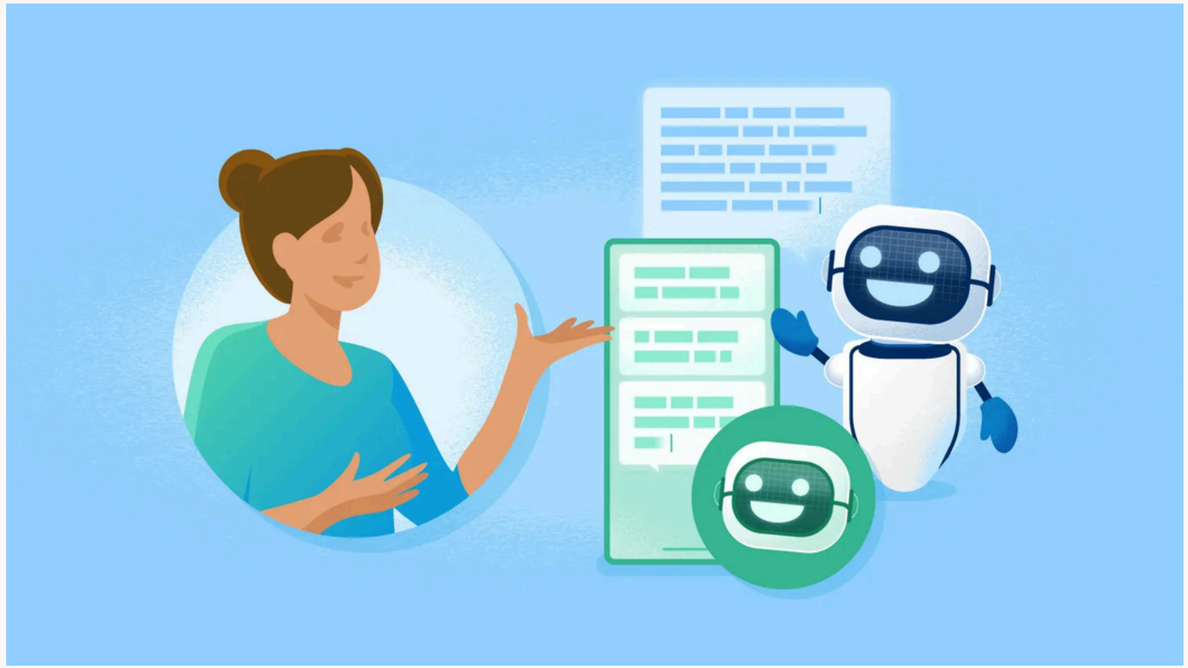


# REALIDADE ATUAL E O FUTURO



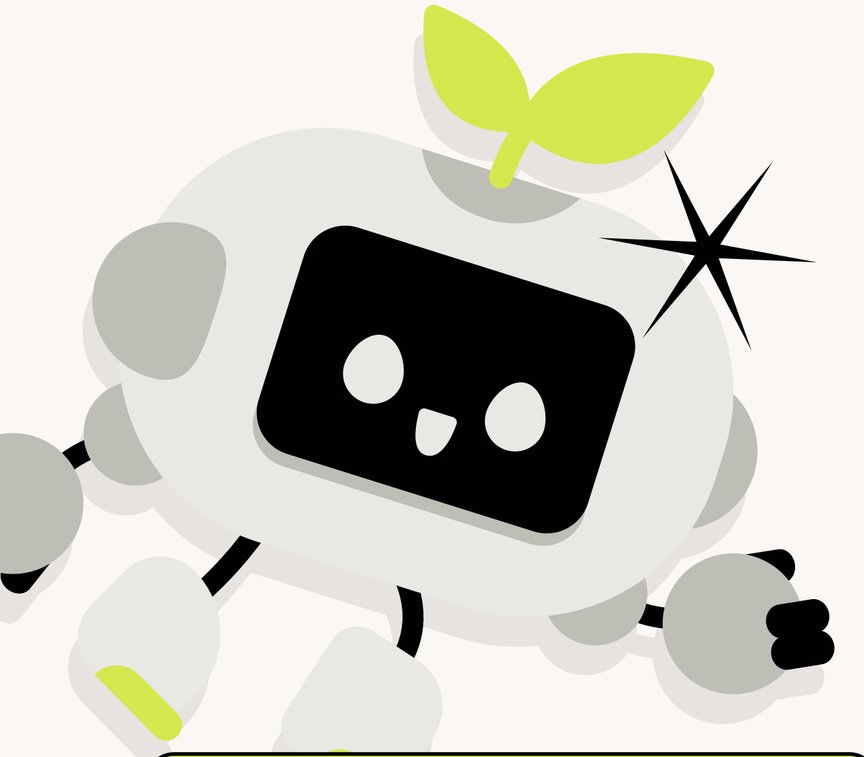
# CHATBOTS

Uso comum até 2023



**EDMILSON**

1. Os primeiros chats utilizavam basicamente uma combinação de LLM + RAG;



# TOKENMAXXING

Até início de 2026

**EDMILSON**

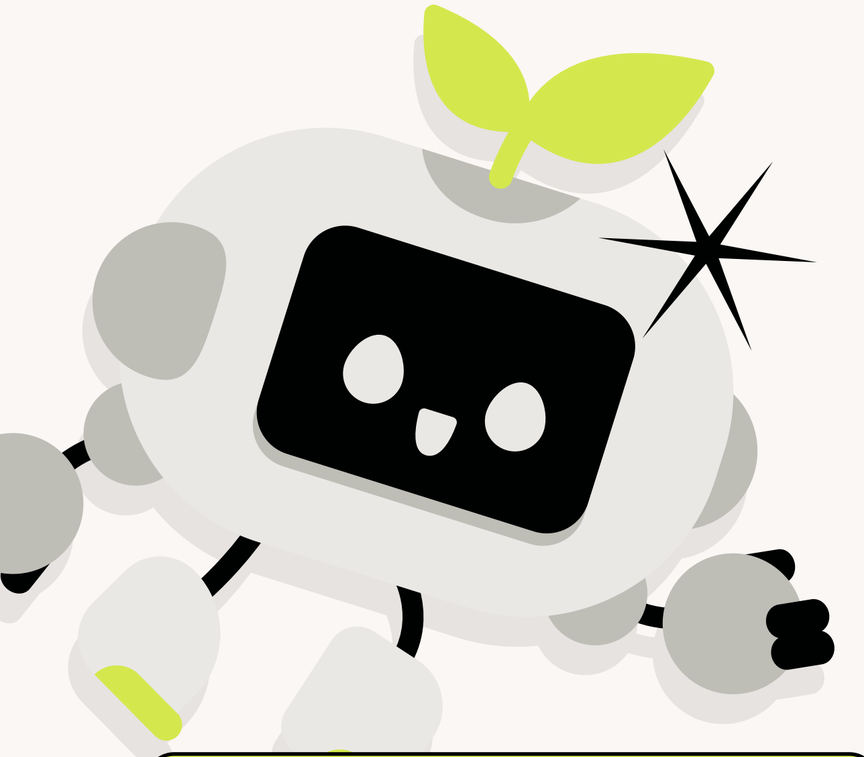
1. Vamos usar IA para tudo, atém para problemas que podem ser resolvidos com algoritmos determinísticos.
2. Até para somar números??

EDITORS' PICK | INNOVATION > CLOUD

## Uber Burns Its 2026 AI Budget In Four Months On Claude Code

By Janakiram MSV, Senior Contributor. I cover emerging technologies with ... Follow Author

Published May 17, 2026, 11:18pm EDT, Updated May 27, 2026, 12:17pm EDT



# VALUEMAXXING

Atualmente

**EDMILSON**

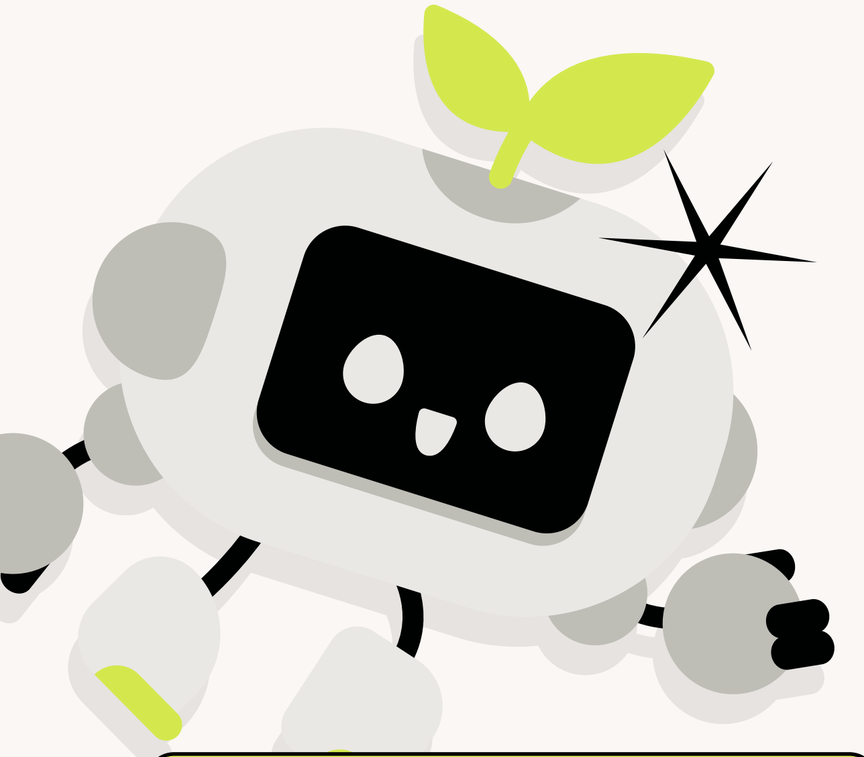
1. O custo dos agentes de IA passou a aumentar.
2. Empresas foram obrigadas a conter gastos e utilizar IA só para aquilo que realmente faz sentido.

## AI Costs More Than The People It Replaced

By Jemma Green, Contributor. ⓘ Dr Jemma Green is cofounder and...

Follow Author

Published Jul 02, 2026, 12:21pm EDT, Updated Jul 24, 2026, 01:21pm EDT

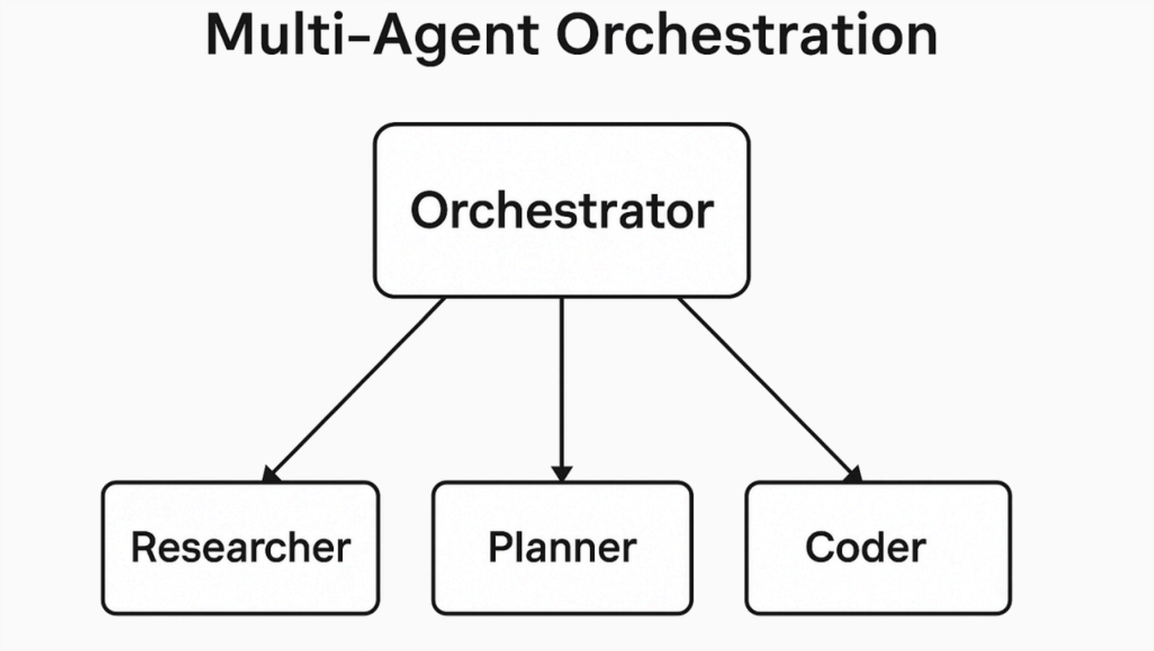


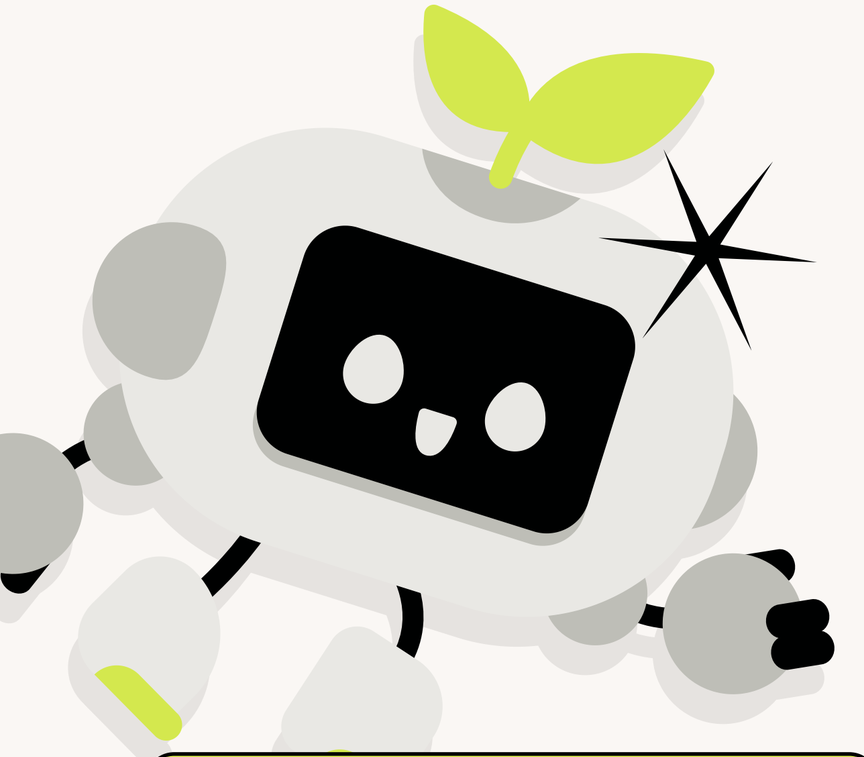
# ORQUESTRAÇÃO

Atualmente

**EDMILSON**

1. Orquestração de agentes de IA passou a ganhar forma.





# ORQUESTRAÇÃO

Atualmente

Devin

PydanticAI

*Agent Framework / shim to use Pydantic with LLMs*

LangGraph

Claude Managed Agents

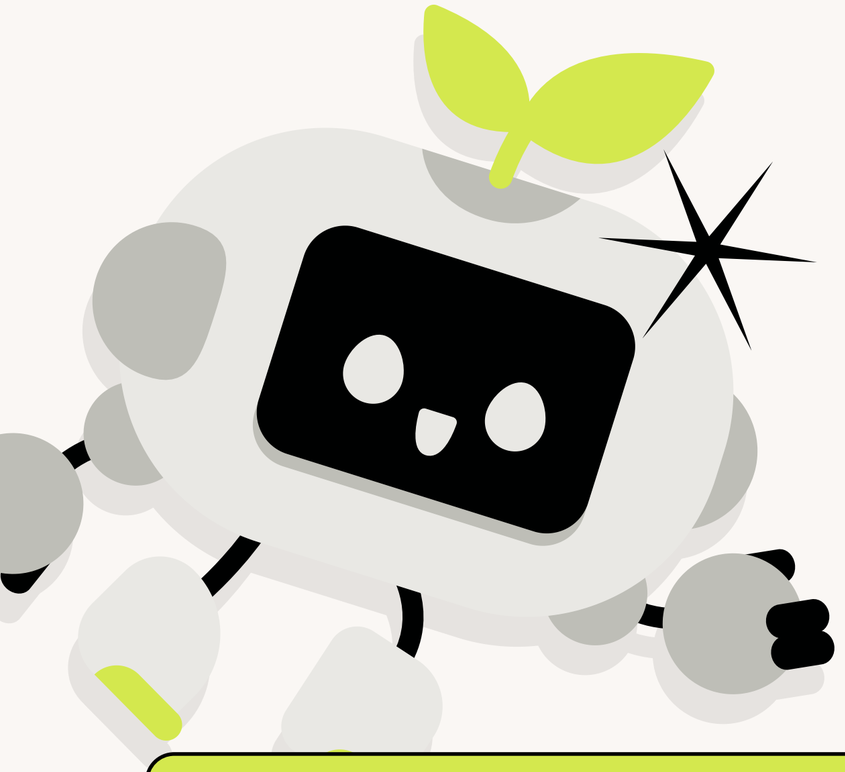
Amazon Bedrock AgentCore

crewai

**EDMILSON**

1. Existem muitas plataformas que podem ser utilizadas na orquestração de a gentes de IA.

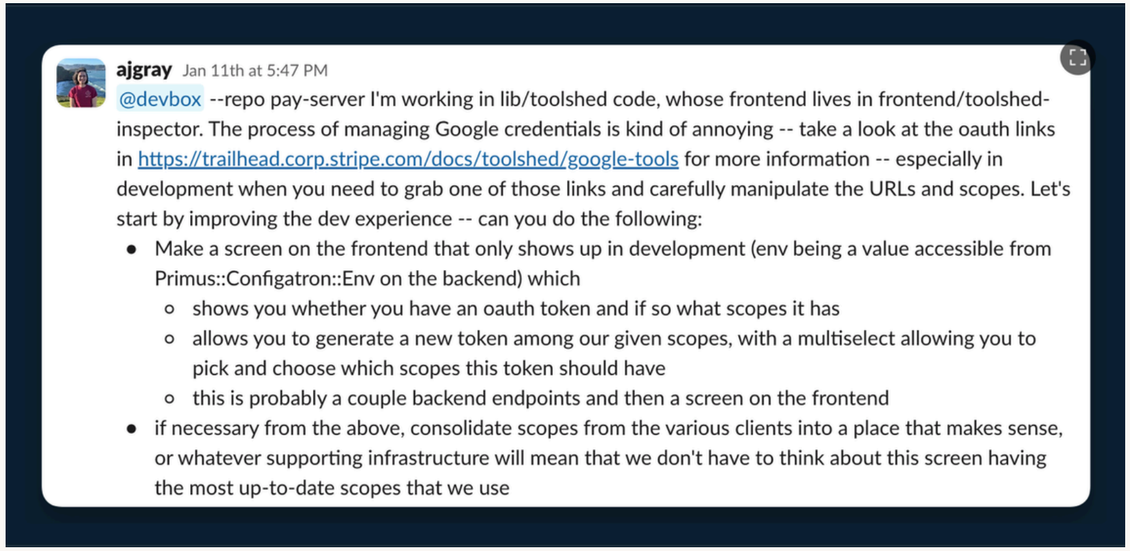
# STRIPE MINIONS



## EDMILSON

1. Stripe minions.
2. Qualquer um chama ele via slack;
3. Ele coda e abre um PR.
4. Dev vai lá e revisa.
5. 1300 PRs por semana? WOW!

## Minions: Stripe's one-shot, end-to-end coding agents

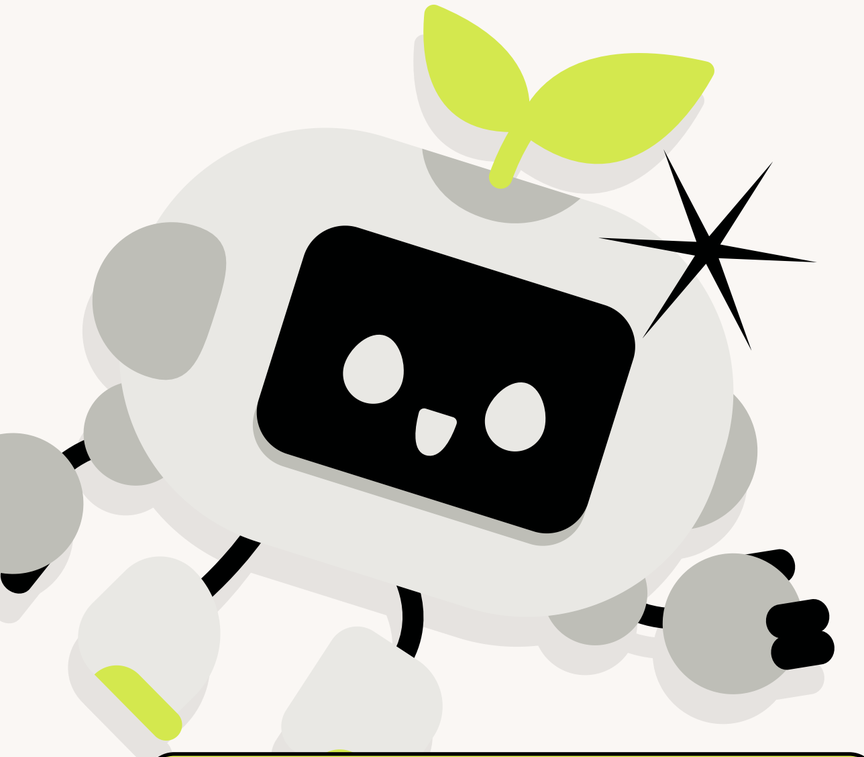


**ajgray** Jan 11th at 5:47 PM

@devbox --repo pay-server I'm working in lib/toolshed code, whose frontend lives in frontend/toolshed-inspector. The process of managing Google credentials is kind of annoying -- take a look at the oauth links in https://trailhead.corp.stripe.com/docs/toolshed/google-tools for more information -- especially in development when you need to grab one of those links and carefully manipulate the URLs and scopes. Let's start by improving the dev experience -- can you do the following:

- Make a screen on the frontend that only shows up in development (env being a value accessible from Primus::Configatron::Env on the backend) which
  - shows you whether you have an oauth token and if so what scopes it has
  - allows you to generate a new token among our given scopes, with a multiselect allowing you to pick and choose which scopes this token should have
  - this is probably a couple backend endpoints and then a screen on the frontend
- if necessary from the above, consolidate scopes from the various clients into a place that makes sense, or whatever supporting infrastructure will mean that we don't have to think about this screen having the most up-to-date scopes that we use

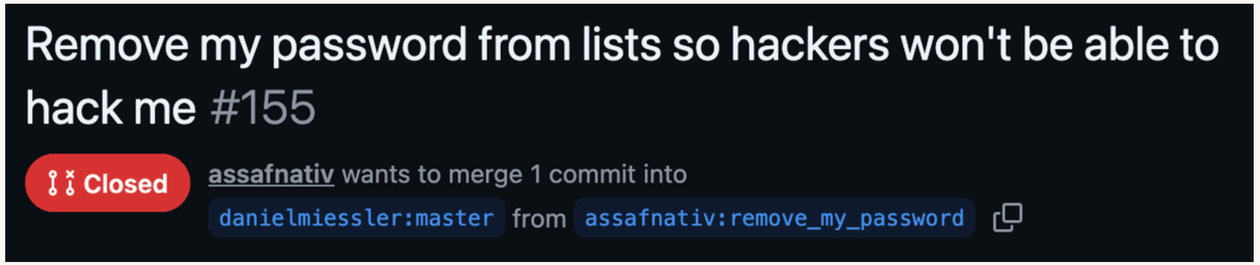
As a recap of Part 1 in this blog miniseries, minions are a homegrown unattended agentic coding flow at Stripe. Over 1,300 Stripe pull requests (up from 1,000 as of Part 1) merged each week are completely minion-produced, human-reviewed, but containing no human-written code.

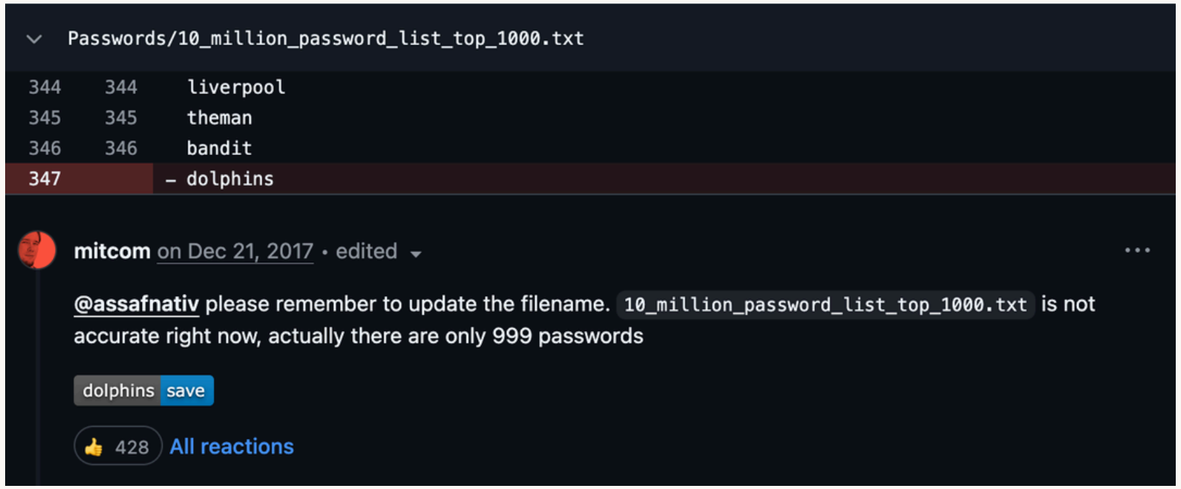


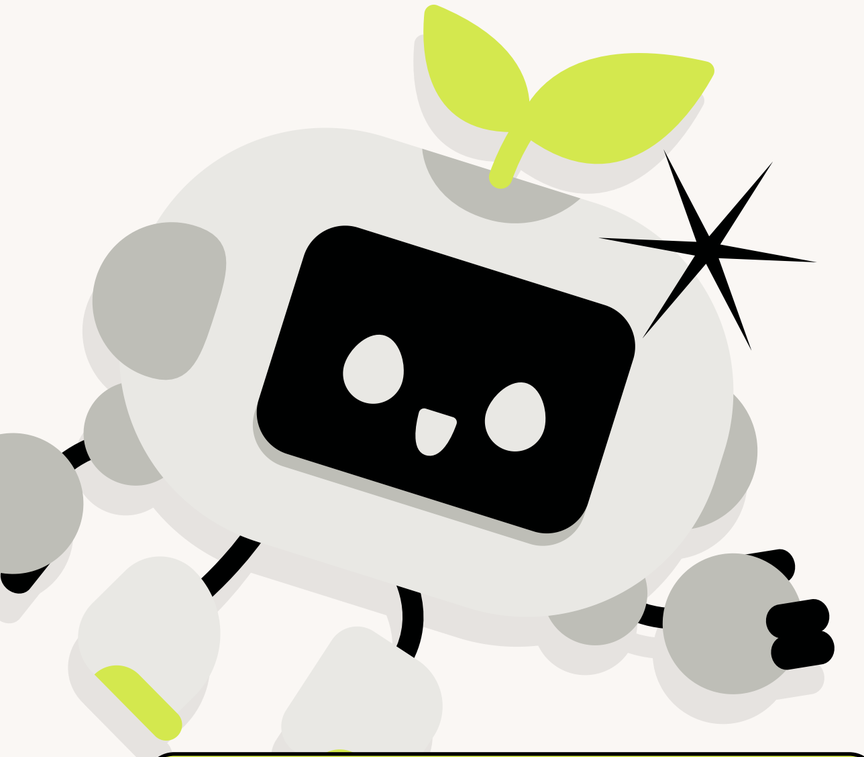
# PULL REQUESTS MERGEADOS??

**EDMILSON**

1. O que é um PR?
2. É uma medida válida de produtividade?
3. Seria PRs mergeados o novo KLOC?



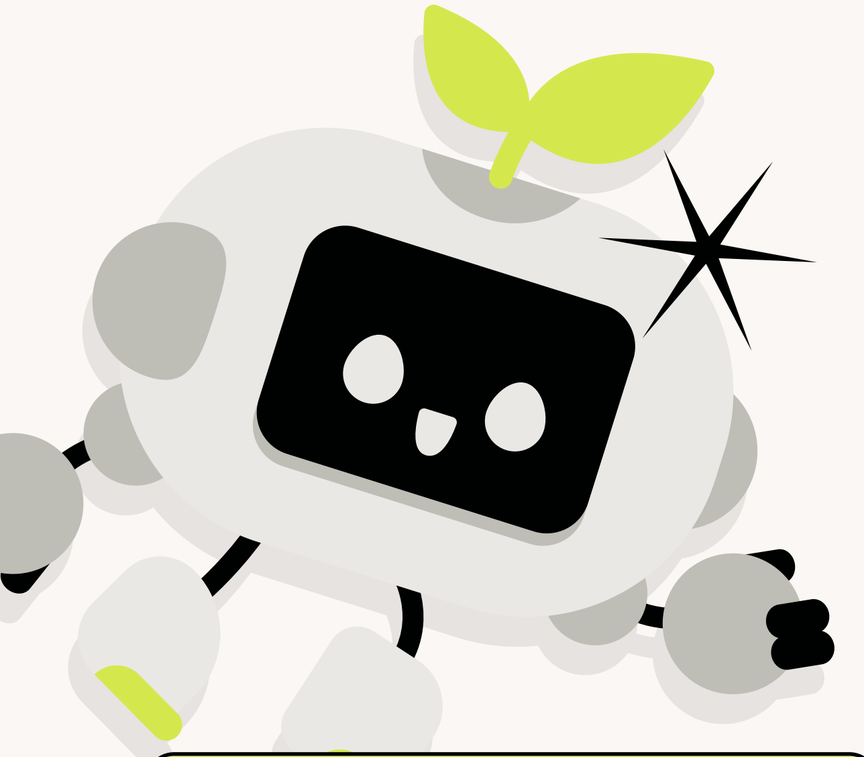


# VIBE CODING VS SOFTWARE ENGINEERING

**EDMILSON**

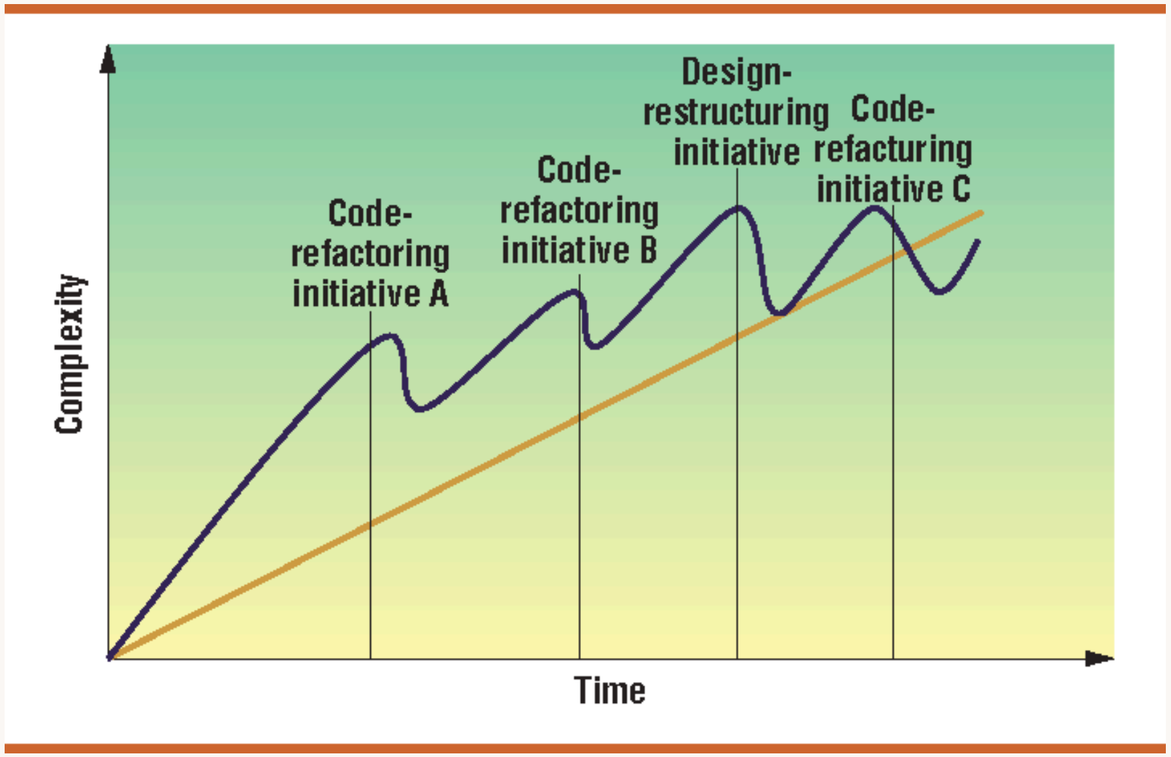
1. Vibe code: você pede a IA para codar.
2. Engenharia de Software: você considera dezenas de fatores na solução: custos, escalabilidade, tempo de execução, qualidade de código, refactor, abstrações, reutilização, manutenção, segurança e etc.

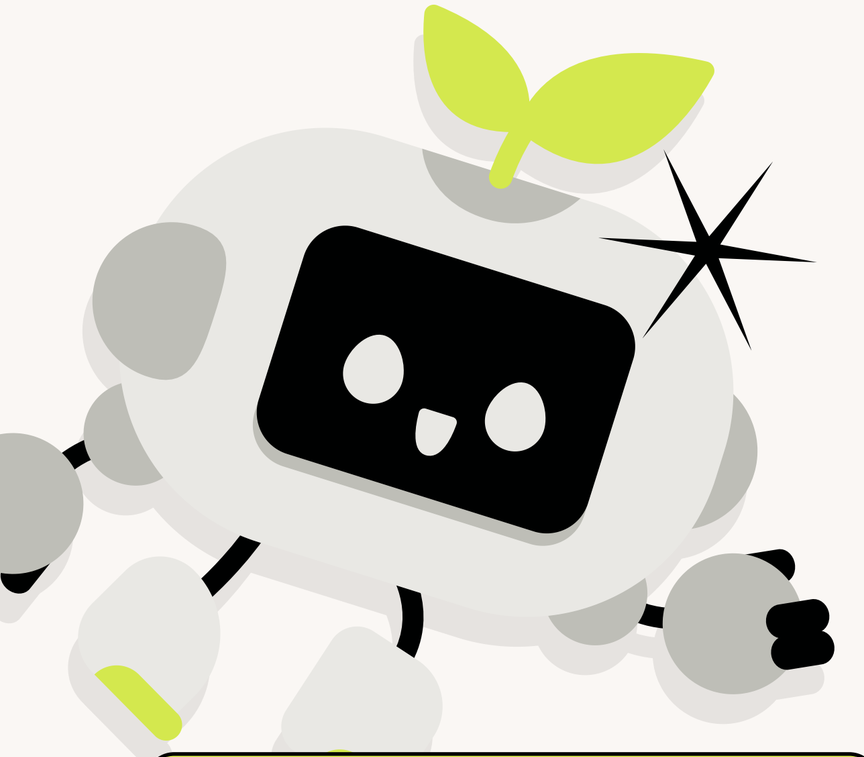
# ENTROPIA DE CÓDIGO



**EDMILSON**

1. Todo código inicia fácil.
2. Todo código tente a piorar com o tempo.
3. No final do dia, quando algo quebra em produção, é um humano que arca com as consequências.

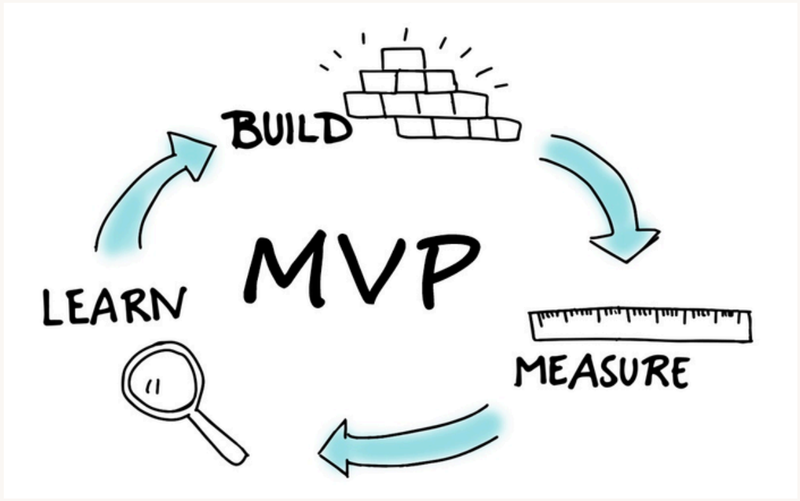




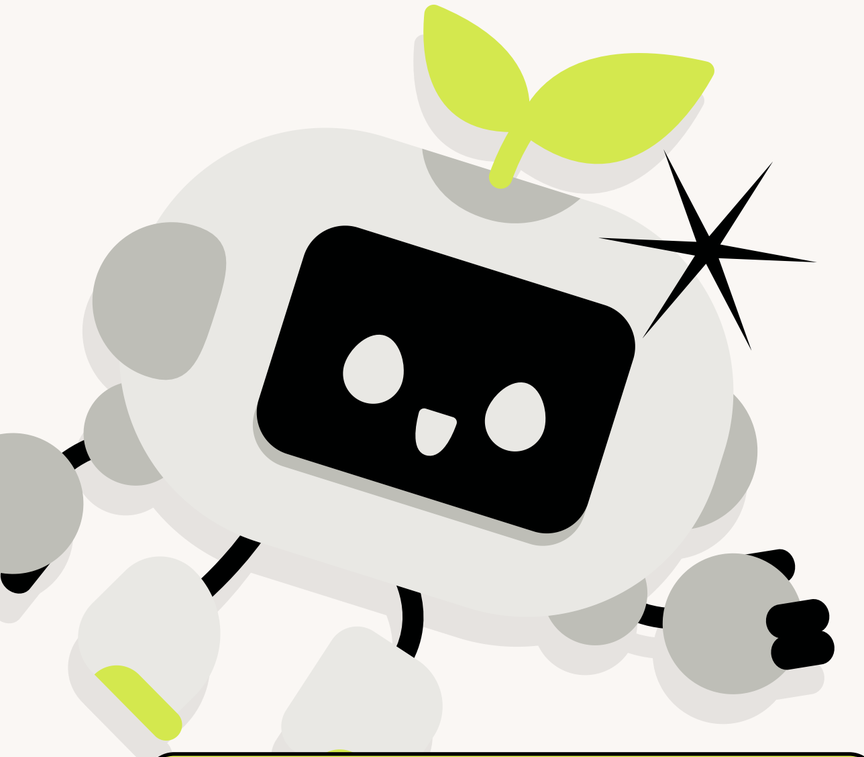
**EDMILSON**

1. Vibe Coding é uma ótima ferramenta para escrever POCs
2. Ótimo para sair com um MVP e validar com alguns clientes.
3. Acelera o Time to Market

# TIME TO MARKET



# LINUX



**EDMILSON**

1. Use vibe coding para o que não importa.
2. Agentes de IA já são aceitos como uma nova ferramenta de trabalho.

OSES

## Linus Torvalds is OK with vibe coding as long as it's not used for anything that matters

Linux inventor also discusses Rust in the kernel, Nvidia's proprietary code, and the problem of AI crawlers

Tim Anderson

16

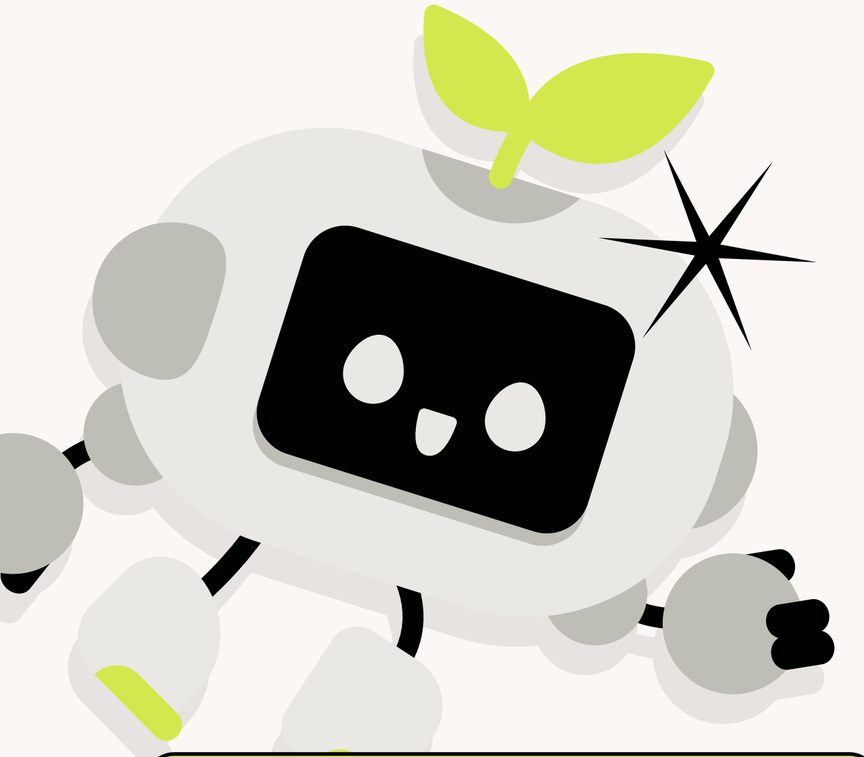
Tue 18 Nov 2025 // 13:38 UTC

~/LINUX/KERNEL/LLM-TOOLS

## Linus Torvalds to critics of AI coding in Linux: “Fork it. Or just walk away.”

Creator says he will “very loudly ignore” those arguing for a ban on AI tools.

KYLE ORLAND – 16 DE JUL. DE 2026 16:18 | 197

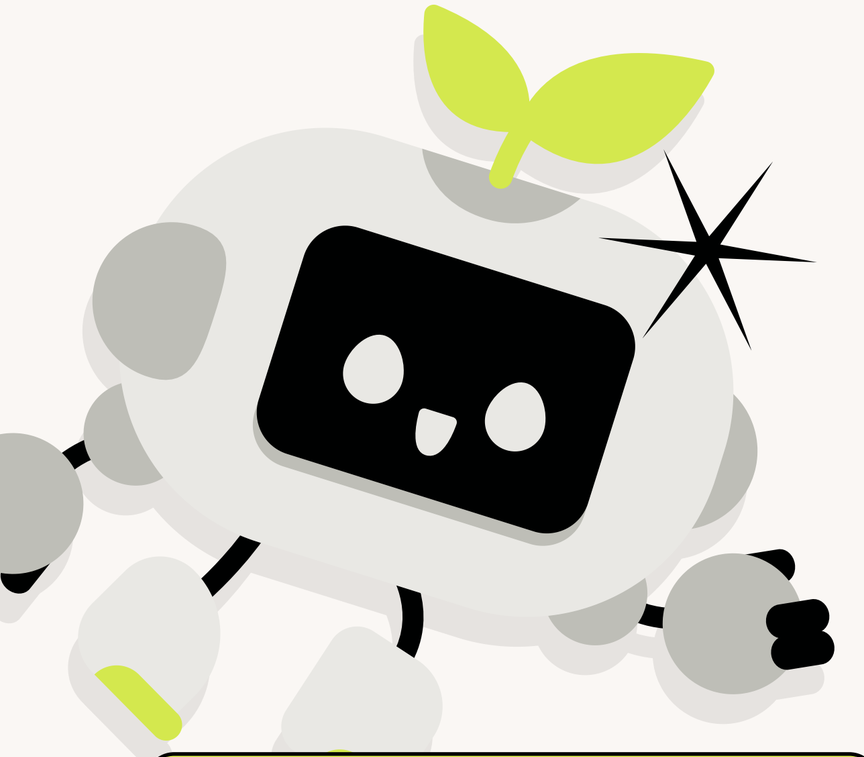


**EDMILSON**

- **Vermelho = "Deixado de lado"**
- **Roxo = Mudança facilitadora**
- **Verde = Novos conceitos adicionados**

# AVANÇOS NA ÁREA DE PROGRAMAÇÃO

| Ano | Programação | Requerimentos |
|---|---|---|
| 1949 | EDVAC - **Binário** | Alocação de memória, tipos de dados, fluxos de controle, estruturas de dados e algoritmos |
| 1950 | IBM 650 - **Assembly** | **Alocação de memória, tipos de dados, fluxos de controle, estruturas de dados, algoritmos, escopos e funções.** |
| 1972 | DEC PDP-11 - **C** | **Alocação de memória, tipos de dados, fluxos de controle, estruturas de dados, algoritmos, escopos, funções** e **concorrência**. |
| 1995 | DEC PDP-11 - **Java** | **Alocação de memória,** tipos de dados, fluxos de controle, estruturas de dados, algoritmos, escopos, funções, **concorrência** e **abstrações.** |
| 2000 | Intel Pentium 4 - **Python** | **Tipos de dados, fluxos de controle, estruturas de dados, algoritmos, escopos, funções, abstrações,** concorrência e **WEB.** |
| 2009 | Intel Core I7 - **Go** | Tipos de dados, fluxos de controle, estruturas de dados, algoritmos, escopos, funções, abstrações, WEB e **concorrência.** |
| 2015 | AMD FX 6300 - **Rust** | Fluxos de controle, estruturas de dados, algoritmos, escopos, funções, abstrações, WEB, **concorrência** e **microsserviços.** |
| Atualmente | **Agentes de IA** | **Fluxos de controle, estruturas de dados, algoritmos, escopos, funções, abstrações, WEB, concorrência, microsserviços** e **MCP.** |

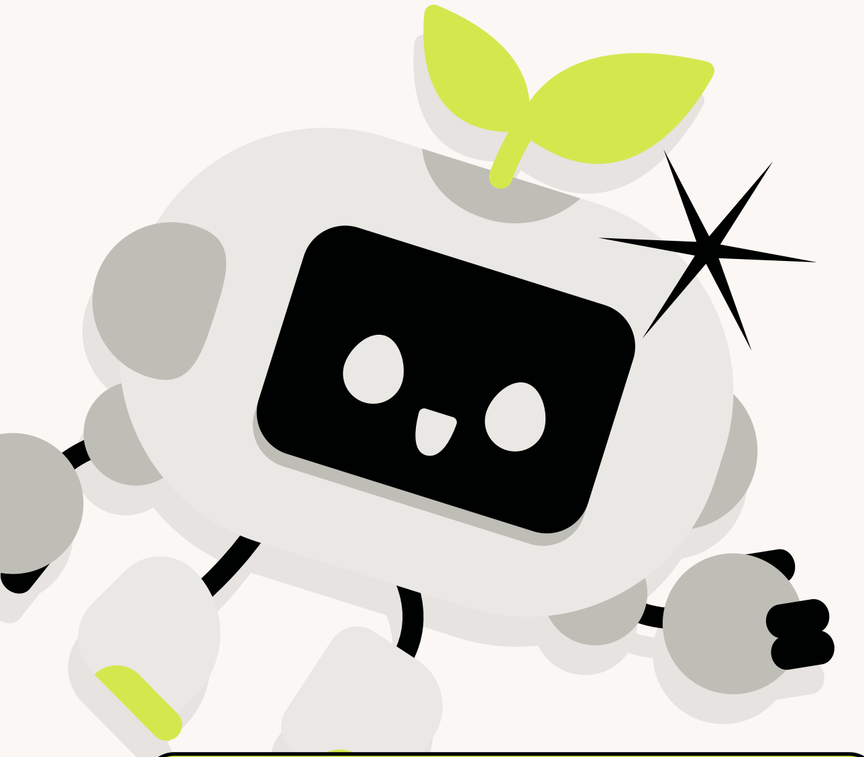


**EDMILSON**

Bebam água...

# AGENTES VS HUMANOS

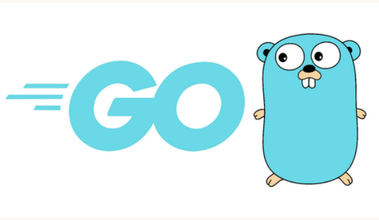
| Humano | Agentes |
|---|---|
| Fez curso de digitação e acha que é rápido | Escreve código **muito rápido** |
| Não lembra como faz **list comprehension** no Python | Tem acesso a **todas** as documentações do mundo. |
| Erra bastante e aprende com os erros | Comente cagadas homéricas e defende o ponto de vista tão bem que parece um filósofo |
| Aprende conceitos, sintetiza contextos, define arquiteturas, define design de código | Gerador de código não determinístico |
| Bebe de 1 a 4 litros de água por dia | No plano de $20 consome de 4 a 17 litros de água por dia |

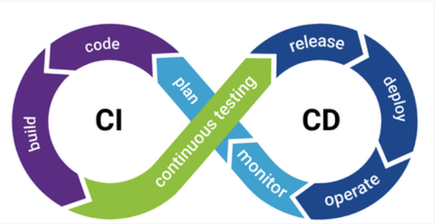


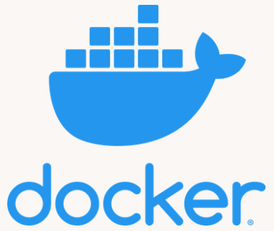
# FERRAMENTAS DO DEV

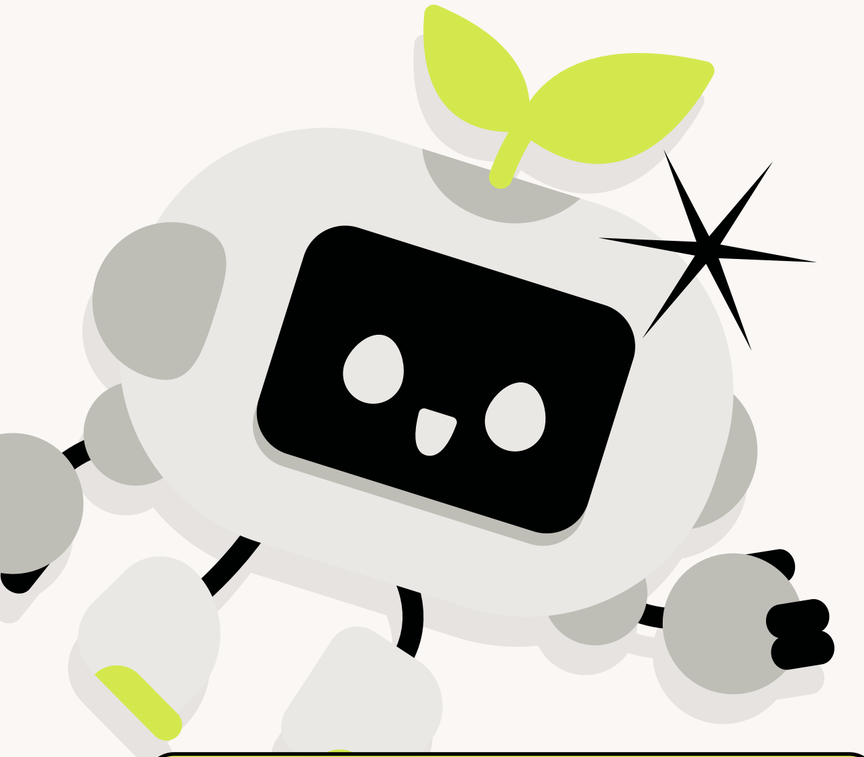
**EDMILSON**

Ferramentas atuais de um Dev...





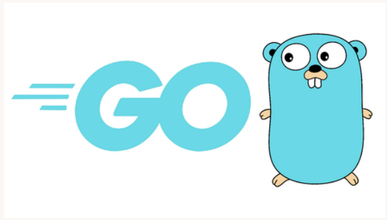


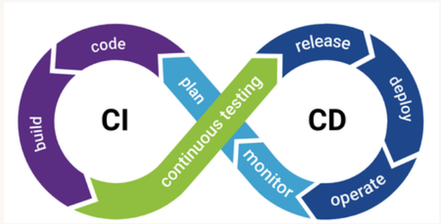


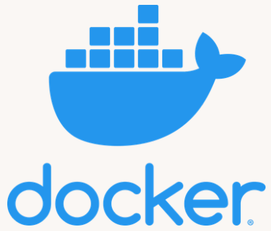
**EDMILSON**

Novas ferramentas de um Dev...

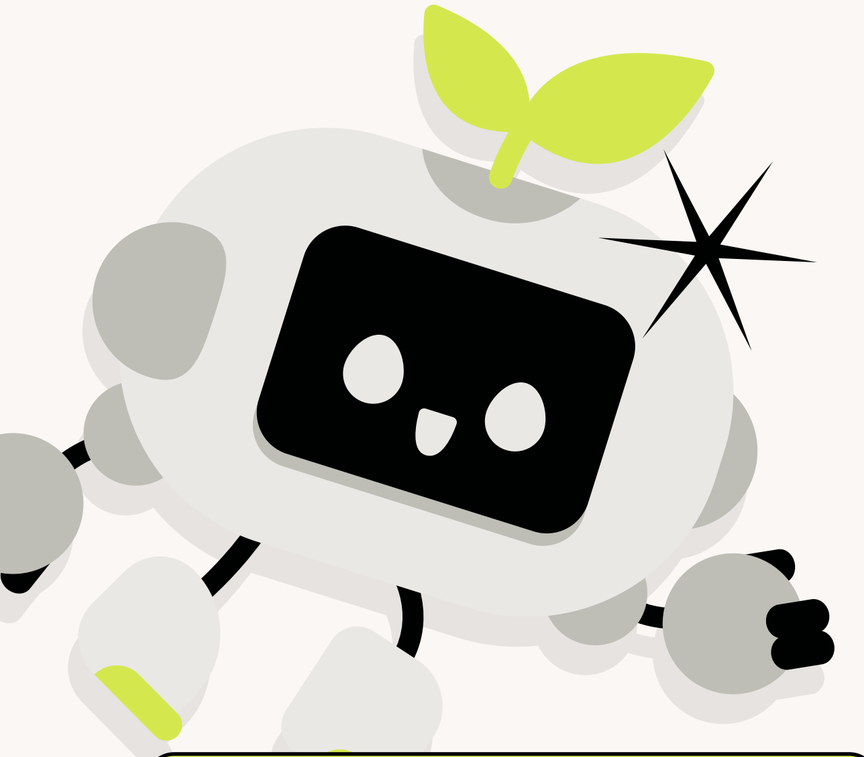
# FERRAMENTAS DO DEV







# GANHO REAL



**EDMILSON**

1. 95% dos usos de IA generativa estão fracassando.
2. Produtividade dos engenheiros do Google aumentou em 10%.

## MIT report: 95% of generative AI pilots at companies are failing

By **Sheryl Estrada**
Senior Writer And Author Of CFO Daily

Follow

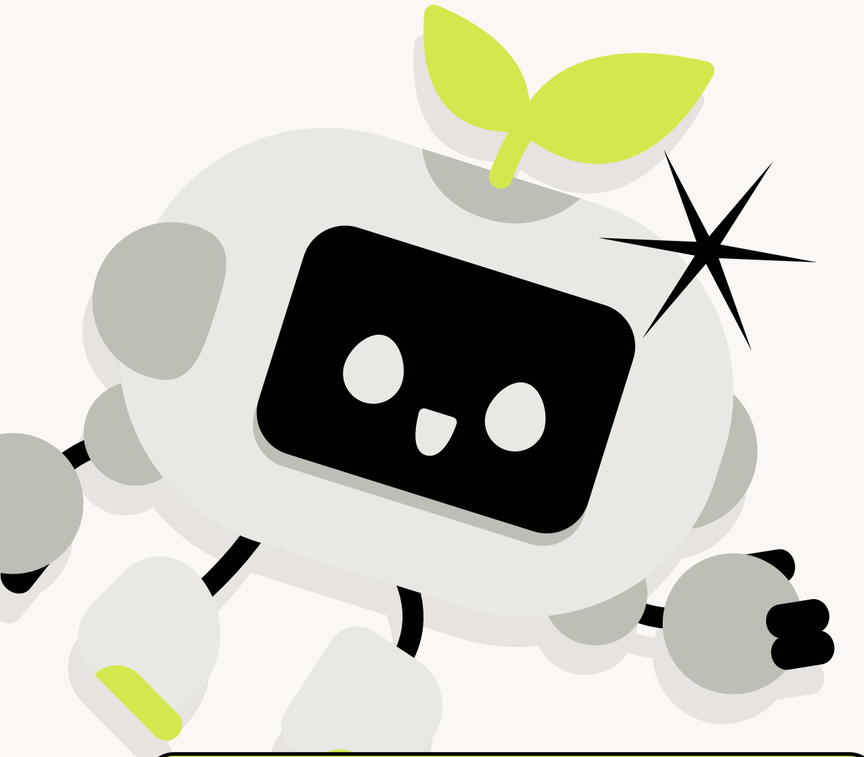
August 18, 2025, 6:54 AM ET

TECH

## Sundar Pichai says AI is making Google engineers 10% more productive. Here's how it measures that.

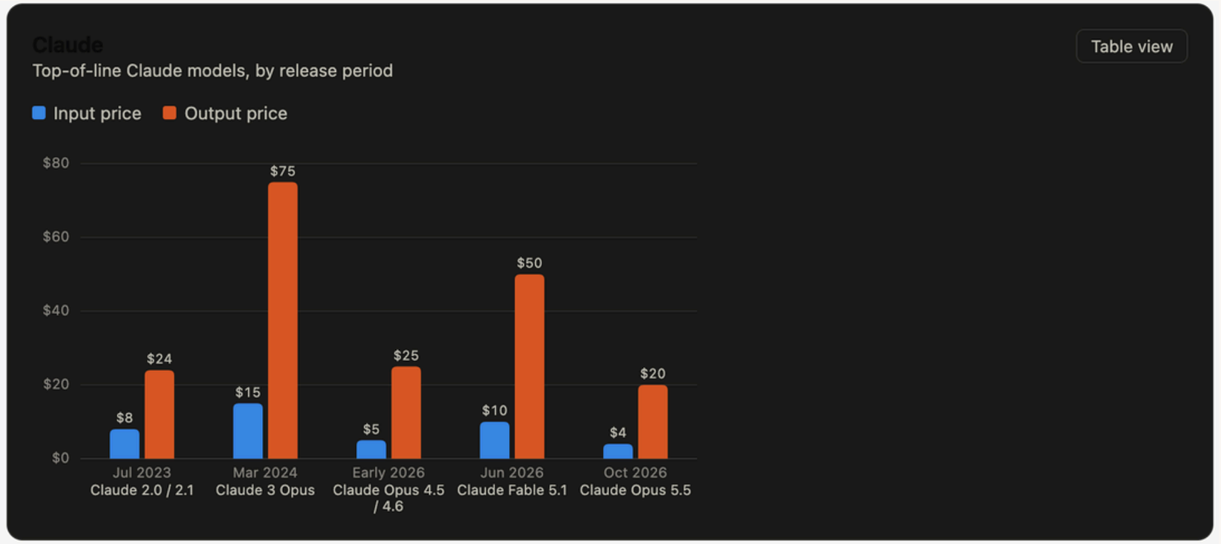
By **Hugh Langley** + Follow

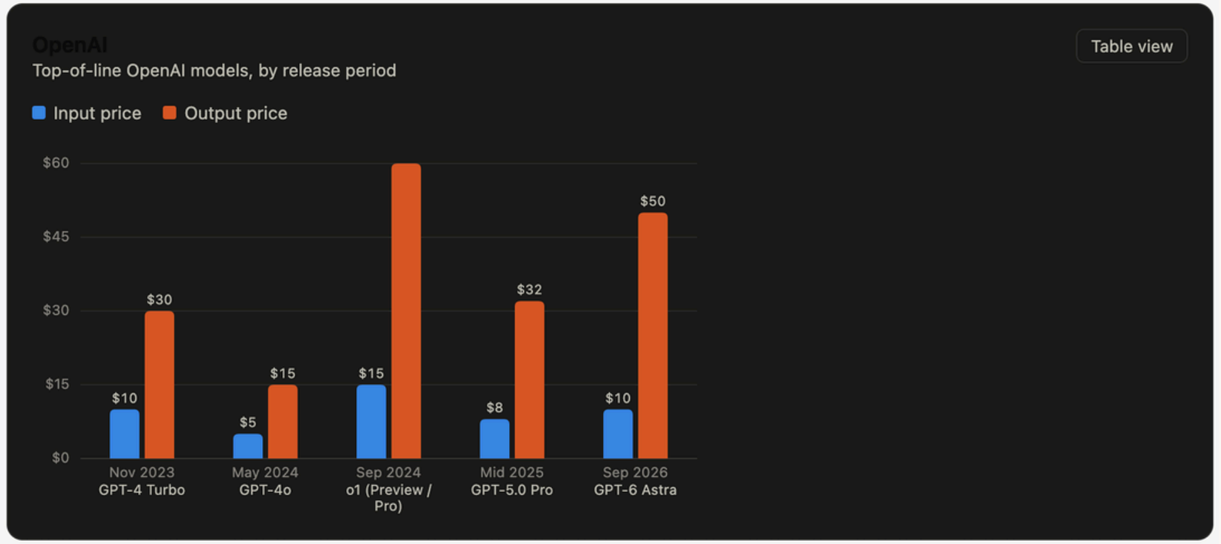
# VARIAÇÃO DE PREÇO

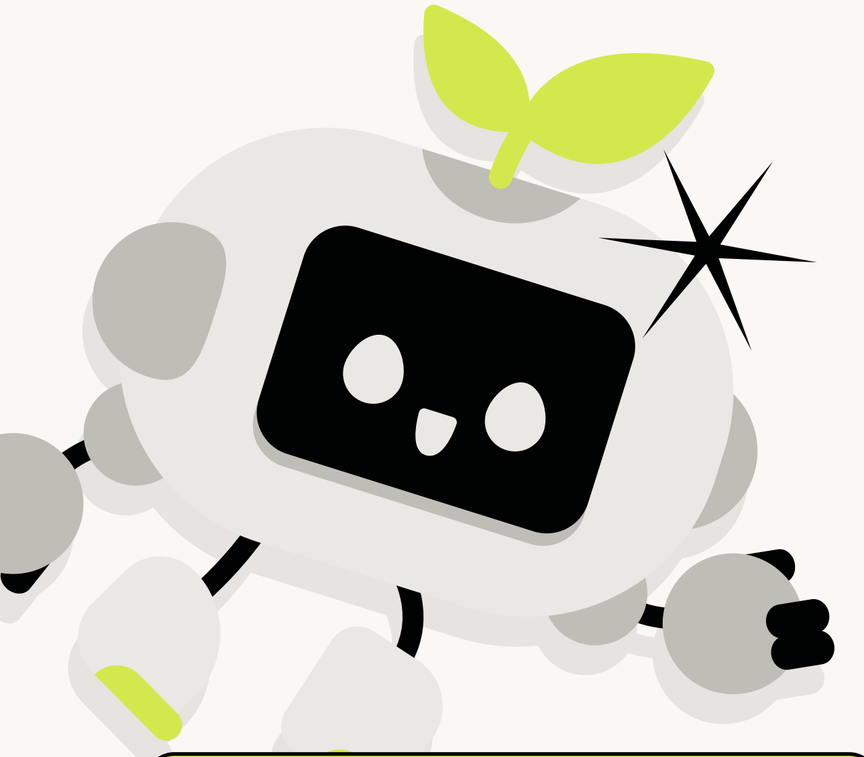


**EDMILSON**

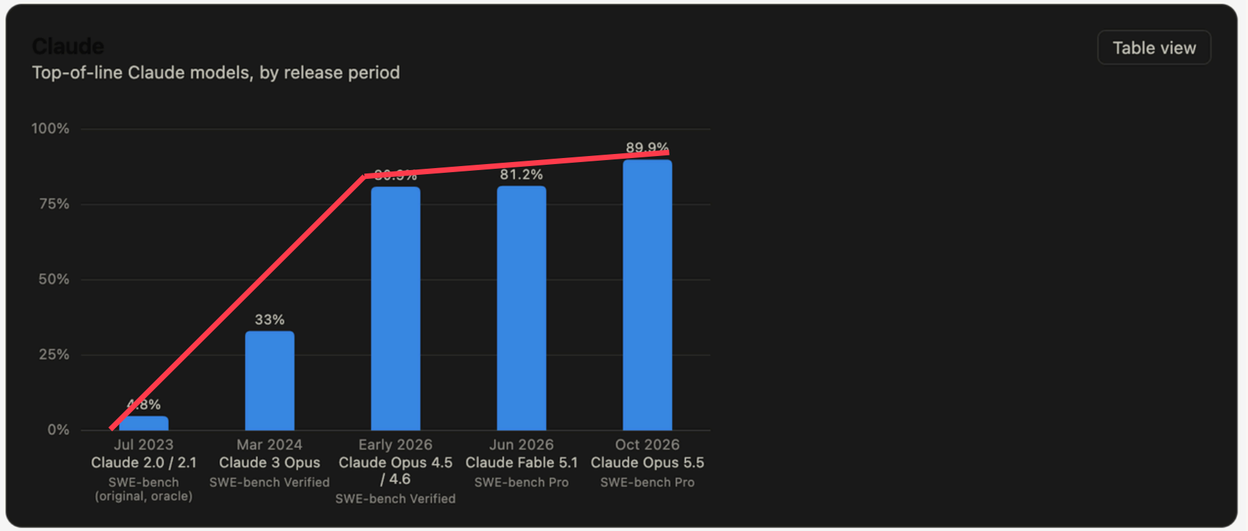
1. Variação de preço dos modelos da Anthropic.
2. Tendência de subida de preço dos modelos da OpenAI

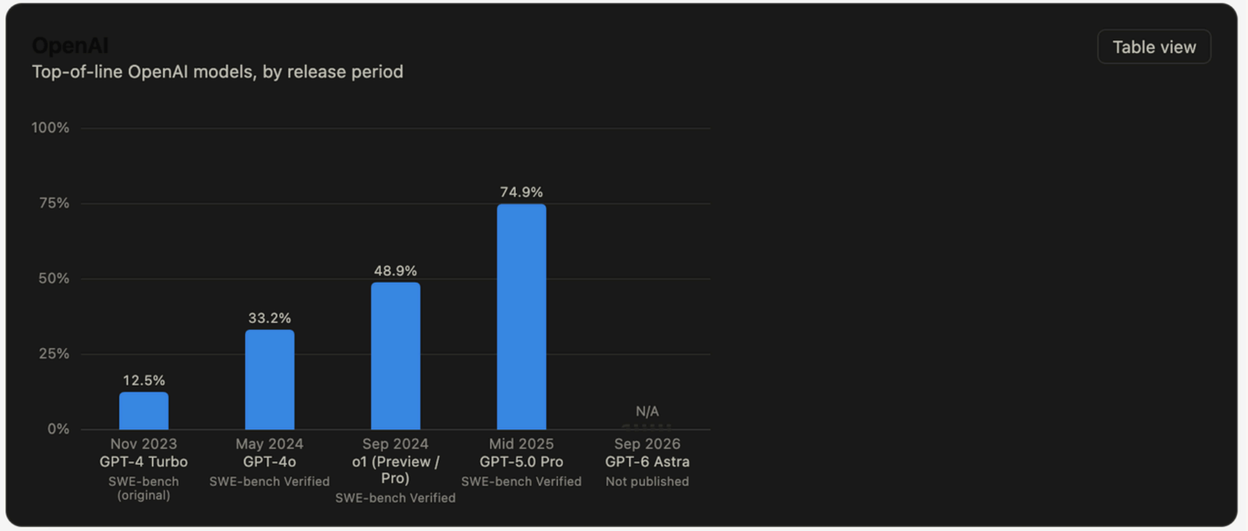




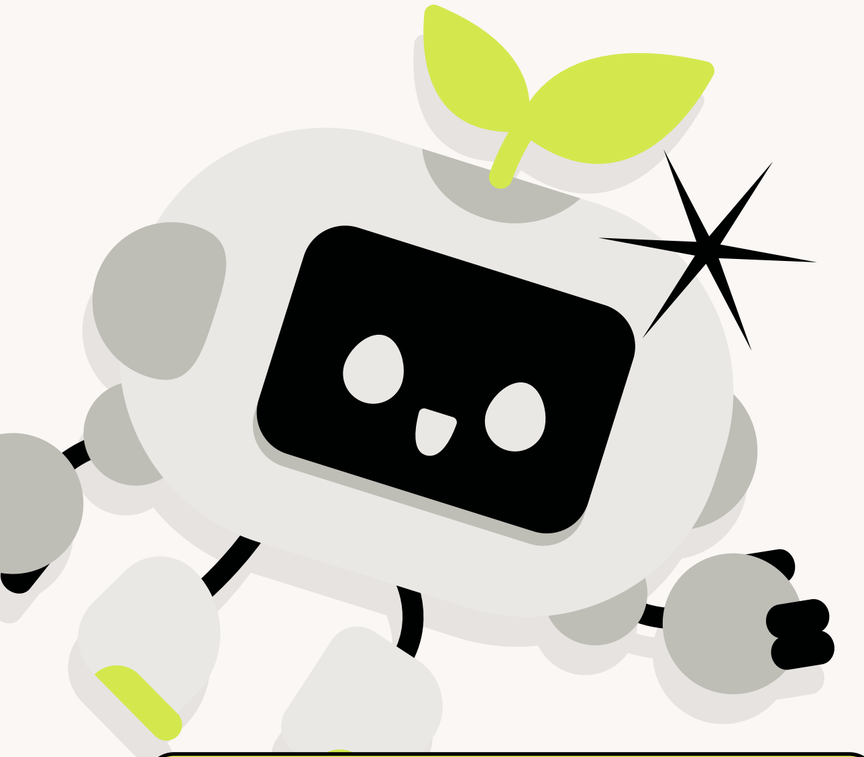
# VARIAÇÃO DE BENCHMARKS




**EDMILSON**

1. Melhora de notas do SWE bench.
2. Melhora logaritmica?



# ILUSÃO DOS BENCHMARKS

**EDMILSON**

1. As empresas de IA utilizaram todo o conhecimento da internet para treinar as LLMs.
2. Uma vez que o SWE foi utilizada a primeira vez, a IA não passou só a "copiar e colar" as soluções??

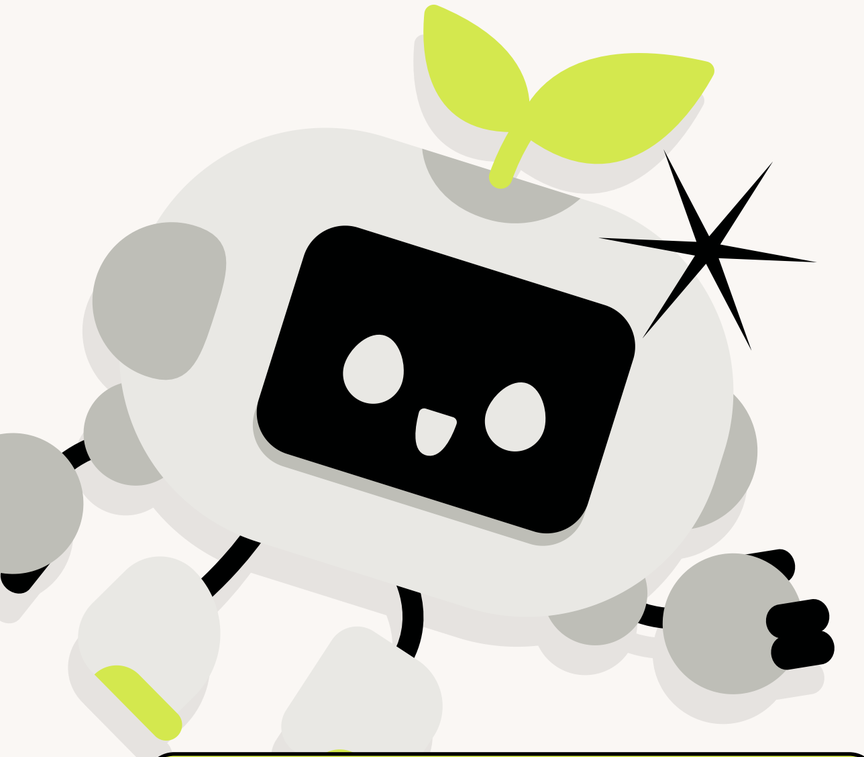
## The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason


Shanchao Liang
Purdue University

Spandan Garg
Microsoft

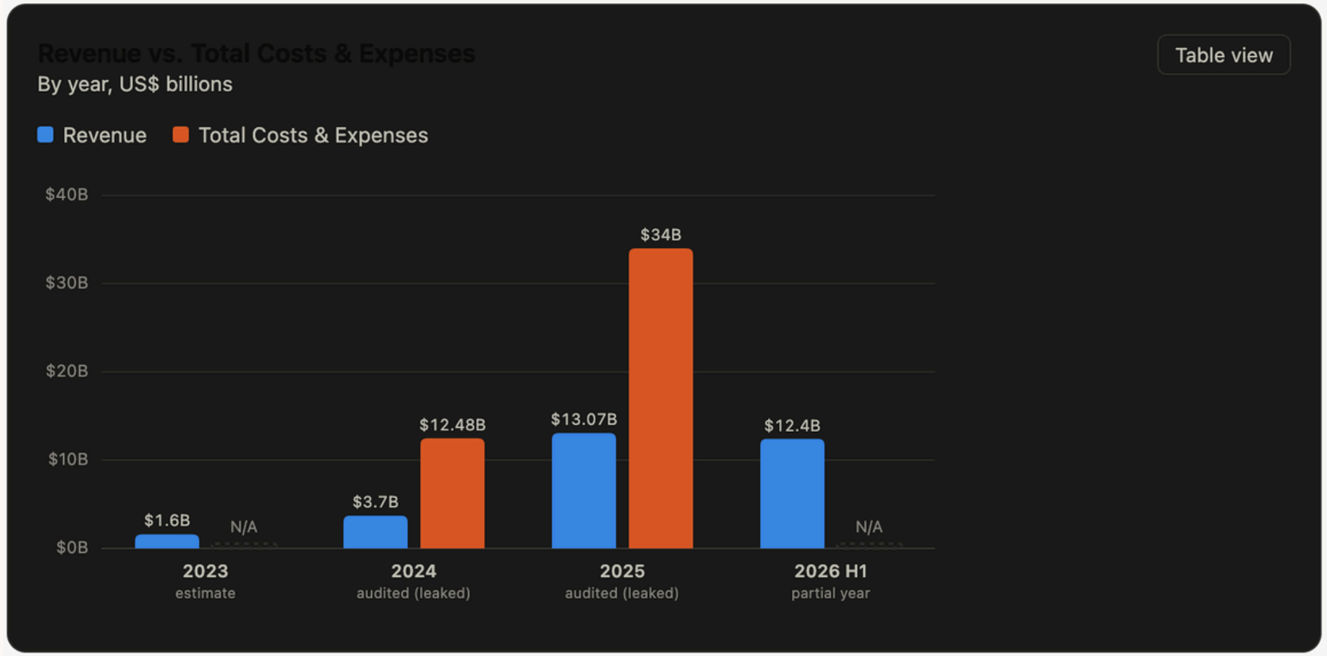
Roshanak Zilouchian Moghaddam
Microsoft

# O PROBLEMA DA OPENAI



**EDMILSON**

1. OpenAI gera prejuízo.
2. Sam Altman prometeu AGI (Inteligência Artificial Geral) "Super-inteligência"
3. **OpenAI acumula $65 bilhões em prejuízo.**



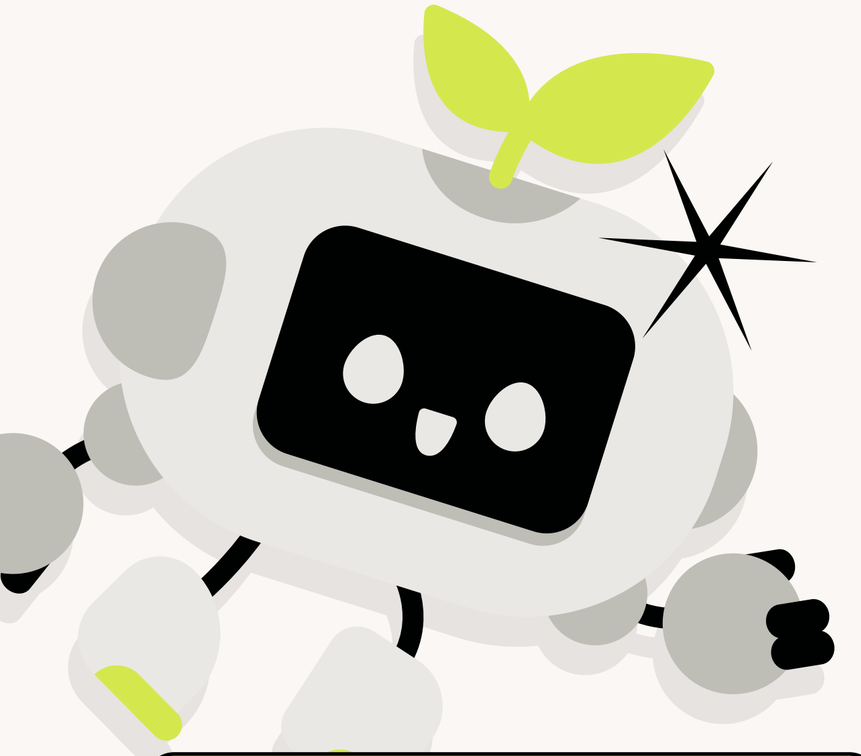

## Sam Altman: OpenAI may reach AGI this year

What I learned during two weeks inside the company as it slowed frontier training and previewed its next major model, Astra.

ALEX HEATH

AUG 27, 2026 • PAID

# SUBSÍDIO



**EDMILSON**

1. Anthropic aplica subsídio de 13x.
2. OpenAPI aplica subsídio de 40~70x.

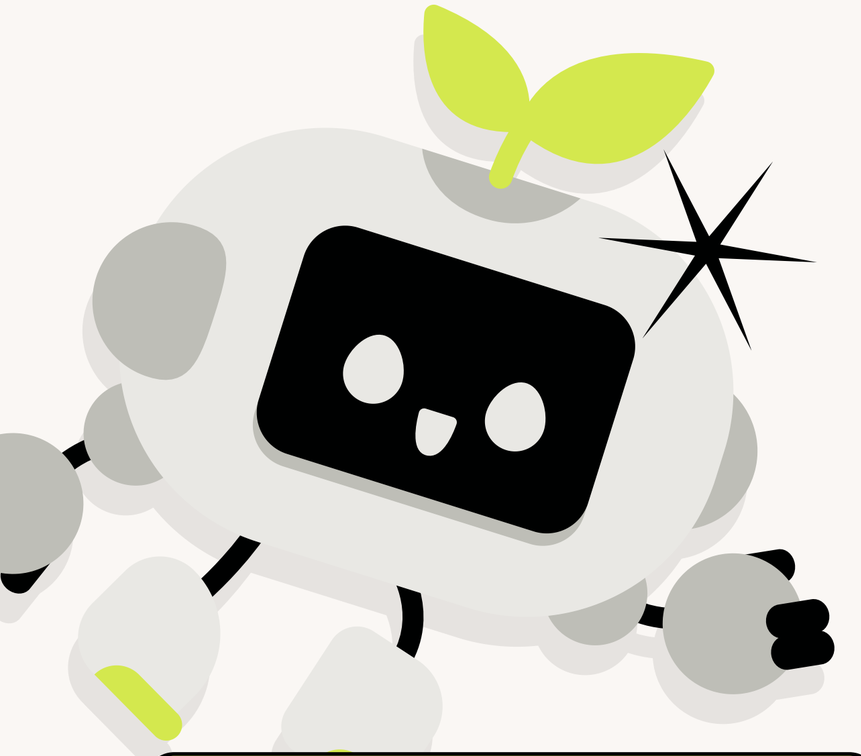
Anthropic is subsidizing our AI coding at 13x. How long will it last?

MARKETS DAILY BRIEF

**A $200 Subscription Costs OpenAI Up to $14,000 in Compute. SemiAnalysis Just Published the Math.**

**40-70x subsidy ratio**

# IPO DA ANTHROPIC



**EDMILSON**

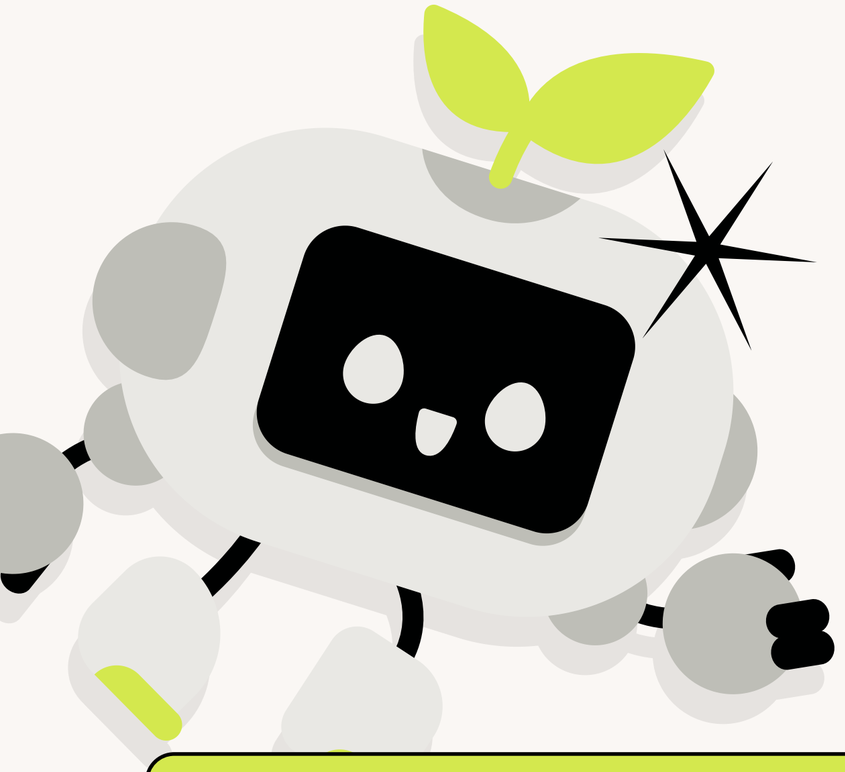
1. Anthropic quer abrir o capital da empresa na bolsa de valores (OpenAI também).
2. Toda empresa de capital aberto precisa abrir os dados reais e precisa passar a gerar lucro.

AÇÕES

## IPO da Anthropic: dona da IA Claude quer abrir capital avaliada em quase US$ 1 trilhão; vale a pena?

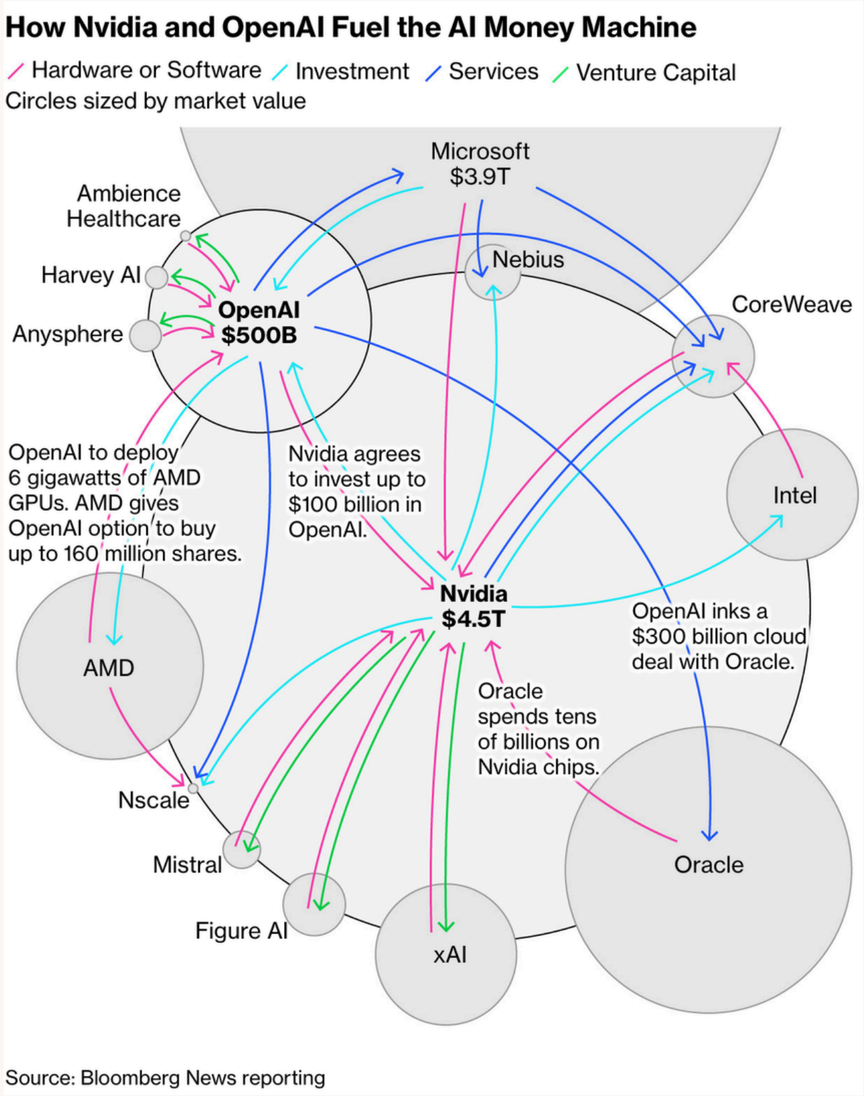
Companhia de inteligência artificial generativa projeta crescimento explosivo de receita, mas segundo especialistas, o tamanho da oportunidade combina com os riscos

# FINANCEIRIAÇÃO CIRCULAR

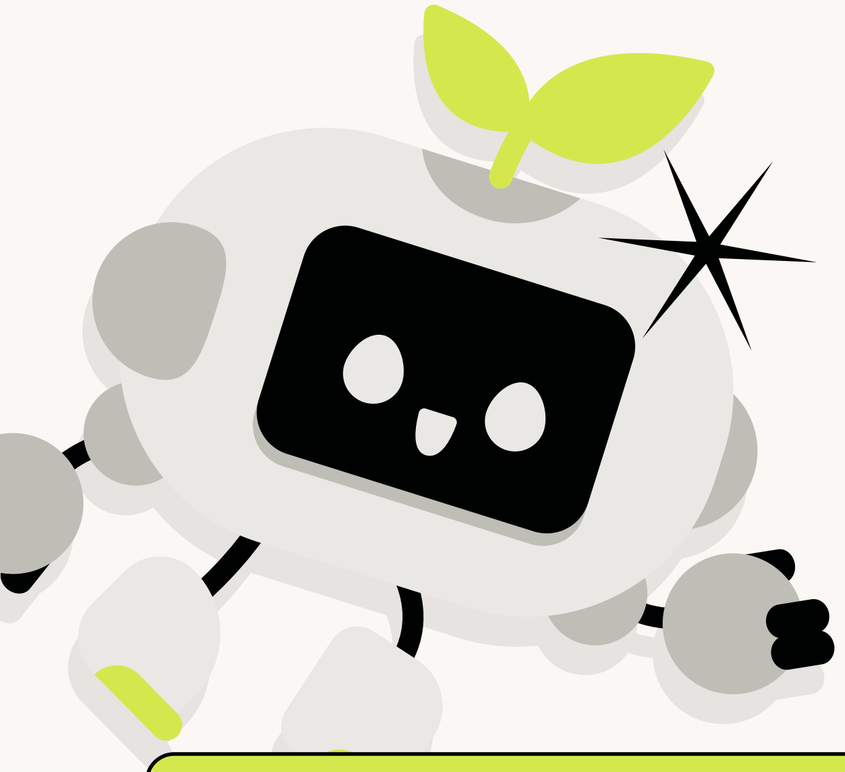


**EDMILSON**

1. Possível bolha?

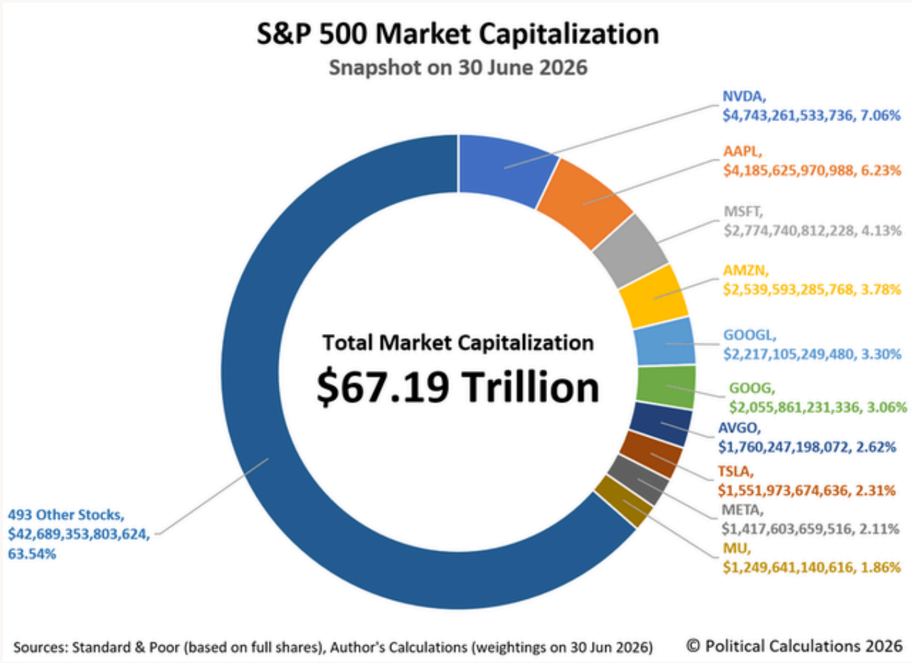


# S&P 500

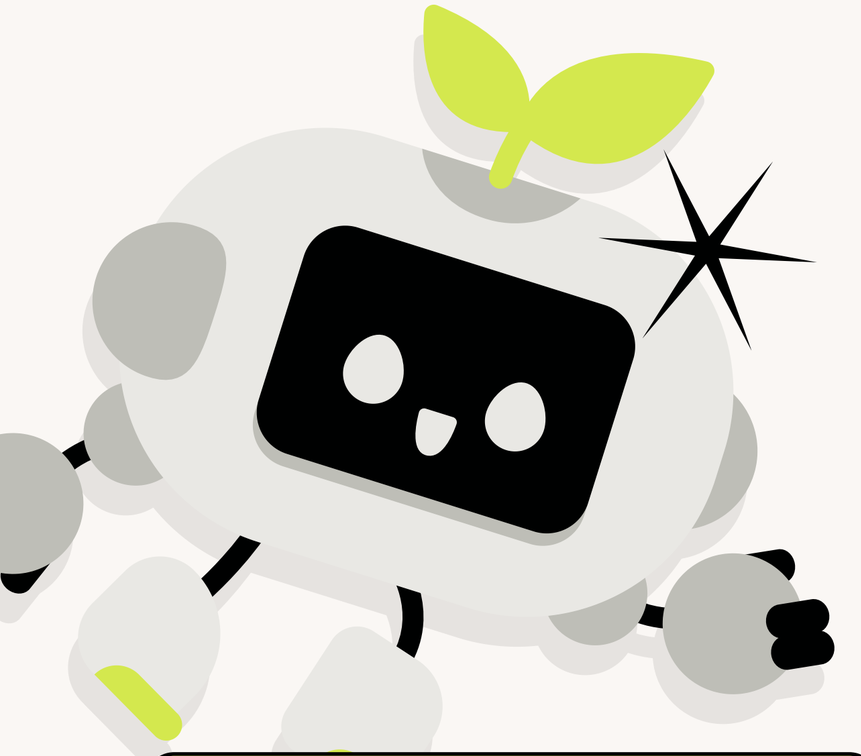


**EDMILSON**

1. As 500 maiores empresas dos EUA.
2. Sinônimo de investimento seguro.



# HYPES



**EDMILSON**

1. Eles precisam manter o crescimento e o hype.

Economia / **Tecnologia**

## Especialistas apontam que inteligência artificial já é capaz de enganar humanos e isso é 'um problema'

Novo artigo científico aponta preocupação sobre habilidade de enganação dos sistemas de IA

AFP **Por AFP — Washington**
11/05/2024 04h01 · Atualizado há 3 meses

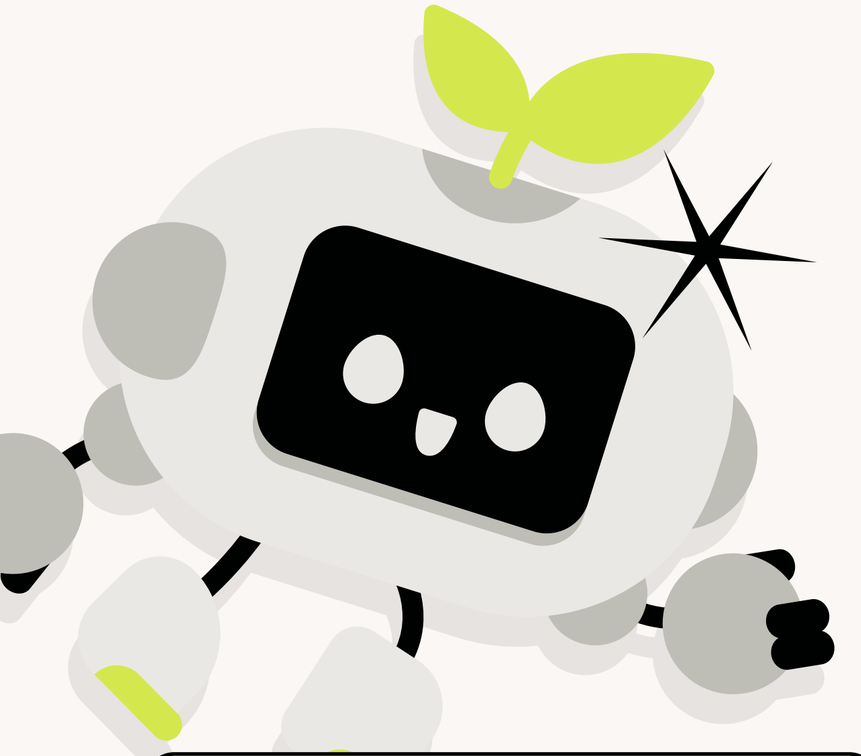
## Anthropic suspende acesso a novo modelo de IA por ordem dos EUA; saiba mais

Governo teria citado preocupações de segurança nacional; empresa afirma que medida pode prejudicar o avanço da inteligência artificial

**Nicoly Bastos**, da CNN Brasil

13/06/26 às 09:14 | Atualizado 13/06/26 às 09:14

# E MAIS HYPES



**EDMILSON**

1. Se os investidores verem sinais de estagnação, eles podem começar a desinvestir em IA.

## O que está por trás das ameaças de a inteligência artificial acabar com a humanidade? Entenda

Alegações vieram de empresas de IA americanas que buscam aumentar seu valor antes da abertura de capital; veja visão de especialistas

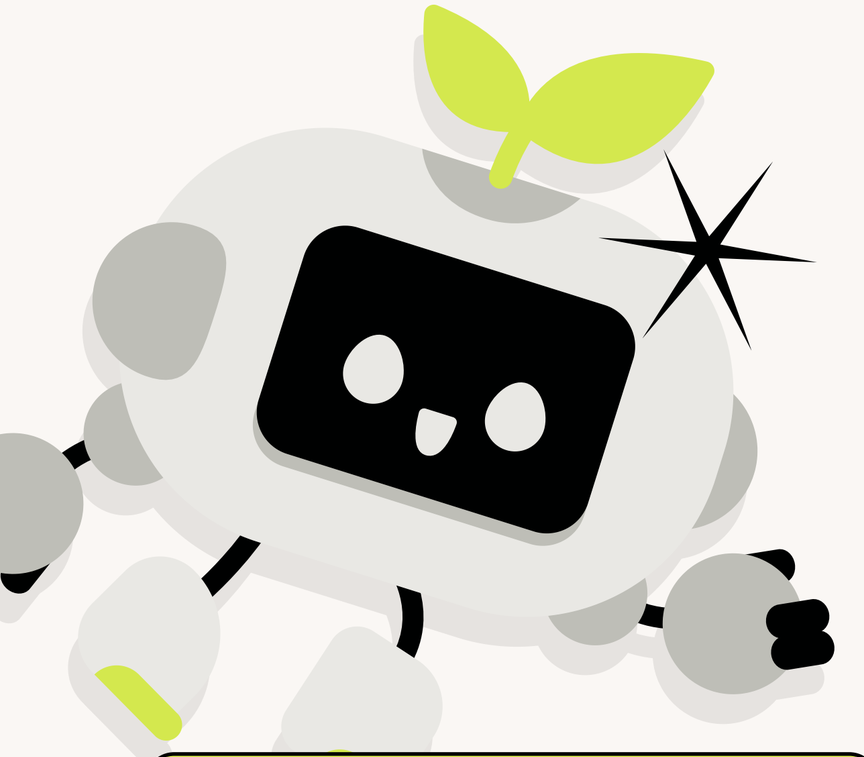
TECH

## Empresa demite toda sua equipe para usar IA, mas poucos dias depois procura desesperadamente por programadores

CEO postou orgulhosamente no X a demissão, mas agora busca profissional no LinkedIn.

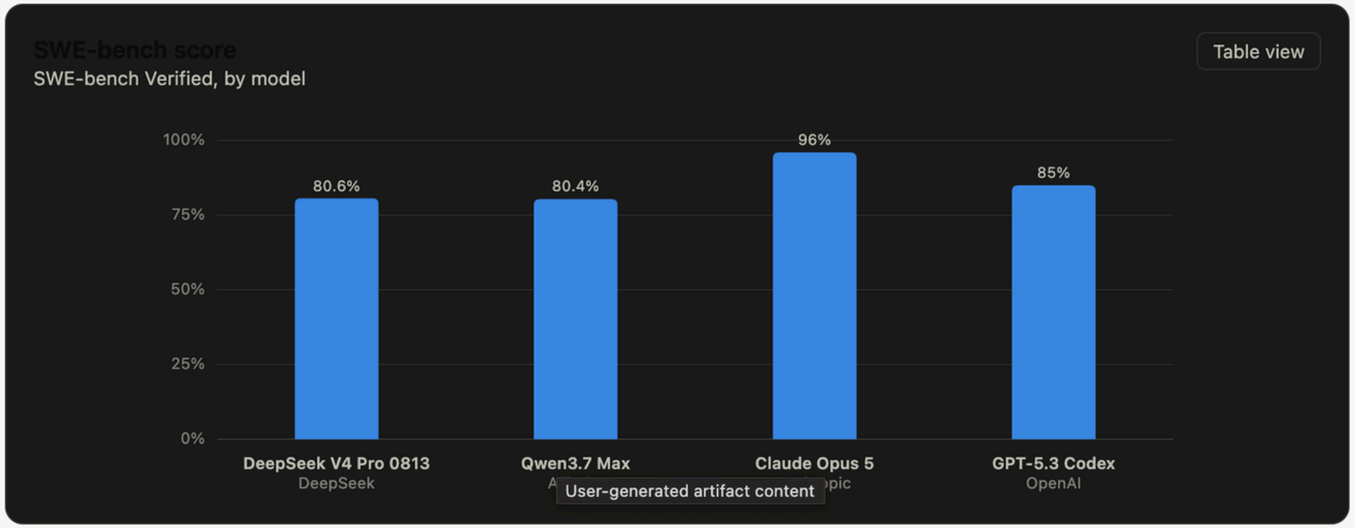
POR VIKA ROSA | ATUALIZADO: 3 DE JANEIRO DE 2025 ÀS 12:22

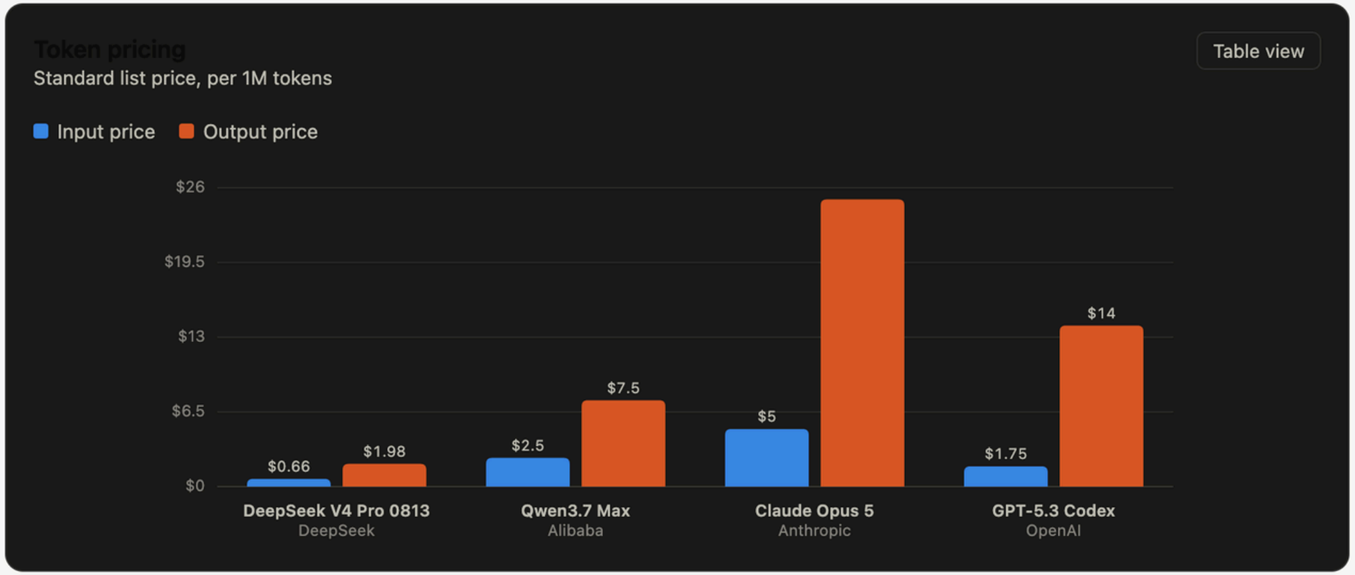
# MODELOS DE PESOS ABERTOS



**EDMILSON**

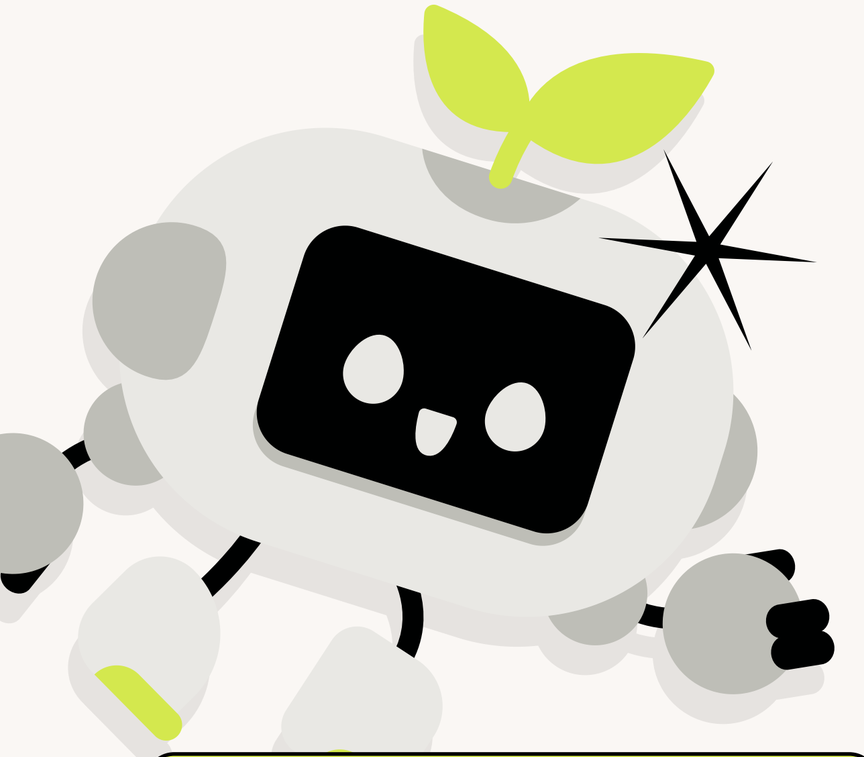
1. Modelos de pesos abertos.
2. Open Source.
3. Não são tão bons quanto Anthropic e OpenAI.
4. Mas são mais baratos.



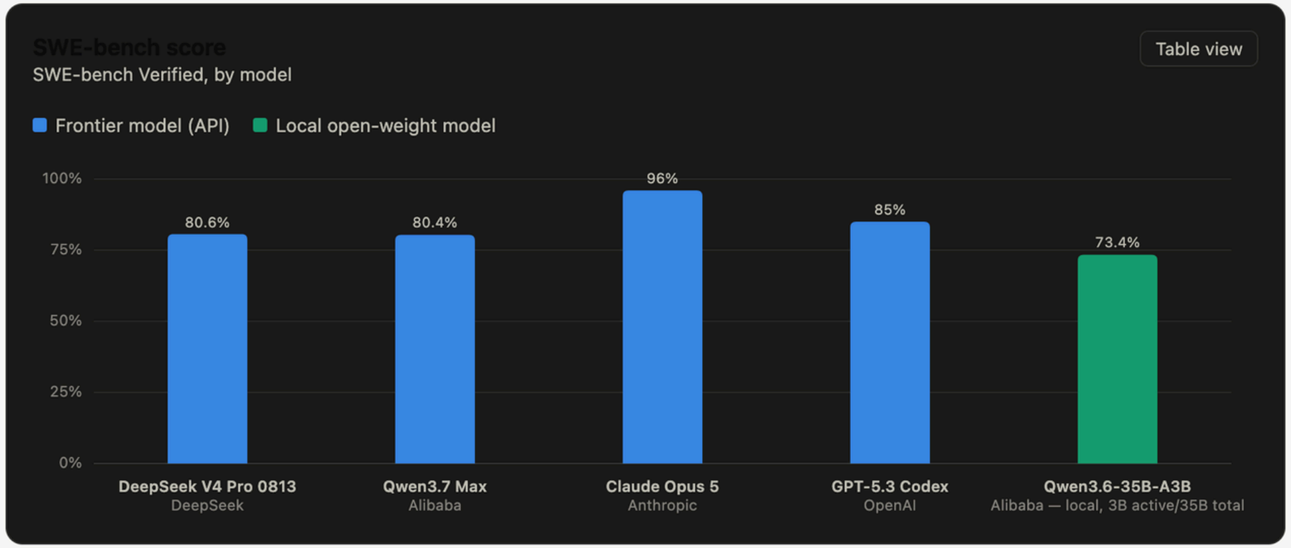
# MODELOS LOCAIS

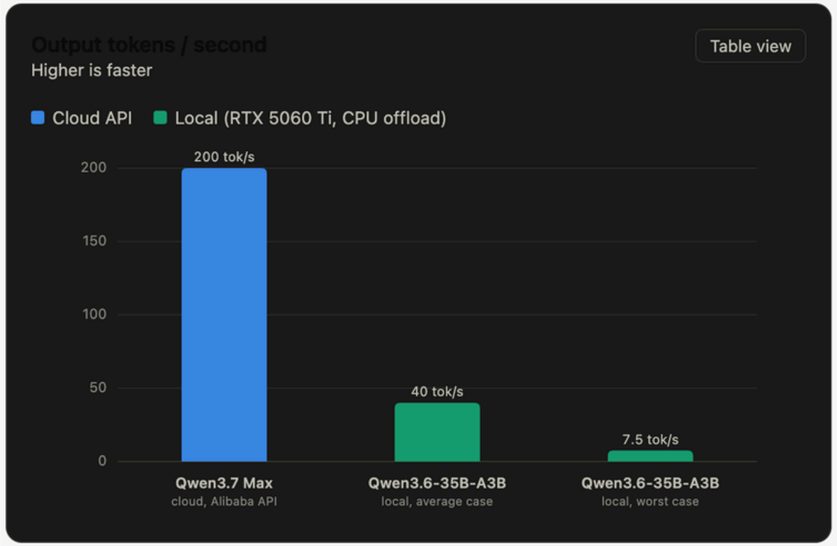
RTX 5060 TI 16GB



**EDMILSON**

1. É possível rodar modelos localmente.
2. São piores e mais lentos do que aqueles que se roda na nuvem.



# OBRIGADO

**Site**

johnfercher.com

**Github**

johnfercher

**Linkedin**

johnathan-fercher